Faculty of Natural and Mathematical Sciences
Department of Informatics

Bush House, King's College
London, Strand Campus, 30
Aldwych, London WC2B 4BG
Telephone 020 7848 2145
Fax 020 7848 2851

7CCSMPRJ

Individual Project Submission 2025

Name: Craig Parker

Student Number: 24061868

Degree Programme: MSc Artificial Intelligence

Project Title: Stochastic SAT Solver using Genetic Algorithms

Supervisor: Professor Hana Chockler

Word count: 12045

RELEASE OF PROJECT

Following the submission of your project, the Department would like to make it publicly available via the library electronic resources. You will retain copyright of the project.

✓ I **agree** to the release of my project

☐ I **do not** agree to the release of my project



Signature:

Date: 17 July 2025

Abstract

This project investigates the development of a Genetic Algorithm (GA)-based solver for the Boolean Satisfiability Problem (SAT), an NP-complete fundamental challenge in computer science with applications ranging from hardware verification to artificial intelligence. The research addresses the limitations of traditional complete solvers (like CDCL) on hard random instances by developing a stochastic approach that combines evolutionary computation with local search techniques.

The core work involved systematic design, implementation, and evaluation of a GA framework specifically adapted for SAT solving. Key components include specialised genetic operators (tournament selection, clause-aware crossover, and guided mutation), weighted fitness functions, and hybrid local search integration. The implementation features a parallel architecture leveraging multi-core processors, with comprehensive testing conducted on both random 3-SAT instances (20-250 variables) and structured graph colouring problems from SATLIB benchmarks.

The principal findings demonstrate that the GA solver achieves 100% success rates on instances up to 100 variables, with median solve times of 218ms for 100-variable random 3-SAT problems. The parallel implementation provides a consistent 2-3× speed-up through 8-thread execution. The study identifies optimal parameter configurations, including a population size of 30, an elitism rate of 10%, and clause-based population initialisation. Unexpectedly, one-point crossover proved more effective than uniform crossover when measured over both random and structured instances.

To Professor Hana Chockler,

for her invaluable guidance, patient mentorship, and insightful feedback
throughout this research journey. Her expertise profoundly
shaped both the technical and academic direction of this work.

To my wife,

for her unwavering support and encouragement during
this challenging yet rewarding academic endeavour.

To the SAT research community,

for maintaining the benchmark libraries and open-source tools
that made this empirical study possible.

— *Craig Parker*
17 July 2025

Stochastic SAT Solver using Genetic Algorithms

Craig Parker

17 July 2025

Contents

1 Introduction **7**
1.1 The Boolean Satisfiability Problem (SAT) . . . 7
1.1.1 Representation: Conjunctive Normal Form (CNF) . . . 7
1.1.2 Importance and Applications . . . 7
1.1.3 SAT Solving Approaches . . . 8
1.2 Introduction to Genetic Algorithms (GAs) . . . 8
1.2.1 Population Representation . . . 8
1.2.2 Why Use GAs for SAT? . . . 8
1.3 Project Objectives . . . 9
1.3.1 Expected Achievements . . . 10
1.4 Report Structure . . . 10

2 Literature Review **10**
2.1 Classical Approaches to SAT Solving . . . 10
2.2 Genetic Algorithm Approach . . . 11
2.2.1 Specialized GA-SAT Architectures . . . 11
2.2.2 Operator Design and Parameter Strategies . . . 12
2.3 Instance-Specific Performance Analysis . . . 12
2.4 Advanced Evolutionary Methods . . . 12
2.5 Unresolved Challenges and Research Gaps . . . 13

3 Requirements and Specification **13**
3.1 Functional Requirements . . . 13
3.1.1 DIMACS Format . . . 13
3.1.2 Expected Output . . . 14
3.2 Genetic Algorithm Components . . . 14
3.3 Hybrid Features . . . 16
3.4 Benchmark Problems . . . 16
3.5 Non-Functional Requirements . . . 17
3.5.1 Technical Constraints . . . 17
3.5.2 Compliance . . . 17
3.5.3 Parameters . . . 17
3.6 Execution Plan . . . 18
3.7 Evaluation Plan . . . 18
3.8 Alternative Approaches . . . 18

4 Design **18**
4.1 Core SAT Algorithm . . . 19
4.2 Genetic Algorithm . . . 20
4.3 Genetic Operators . . . 21
4.3.1 Population Initialisation . . . 21
4.3.2 Fitness . . . 23
4.3.3 Selection . . . 24
4.3.4 Crossover . . . 25
4.3.5 Mutation . . . 26

4.4 Hybrid Methods . . . 27
4.4.1 Local Search . . . 27
4.4.2 Parallelisation . . . 28
4.5 Data Structures . . . 29

5 Implementation **29**
5.1 Project Structure . . . 30
5.2 Interface Methods . . . 31
5.3 Class Diagrams . . . 31
5.4 Demonstration . . . 34
5.5 Execution Plan . . . 35
5.5.1 Uniform Random Instance - 50 variables, 218 clauses . . . 36
5.5.2 Structured Instance (Graph-Colouring) - 90 variables, 300 clauses . . . 38
5.5.3 Population Initialisation . . . 39
5.5.4 Parameter Tuning . . . 41
5.5.5 Local Search . . . 46
5.5.6 Parallel Strategies . . . 47

6 Evalation **49**
6.1 Hard Random Instances - 20 to 250 variables . . . 50
6.1.1 Random 20 . . . 53
6.1.2 Random 50 . . . 54
6.1.3 Random 75 . . . 57
6.1.4 Random 100 . . . 59
6.1.5 Random 125 . . . 60
6.1.6 Random 150 . . . 62
6.1.7 Random 175 . . . 63
6.1.8 Random 200 to Random 250 . . . 65
6.2 Structured Instances - 90 to 300 variables . . . 69
6.2.1 Structured 90 . . . 71
6.2.2 Structured 150 . . . 73
6.2.3 Structured 225 . . . 76
6.2.4 Structured 300 . . . 77
6.3 Comparison . . . 79

7 Legal, Social, Ethical and Professional Issues **80**
7.1 Legal Compliance and Intellectual Property . . . 80
7.2 Social Implications and Security . . . 80
7.3 Ethical Design and Benchmarking . . . 81
7.4 Professional Standards and Sustainability . . . 81

8 Conclusion **81**
8.1 Key Findings . . . 81
8.2 Reflections . . . 82
8.3 Future Work . . . 82

A Outline of Project Structure **85**

B Source Code **86**
B.1 Core Components . . . 86
B.2 Configuration . . . 93
B.3 Factories . . . 97
B.4 Interfaces . . . 99
B.5 Fitness . . . 101
B.6 Local Search . . . 104
B.7 Models . . . 107
B.8 Operators . . . 110
B.8.1 CrossOver . . . 110
B.8.2 Mutation . . . 112

B.8.3 Selection 114
B.9 Population 116
B.10 Utils 121

C Build and Run Instructions **123**

Nomenclature

Symbol	Definition
SAT	Boolean Satisfiability Problem
CNF	Conjunctive Normal Form
GA	Genetic Algorithm
CDCL	Conflict-Driven Clause Learning
DPLL	Davis-Putnam-Logemann-Loveland algorithm
SATLIB	The Satisfiability Library, containing benchmark SAT problems
n	Number of variables in a SAT instance
m	Number of clauses in a SAT instance
x_i	Boolean variable (where $i \in \{1, ..., n\}$)
ℓ	Literal (variable or its negation)
C_j	Clause (disjunction of literals)
ϕ	Boolean formula in CNF: $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$
P	Population size
G	Maximum number of generations
μ	Mutation rate
χ	Crossover rate
ϵ	Elitism rate
$F(\cdot)$	Fitness function
k	Tournament size in selection
T	Tabu tenure (for Tabu Search)
α	Amplification factor (for fitness function)
τ	Temperature (for Simulated Annealing)

Table 1: Mathematical notation and abbreviations used throughout the report

Genetic Algorithm Terminology

- **Individual/Chromosome**: A candidate solution (binary string representing variable assignments)
- **Gene**: Single bit representing one variable's assignment
- **Fitness**: Measure of solution quality (typically clauses satisfied)
- **Generation**: One iteration of the evolutionary cycle
- **Selection**: Process for choosing parents for reproduction
- **Crossover**: Recombination of parent solutions
- **Mutation**: Random modification of offspring

SAT-Specific Terms

- **k-SAT**: SAT problem with exactly k literals per clause
- **Phase Transition**: Critical clause/variable ratio where problems become hardest
- **Backbone Variables**: Variables that take the same value in all satisfying assignments
- **Local Search**: Neighborhood-based improvement heuristic
- **Complete Solver**: Can prove both satisfiability and unsatisfiability
- **Incomplete Solver**: Can find solutions but not prove unsatisfiability

1 Introduction

The Boolean Satisfiability Problem (SAT) is a cornerstone of computational complexity with applications ranging from hardware verification to artificial intelligence. While deterministic solvers like Conflict-Driven Clause Learning (CDCL) dominate the field, stochastic approaches such as Genetic Algorithms (GAs) offer an alternative for navigating the problem's vast search space. This project explores the development of a GA-based SAT solver, investigating how evolutionary strategies can be tailored to efficiently explore feasible solutions. By designing a custom GA SAT solver, this work aims to identify which scenarios work best in achieving optimal performance using both pure GA methods and considering alternative hybrid methods such as local search, offering insights into the adaptability of GAs for constraint satisfaction. [9, 18].

1.1 The Boolean Satisfiability Problem (SAT)

The Boolean Satisfiability Problem (SAT) is a fundamental decision problem in computer science and mathematical logic. Given a Boolean formula composed of logical variables and operators (AND, OR, NOT), SAT asks: *Is there an assignment of truth values (True/False) to the variables that makes the entire formula evaluate to True?* If such an assignment exists, the formula is **satisfiable**; otherwise, it is **unsatisfiable**.

Mathematically, for a Boolean formula ϕ with variables $x_1, x_2, ..., x_n$, we want to determine if: $\exists\ x_1, x_2, ..., x_n \in \{\text{True, False}\}$ such that $\phi(x_1, x_2, ..., x_n) = \text{True}$. If such an assignment exists, ϕ is *satisfiable*; otherwise, it is *unsatisfiable*. One simple, yet complicated formula could be something like

$$\phi = -\left(\{-x_1 \wedge (x_2 \vee -(x_3 \wedge x_4))\} \vee (-(x_2 \vee x_4) \wedge (x_1 \vee -x_3))\right) \wedge ((x_3 \vee -x_2) \wedge (-x_1 \vee (x_4 \wedge -x_2)))\,.$$

It is clear that this can be somewhat hard to read or process, and so in practice SAT problems are typically converted to a conjunctive normal form.

1.1.1 Representation: Conjunctive Normal Form (CNF)

Although SAT can be defined for any Boolean formula, most practical applications use the **Conjunctive Normal Form (CNF)** representation because it provides a standardised structure that simplifies algorithmic processing, allowing algorithms to focus on satisfying every clause, greatly simplifying the problem. Any Boolean formula can be converted to CNF through logical equivalences, flattening nested structures, and applying logical rules such as De Morgan's laws to generate a more structured representation.

A CNF formula consists of Boolean **variables**$(x_1, x_2, ..., x_n)$ that can be True or False, **literals** such as a variable (x) or its negation $(-x)$, **clauses** which are disjunctions (OR, $\vee$) of literals and formula which are a conjunction (AND, $\wedge$) of clauses. For example, a simple CNF formula

$$F = (x_1 \vee -x_2) \wedge (-x_1 \vee x_3) \wedge (x_2 \vee -x_3)$$

has three variables: x_1, x_2, x_3 and three clauses $(x_1 \vee -x_2)$, $(-x_1 \vee x_3)$, and $(-x_2 \vee -x_3)$. To satisfy the formula each clause must be satisfied. If we consider a case where $x_1 = True$, $x_2 = False$, $x_3 = True$ and substitute we can see $(x_1 \vee -x_2) => (True \vee False) => True$, $(-x_1 \vee x_3) => (False \vee True) => True$ $(-x_2 \vee -x_3) => (True \vee False) => True$. Since all clauses are true and satisfied then the whole SAT problem is satisfied with solution (True, False, True).

Quite often, CNF problems can be classified into k-SAT problems where k restricts how many variables we consider in each clause. The most common are 2-SAT, which has been proven to be solvable in polynomial time, and 3-SAT which is NP-Complete. In this project we won't make any assumptions over a specific k-SAT restriction, although most of the testing will be targeting common 3-SAT problems [42].

1.1.2 Importance and Applications

SAT is of immense theoretical and practical significance due to its **NP-Completeness**. SAT was the first problem proven to be **NP-complete** (Cook-Levin Theorem, 1971), meaning it is computationally intractable in the worst case, but efficient solvers exist for many real-world instances. This provides a reference point for analysing the difficulty of problems, and as such thousands of known problems can

be reduced to a SAT problem and are now known to be NP-Complete. Two examples are the travelling salesman and graph colouring [43].

SAT has many applications and is used widely in **Hardware & Software Verification** for model checking, such as verifying chip designs. It plays importance in **Artificial Intelligence** and is used in planning applications, automated reasoning and constraint satisfaction. Many **Optimisation** problems can be reduced to SAT, such as scheduling and routing, as can **Cryptography**, for breaking or analysing cryptographic schemes [44].

The difficulty of solving SAT instances depends on several factors, namely the problem size (number of variables and clauses), clause to variable ratio, problem structure (random or structured instances) and number of backbone variables (variables that must take particular values). Both random and structured hard instances will be analysed in more detail throughout the report. Despite theoretical hardness, SAT solvers routinely handle millions of variables in industrial applications [45].

1.1.3 SAT Solving Approaches

The traditional approach to SAT solving stems from the need to produce a True (Satisfiable) or False (Unsatisfiable) result. These are **complete** methods since they use systematic search, exhaustively exploring the entire search space using various methods such as branching and backtracking. They are guaranteed to either find a satisfying assignment if one exists or prove unsatisfiability if there is no solution. Historically, most research was on complete methods, with DPLL and CDCL being the basis of modern solvers [46].

Incomplete methods however, which are the target of this report, sample the search space heuristically without exhaustive traversal. They can find solutions quickly if they exist but cannot prove unsatisfiability. Incomplete methods usually apply a stochastic approach and can scale well to large problems without an explosion in memory use, also handling problems with a weak structure. They may however get stuck in local optima and be less effective on highly constrained instances such as cryptographic problems. Examples, such as WalkSat will be explained further in the next section [37].

1.2 Introduction to Genetic Algorithms (GAs)

Genetic Algorithms (GAs) are a class of **evolutionary computation** techniques inspired by the principles of **natural selection** and **genetics**. Introduced by John Holland in the 1970s, GAs are heuristic optimisation methods designed to explore large, complex search spaces efficiently [22].

A GA operates on a **population** of candidate solutions (called **individuals** or **chromosomes**), iteratively (as in natural selection) improving them through from one **generation** to the next, replacing weaker individuals and using stronger ones to create new offspring. To achieve this each generation goes through a process of **selection**, where stronger individuals are chosen for reproduction based on a **fitness function**. Pairs of selected parents are then combined through **crossover** whereby genetic information is exchanged to produce offspring. Diversity is added through **mutation** by randomly modifying individuals' genetic information. This process, as shown in Figure 1, continues until a termination condition is met (e.g. convergence, maximum generations, or a satisfactory solution) [18].

1.2.1 Population Representation

Genetic Algorithms have a straightforward representation, which are perfectly suited to represent a SAT solution. Typically, an individual, or a possible GA solution is represented as a binary string, where each bit can take a value of either 1 or 0. For some GA problems multiple bits are used to represent different characteristics of an individual. For a SAT problem the representation is quite natural, with each bit (gene) of an individual (chromosome) representing one specific variable to solve. For example, **011** would represent x1=False, x2 and x3 are both True. This direct mapping makes genetic operations simple to execute and fitness easy to evaluate and has been shown to scale well [23].

1.2.2 Why Use GAs for SAT?

Since SAT is NP-complete, traditional exhaustive search methods (e.g., DPLL, CDCL) can become intractable for large or complex instances. GA's offer several potential advantages for SAT solving, including handling large search spaces effectively. SAT's solution space grows exponentially with the number of variables, yet GA can efficiently explore promising regions without exhaustive enumeration [35].

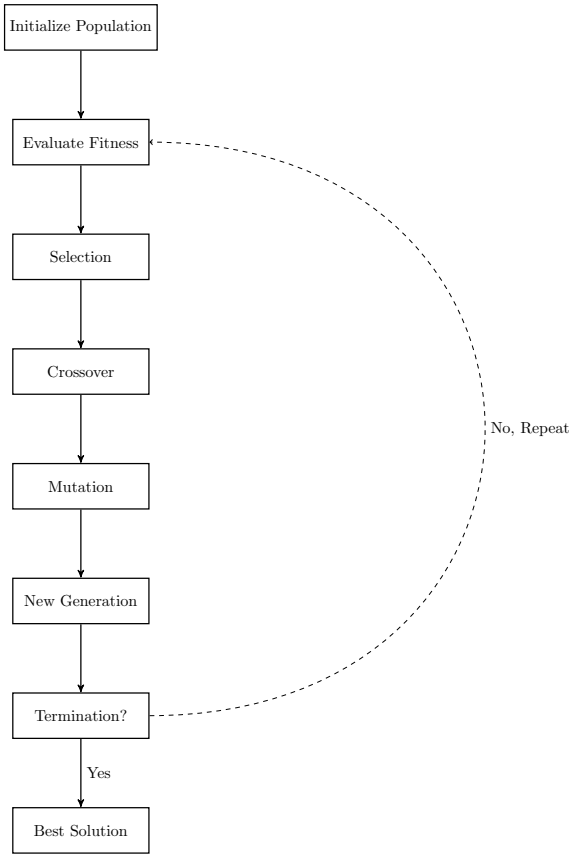


Figure 1: Genetic Algorithm Workflow

Another theoretical key benefit is diversity, which aims to provide robustness against local optima. Representation is ideally suited to the SAT problem, and fitness can easily be established by simple calculations such as number of satisfied clauses. GA's are also inherently parallel, making them suitable for distributed computing. Operations and evaluation of fitness could all be done in parallel, as well as evaluating separate populations independently or even performing inter-population evolution [47].

In practice pure GA, while theoretically suitable, may face difficulties in SAT solving. This can be due to premature convergence, whereby diversity is lost too quickly, leading to suboptimal solutions. However GA has been shown to work well when combined with other hybrid approaches, such as local search or clause learning, exploiting both local and global optimisation. In this study I will start with pure methods, then look at how hybrid approaches can improve performance. This work investigates how GAs can be adapted for SAT, evaluating their effectiveness against classical solvers and identifying problem classes where they excel [40].

1.3 Project Objectives

The primary aim of this research is to design, implement, and evaluate a Genetic Algorithm based SAT solver while exploring the effectiveness of different evolutionary strategies in solving Boolean satisfiability problems. Rather than developing the most competitive SAT solver outright, this project focuses on analysing and comparing different GA **configurations (selection, crossover, mutation)**, identifying which heuristics and **parameters (e.g., fitness function, population size, mutation rate)** perform best on different SAT problem classes and finally investigating hybrid approaches such as combining GA with local search or whether any aspects of clause learning or traditional complete methods can be combined. This work contributes to the broader understanding of how evolutionary computation can be applied to NP-complete problems, particularly in cases where traditional solvers may struggle [48].

1.3.1 Expected Achievements

The proposed GA-based SAT solver will be evaluated on a number of metrics. Firstly, I will assess scalability, seeing how the solver performs as the problem size (number of variables/clauses) increases and whether different approaches are required as the search space increases. Next, I will look at efficiency, measuring runtime and convergence speed on multiple benchmark SAT instances and compare performance against both classical and stochastic solvers [23].

Starting with a pure GA model, I will test and evaluate multiple permutations, combining different implementations of crossover, selection, mutation, and fitness. Following this, I will explore different parameter sensitivity, to help evaluate best options such as population size or mutation rate. Then finally I will provide insights into optimal GA configurations for different SAT problem types [48].

1.4 Report Structure

This dissertation is structured to systematically explore the development and analysis of a Genetic Algorithm (GA)-based SAT solver. Each section builds upon the previous one, ensuring a logical progression from theoretical foundations to empirical validation. The report is organized as follows:

- **Literature Review** - Surveys existing work on SAT solvers, including classical (DPLL, CDCL) and stochastic (local search, GAs) methods such as WalkSat, critically examining prior research on evolutionary approaches to SAT, identifying gaps and opportunities.
- **Specification and Design** - Defines the requirements and architectural design of the GA-based SAT solver, starting with the initial groundwork, then detailing the selection of genetic operators (crossover, mutation), representation schemes, and fitness functions. Also, I discuss hybrid strategies (e.g., GA combined with local search) and parameter choices (e.g., population size) and finally how performance is measured and metrics used.
- **Implementation** - Describes the practical development of the solver, including tools used and optimisation techniques employed. Explains key algorithmic decisions and challenges encountered during coding. Shows testing results and explores best combinations of operators and parameters used to achieve optimal results. Also compares results between different SAT problems such as random or structured. Here I fine tune the solver.
- **Evaluation** - Evaluates experimental results on benchmark SAT instances, comparing different GA configurations over a comprehensive benchmark of size and type. Analyses performance metrics (solution quality, runtime, scalability) against baseline solvers and interprets the findings.
- **Ethical and Professional Issues** - Reflects on ethical considerations in algorithmic research (e.g., bias in benchmark selection). Discusses professional standards in software development and academic integrity.
- **Conclusion and Future Work** - Summarizes key contributions and answers to the research questions. Proposes directions for improving the solver.

2 Literature Review

Approaches to both SAT solving and genetic algorithms have evolved significantly over the years. The more research on both topics the more it is evident that the possibilities of things to try are practically endless. When exploring GA, there are multiple implementations to try for all aspects of population evolution such as crossover or selection strategies, as well as an infinite number of parameter values for settings like population size and mutation rates. The primary focus of this literature review is to understand what specifically has worked well for GA-specific SAT solvers and related hybrids, and to understand what gaps there are in which my research will focus on.

2.1 Classical Approaches to SAT Solving

The foundation of modern SAT solving began by considering complete solutions, those that determine if a SAT problem is satisfiable or not. These methods originated from the landmark DPLL algorithm [11] in the 1960s, establishing a systematic framework for exhaustive search. The algorithm operates through

recursive backtracking, employing unit propagation and pure literal elimination to prune the search space. Subsequently, Conflict-Driven Clause Learning (CDCL) solvers made significant improvements, with the GRASP solver [33] introducing clause learning and non-chronological backtracking. Modern implementations like MiniSAT [15] and Glucose [1] have refined these techniques through sophisticated branching heuristics and lazy data structures, enabling them to handle industrial-scale problems with millions of variables. However, despite these optimisations, the exponential complexity of complete methods persists, particularly for random instances near the phase transition threshold where SAT solving becomes most difficult [5].

Incomplete methods, in contrast, focus on finding a solution to solvable SAT problems, without consideration as to completeness in whether the problem can be solved or not. Early implementations include the GSAT [36] and WalkSAT [37] algorithms. These stochastic local search techniques favor probabilistic hill-climbing strategies, demonstrating efficiency on satisfiable instances by focusing on local improvements to the number of satisfied clauses. WalkSAT's noise parameter mechanism, which balances exploration and exploitation, proved effective in unstructured problems. However, these methods, other than non-ability in detecting unsatisfiability, exhibit some limitations, namely susceptibility to local optima in landscapes with deep valleys, and poor performance on highly constrained instances such as in cryptographic applications [10]. The division between these two approaches has driven research into ways that seek to combine their respective strengths.

2.2 Genetic Algorithm Approach

John Holland's schema theorem [22] provided the theoretical foundation for using population-based search with genetic algorithms, employing direct binary encoding of variable assignments. Mitchell and Forrest's work [35] demonstrated how standard GA operators could navigate SAT's discrete landscape, using fitness functions based on clause satisfaction counts. However, these initial approaches revealed significant challenges, particularly the interactive nature of variables, where the fitness contribution of one variable depends critically on others. This frequently led to premature convergence due to population homogeneity. Subsequent research focused on specialized genetic operators to address these limitations. Eiben and van der Hauw [14] developed adaptive mutation rates that responded to clause satisfaction patterns, while Gottlieb and Raidl [20] investigated permutation-based representations to handle variable dependencies.

Hybrid architectures appear to be the most promising, combining GAs with local search, as shown by the GA-Tabu approach developed by Lü and Hao [32]. This approach leveraged GA's global exploration capabilities while using local search for intensive exploitation, addressing the depth/breadth tradeoff in SAT's irregular fitness landscapes. Their solver combines a standard genetic algorithm with Tabu Search's short-term memory mechanism, where the Tabu component acts as a local intensification operator applied to the best available solutions. This dual-strategy approach demonstrated superior performance on structured SAT instances by maintaining the GA's global diversity while using Tabu's prohibition lists to systematically escape local optima. Experimental results on SATLIB benchmarks showed particular effectiveness on graph coloring and planning problems, where the hybrid solver achieved 15-20% higher success rates than pure GA or Tabu implementations alone.

Several other evolutionary variants have shown promise in specialized SAT domains. The Gasat solver proposed by Fang et al. [16] incorporates a novel constraint-based crossover operator that analyzes clause dependencies during recombination, preserving building blocks of partial assignments. This approach proved particularly effective on crafted instances with a hidden structure, though at increased computational cost per generation. Meanwhile, the Adaptive GA framework of Cai and Su [4] dynamically adjusts mutation rates based on variable participation in unsatisfied clauses, showing 30% faster convergence on industrial benchmarks compared to static parameter configurations.

2.2.1 Specialized GA-SAT Architectures

Recent advances in evolutionary SAT solving have produced several notable frameworks. The SAWEA (Stepwise Adaptation of Weights Evolutionary Algorithm) introduced by Eiben and van der Hauw [13] employs dynamic clause weighting, where unsatisfied clauses gradually increase in weight to guide the search. This approach proved particularly effective on crafted instances, achieving 92% success rates on SAT-encoded graph coloring problems.

FlipGA [17] revolutionized mutation operators by incorporating a stochastic local search-inspired "flip" operation. Rather than standard bit-flips, FlipGA evaluates the fitness impact of flipping each

variable in a candidate solution, implementing the most beneficial mutations. This greedy-localized approach reduced convergence time by 40% on random 3-SAT instances near the phase transition.

The RFEA2 (Repair-based Fitness Evolutionary Algorithm) framework [31] addresses constraint satisfaction through a two-phase process: initial population generation with a repair operator to ensure minimum clause satisfaction, followed by evolutionary optimisation. Experimental results demonstrated superior performance on industrial benchmarks with long, structured clauses.

ASAP (Adaptive SAWEA with Probabilistic selection) [41] extended SAWEA by integrating tournament selection with adaptive probabilities based on clause difficulty metrics. This hybrid showed particular strength on SAT-encoded scheduling problems, outperforming WalkSAT by 25% in solution quality.

2.2.2 Operator Design and Parameter Strategies

Research has identified optimal configurations for evolutionary components. In Selection, tournament selection of a size between 3 and 5 dominates literature due to its selective pressure control [19]. However, scaling issues led to the adoption of elitist-preservation strategies, where the top 10-15% solutions propagate unchanged [12]. For Crossover, uniform crossover (75-90% rate) outperforms single-point for SAT due to schema disruption needs [38]. The Greedy Partition Crossover (GPX) variant [7] shows promise for structured instances by analysing clause satisfaction during recombination. With Mutation, adaptive rates of between 0.5% and 5% per variable based on stagnation detection yield best results [39]. The Clause-Triggered Mutation strategy [28] increases mutation probability for variables in unsatisfied clauses. When considering Fitness, specifically with regard to structured instances, research has shown advanced implementations use weighted clause satisfaction (60%), solution length minimisation (20%), and diversity metrics (20%). Theoretical results suggest maximum clause satisfaction requires exponential population sizes [23].

2.3 Instance-Specific Performance Analysis

When comparing Random vs. Structured Instances, evolutionary approaches exhibit markedly different performance across instance types. Firstly, if we look at random instances, where most research targets 3-SAT, most analysis is done in the phase transition region, where problem instances shift abruptly from being almost always satisfiable to almost always unsatisfiable as constraints increase. This threshold represents the hardest region for SAT solvers and reveals deep connections between computational complexity and statistical physics [34]. We measure the threshold by considering the clause to variable ratio, with the critical ratio known to be around 4.26 [27]. GA algorithm performance declines sharply at this region, with hard problems being solved by pure GA 30% of the time, compared with hybrids like FlipGA achieving closer to 70% success rates. Here diversity preservation becomes critical [40].

If we look at more structured or industrial instances, it is usually the case that algorithms that work well for one type, often perform poorly on others. RFEA2 excels on circuit verification problems (85% success vs. 50% for CDCL) [31]. Clause weighting (SAWEA) proves essential for logistics planning benchmarks [13] and multi-population strategies outperform single-population by 2-3x on SAT-encoded chess problems [6].

2.4 Advanced Evolutionary Methods

The most significant recent advancements have occurred through the integration of evolutionary methods with techniques borrowed from complete solvers. MAXSAT [2] demonstrated how clause weighting strategies from CDCL solvers could guide GA search trajectories. Tomassini and Verel's [40] multi-population approaches showed improved performance on structured instances through spatially structured evolutionary algorithms that maintained subpopulation diversity. These developments have been complemented by theoretical insights into parameter sensitivity [24]. However, these guidelines remain problem-dependent, with industrial benchmarks often requiring different configurations than random k-SAT instances [29].

Recent work has also explored parallel implementations to address scalability limitations. The Co-Evolutionary SAT solver (CESAT) by Chen et al. [8] employs multiple subpopulations with different search strategies - one population uses traditional clause-counting fitness while another evolves weighting schemes for clauses. This division of work demonstrated improved performance on very large instances (>10,000 variables) by allowing simultaneous exploration of solution space and search strategy space.

However, these advanced implementations introduce complex parameter tuning challenges that have limited their widespread adoption [26].

The most successful applications of evolutionary methods have emerged in scenarios where traditional solvers struggle: fuzzy SAT problems with probabilistic constraints [21], dynamic SAT instances where clauses change over time [25], and multi-objective SAT formulations [30]. These developments suggest that evolutionary approaches may find their strongest niche not as replacements for classical solvers, but as specialized tools for non-standard SAT variants where their flexibility provides unique advantages.

2.5 Unresolved Challenges and Research Gaps

Despite these advancements, three critical gaps persist in the literature. First, comparative studies of genetic operators have been largely limited to small-scale or artificial problems, with insufficient analysis of how different crossover mechanisms (e.g., uniform vs. single-point) perform on industrial benchmarks [29]. Second, the interaction between evolutionary parameters and problem structure remains poorly understood, particularly how clause-to-variable ratios affect optimal GA configuration [24]. Third, while hybrid methods show promise, systematic frameworks for combining GAs with modern CDCL techniques like variable activity heuristics or phase saving are notably absent [3].

This study directly addresses these gaps through three novel contributions: a comprehensive empirical comparison of GA configurations across the SAT problem spectrum, a complete development of parameter analysis and tuning responsive to problem characteristics, and innovative hybridisation strategies that integrate evolutionary search with clause learning and parallelisation. The research aims to establish clearer guidelines for evolutionary SAT solving while identifying promising directions for future solver development.

3 Requirements and Specification

The purpose of this study is diverse, create a SAT solver using genetic algorithms, exploring each aspect of the GA life-cycle, determining how each component best integrates with others, testing each combination together to decide the optimal structure. In addition, each unique parameter will be investigated to determine optimality. Starting with a pure GA model in isolation, I will then proceed to include hybrid solutions including local search and concurrency. Tests will encompass a variety of SAT problems including random and structured, and cover levels of difficulty in variable and constraint sizing. While the plan is to optimise and produce the fastest SAT solver, the analysis is to discover an ideal framework and not strictly to compete with industrial benchmarks that have dedicated years of research.

3.1 Functional Requirements

The core SAT solver, a console application that can be run from the command line, will accept a SAT problem in Conjunctive Normal Form (CNF). This will be in the conventional format used in SAT competitions, established by the Centre for Discrete Mathematics and Theoretical Computer Sciences (DIMACS)[49], and ensures compatibility with existing SAT solvers, facilitating good benchmarking. The solver will take as first parameter a path to a single CNF file or folder containing multiple CNF files (hence making it easier to do batch testing).

```
SAT.GA <file.cnf>|<folder>
```

3.1.1 DIMACS Format

The DIMACS format is structured into three main sections, comments (c), a problem line (p) that defines the problem type, number of variables and clauses, and then the specific clauses (ending with a 0). Each clause line lists literals, where the number represents the variable and the sign represents a true or false condition. This is easiest shown with an example.

```
c Example DIMACS file in CNF format with 5 variables and 3 clauses
p cnf 5 3
1 -2 5 0
-1 3 0
2 -3 -5 0
```

This example problem represents

$$(x_1 \vee \neg x_2 \vee x_5) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3 \vee \neg x_5)$$

3.1.2 Expected Output

The output must show either SATISFIABLE or TIMEOUT, depending if the problem was solved in time (configurable). If solved it should show a solution in the DIMACS format. Three metrics will be used to assess the quality of the performance, namely number of generations required, number of restarts needed, and elapsed time. An example output would look like this:

```
SATISFIABLE
1 -2 -3 4 -5 6 -7 -8 -9 -10 -11 -12 13 14 15 -16 17 -18 -19 20
```

See Table 2 for sample output.

Configuration	Value
File Path	`uf20-01.cnf`
Variables	20
Clauses	91
Generation	3
Restarts	0
Best Fitness	1
Elapsed Time	28.859ms
Message	Time taken: 28.859ms

Table 2: SAT Solver Metrics Summary

Since the application will support batch runs, we also must output statistics needed for proper analysis. Include statistics while application runs so user can check progress. See Table 3 for sample output.

Metric	Value
Solved Instances	50/50
Average Time	0.016s
Average Generations	2.50
Average Restarts	0.00
Min Time	0.002s
Max Time	0.029s

Table 3: Aggregate Statistics

3.2 Genetic Algorithm Components

All operations, parameters and permutations are to be tested. Here I provide a detailed description of each method to be tested. A summary is provided in Table 4.

1. A ***population*** must be initialised. Typically, this is a simple random allocation of true/false states to each variable, but I want to explore more efficient ways to do this. The population is formed by repeatedly creating individuals up to the population size configured, which must also be investigated, using the following methods:

   (a) **Random allocation** – each variable will be randomly assigned either a true or false value with 50% probability.

   (b) **Clause-based allocation** – each variable is randomly assigned based on their weighting of false to true ratio over all clauses. For instance, if a variable is featured 3 times as false and 7 times as true, we randomly assign that variable with a 70% chance to be true.

   (c) **Diversity-based allocation** – New individuals are created that are most diverse from the existing population.

2. The population now must be measured for ***fitness***. Our goal is to find an individual which satisfies all SAT clauses, at which point we can terminate and produce a satisfiable solution. SAT problems fit perfectly into this GA framework as fitness can be directly measured as to how well an individual has solved the problem. The following methods will be implemented:

   (a) **Max-SAT** – the simplest measure, aggregating how many clauses have been satisfied out of all possible clauses. Hence, if 40 out of 50 clauses are true with assigned variables, then our fitness function would return 0.8.

   (b) **Weighted-SAT** – a more sophisticated measure, determined by assigning higher weights to more difficult-to-achieve clauses, while reducing weights of easy ones. The fitness function aggregates the sum of weights.

   (c) **Amplification Fitness** – Similar to Max-SAT, except it amplifies stronger results, so they are more likely to be selected, using an amplification factor. E.g. For fitness functions giving a standard result of 80% and 90% with an amplification factor of 2, the amplified result would be $0.8^2 = 0.64$ and $0.9^2 = 0.81$, giving the latter a higher chance to be selected.

3. ***Selection*** is the process of choosing parent solutions from a population to produce offspring, favouring individuals with a higher fitness, yet balancing exploration (diversity) and exploitation (elite solutions). Methods to be tested are quite standard in GA literature, namely:

   (a) **Tournament** – Randomly pick k (configurable) individuals, pick the fittest. Ensures some diversity but favours stronger candidates.

   (b) **Roulette Wheel** – All individuals of the population are stacked with the chance to be selected proportionate to their fitness function.

   (c) **Rank-based** – The population is first ranked in descending order of their fitness function. Individuals are then selected randomly in proportion to their ranking, with higher ranked individuals more likely to be selected. Fitness values are no longer considered. So, if 2 individuals have a close fitness function, the higher one will have significantly more chance to be selected than based on pure fitness alone. On the other hand, it also reduces the dominance of super-fit individuals.

4. Evolution of the next generation now begins with ***crossover***, where we combine genetic material (individual variable values true/false) from two parent solutions to create offspring, promoting exploration of the solution space. Here we will experiment with:

   (a) **N-Point** – multiple point crossover where we split each parent at n random points, swapping segments of values at those points. Most commonly one- and two-point crossover are considered but this will be configured to support any. A simple one-point crossover example would be:
   Parent 1: 110—010 + Parent 2: 101—100 → Child 1: 110100 + Child 2: 101010.

   (b) **Uniform** – Each gene (bit/variable) is chosen randomly from either parent. For each position, randomly pick a parent to inherit from with equal chance.

   (c) **Clause Satisfaction** – A specialized crossover operator prioritizes preserving and combining variable assignments that satisfy clauses in parent solutions. After identifying which clauses are satisfied by each parent and which variables satisfy them, a higher weighted chance must be put on variables of one parent that satisfy more clauses than the other parent.

5. The next step is ***mutation***, which seeks to introduce small, random changes into the population to maintain diversity and escape local optima. Key mechanisms explored are:

   (a) **Bit-Flip Mutation** – the most common technique, randomly flip one bit of an individual with the probability equivalent to the mutation rate.

   (b) **N-Bit Mutation** – Mutate a fixed percentage of the population based on mutation rate. Typically, only 1 bit gets mutated but we will experiment with multiple.

   (c) **Guided Mutation** – Find variables in clauses that, if flipped, result in an increase in satisfaction. Randomly choose one of these variables.

6. Lastly, we must ***terminate*** after either a solution is found or after a configured number of generations.

Selection	Crossover	Mutation	Fitness	Population
Rank	N-Point	Bit-Flip	Max-SAT	Random
Roulette	Uniform	N-Bit	Weighted	Clause
Tournament	Clause	Guided	Amplified	Diversity

Table 4: Genetic Algorithm Components

3.3 Hybrid Features

From research, it is likely that a pure GA implementation is not enough for an optimal solution, so we will also explore local search integration and parallelisation. Table 3.3 summarises the features.

1. ***Local Search:*** I will explore how refining solutions by making small, incremental changes to improve clause satisfaction can, alongside the pure GA model, improve SAT optimality. Local search strategies for selecting which individuals are to be mutated will be investigated to avoid impacting runtime.
    - **Hill Climbing** – a simple greedy mechanism that flips one variable at a time, keeping changes that improve fitness.
    - **Tabu Search** – more flexible, avoiding recent moves to explore new areas and escape local optima. Uses a tabu list for temporarily banned moves and may accept worse moves.
    - **Simulated Annealing** – uses a probabilistic acceptance of worse moves, with a temperature schedule that allows broad exploration early and becomes greedier over time. Excels at global exploration in early stages.
2. ***Parallelisation:*** I will pursue parallelisation within each generation and measure performance gains. Populations are independent of each other.

Local Search	Parallelisation
Hill Climbing	Independent Populations
Tabu Search	
Simulated Annealing	

Table 5: Hybrid Features

3.4 Benchmark Problems

Kindly provided by The University of British Columbia[1], a number of benchmark problems will be tested to provide coverage of both random and structured instances. All instances are satisfiable. For each instance, I will develop and test based on the first 100 problems and evaluate on all instances.

1. **Uniform Random 3-SAT** – hard random problems, targeting the phase transition region (4.3 clauses per variable):
    - `uf20-91`: 20 variables, 91 clauses
    - `uf50-218`: 50 variables, 218 clauses
    - `uf75-325`: 75 variables, 325 clauses
    - `uf100-430`: 100 variables, 430 clauses
    - `uf125-538`: 125 variables, 538 clauses
    - `uf150-645`: 150 variables, 645 clauses
    - `uf175-753`: 175 variables, 753 clauses
    - `uf200-860`: 200 variables, 860 clauses
    - `uf225-960`: 225 variables, 960 clauses

[1] See: https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html

- `uf250-1065`: 250 variables, 1065 clauses

2. **Flat Graph Colouring** – structured problems, all 3-colourable:
   - `flat30-60`: 30 vertices, 60 edges (90 variables, 300 clauses)
   - `flat50-115`: 50 vertices, 115 edges (150 variables, 545 clauses)
   - `flat75-180`: 75 vertices, 180 edges (225 variables, 840 clauses)
   - `flat100-239`: 100 vertices, 239 edges (300 variables, 1117 clauses)

3.5 Non-Functional Requirements

3.5.1 Technical Constraints

Programming language used will be C#, limited to standard libraries. This has a nice balance between readability and performance, allowing the focus to be on functionality without losing too much in optimal performance. Application will be cross-platform and will be supportive of both Windows and Linux. Testing will be performed on a Dell Windows Laptop, x64 based processor, 32GB RAM and Intel Core i7-6700HQ @2.60GHz processor.

3.5.2 Compliance

To ensure reproducibility, seed control should be offered so random number generation can be consistently re-run to achieve same results, and can ensure consistent and fair testing over different methods. Solutions will be validated and checked with external checkers to ensure correctness. Benchmarks used will include all metrics, generation count, restart count and elapsed time.

3.5.3 Parameters

Command-line interface with configurable parameters. See Table 6 for details:

```
SAT.GA <file.cnf>|<folder> [-optional flag] [optional value]
```

Option	Argument	Description	Example
-p, –population	`<int>`	Population size	100
-g, –generations	`<int>`	Number of generations	500
-m, –mutation-rate	`<double>`	Mutation probability	0.01
-c, –crossover-rate	`<double>`	Crossover probability	0.9
-e, –elitism-rate	`<double>`	Elitism rate	0.1
-s, –seed	`<int>`	Random seed	null
-t, –tournament	`<int>`	Tournament size	3
–tabu	`<int>`	Tabu tenure	5
-a, –amplification	`<double>`	Amplification factor	2.0
-l, –local-search	`<bool>`	Enable local search	true
-fc, –file-count	`<int>`	Max files processed (folder mode)	1000
-tc, –thread-count	`<int>`	Number of parallel threads	1
–mutation-bits	`<int>`	Number of bits to mutate	1
–local-search-method	`<string>`	Local search method	”Tabu”
–selection	`<string>`	Selection operator	”Tournament”
–crossover	`<string>`	Crossover operator	”Uniform”
–mutation	`<string>`	Mutation operator	”Guided”
–fitness	`<string>`	Fitness function	”Amplified”
–generator	`<string>`	Population generator	”Clause”
-h, –help	-	Show help message	-

Table 6: Configuration Options

3.6 Execution Plan

To systematically evaluate the effectiveness of various genetic algorithm (GA) configurations and hybrid approaches, the following experimental plan will be followed. This ensures thorough testing across both random and structured benchmark problems, as well as exploration of enhancements such as local search and parallelisation.

1. For the benchmark `uf50-218`, test all permutations of pure GA operations, identifying the most promising configurations for random instances.
2. For the benchmark `flat50-115`, test all permutations of pure GA operations, identifying the most promising configurations for graph colouring (structured) instances.
3. Compare different population initialisation methods on random instances with 100 variables, using the best options found in previous steps
4. Parameter tuning for restart rate, population size, elitism rate and tournament size, using best options found in previous steps
5. For the implemented local search methods, find which one works best in combination with the best GA setup found so far.
6. Compare parallelisation with different thread count for performance improvements.

3.7 Evaluation Plan

Following the execution plan used to determine our best GA SAT solver, we will evaluate this on all the benchmarks described above. Evaluation will happen for our best algorithm using both parallel and non-parallel implementations, and compare against industry benchmarks of MiniSAT and WalkSAT.

1. Generate metrics for all random instances (ranging from 25 to 250 variables).
2. Generate metrics for all structured instances (ranging from 90 to 300 variables).

3.8 Alternative Approaches

In this study it is impossible to consider every possible permutation and possibility. Some notable operations I had considered but haven't included are seen in Table 7. During development I did considerable experimentation and tried alternative approaches such as switching crossover or mutation strategies over different generations bad had limited success.

Component	**Alternative Ideas Considered**
Crossover	More than 2 parents, More than 2 children
Fitness Functions	Adaptive fitness, variable conflict penalty, penalty-based fitness, total literals satisfied
Population	Growing or shrinking population
Parameter Tuning	Dynamic parameters, adjusting per problem size
Parallelisation	Cross-Population between threads, Different strategy per thread
Local Search	WalkSAT, GSAT, Variable Neighborhood Search
Other Hybrid Methods	Techniques from DPLL/CDCL solvers, Reinforcement Learning

Table 7: Alternative techniques considered for different SAT solver components

4 Design

In this section, I expand on the requirements and specifications, providing details in the form of flow diagrams and pseudocode. Starting with high-level requirements, I will then focus on individual components and then on specific implementations.

4.1 Core SAT Algorithm

The core algorithm, a GA-based SAT solver is specified in section 3.1. The following diagram (Figure 2) demonstrates the program flow. The SAT.GA application accepts a folder or file path of CNF files in DIMACS syntax. These are each parsed into a SAT problem and run through the genetic algorithm, outputting either a satisfiable solution or times out after a configured number of generations. Also, see Algorithm 1.

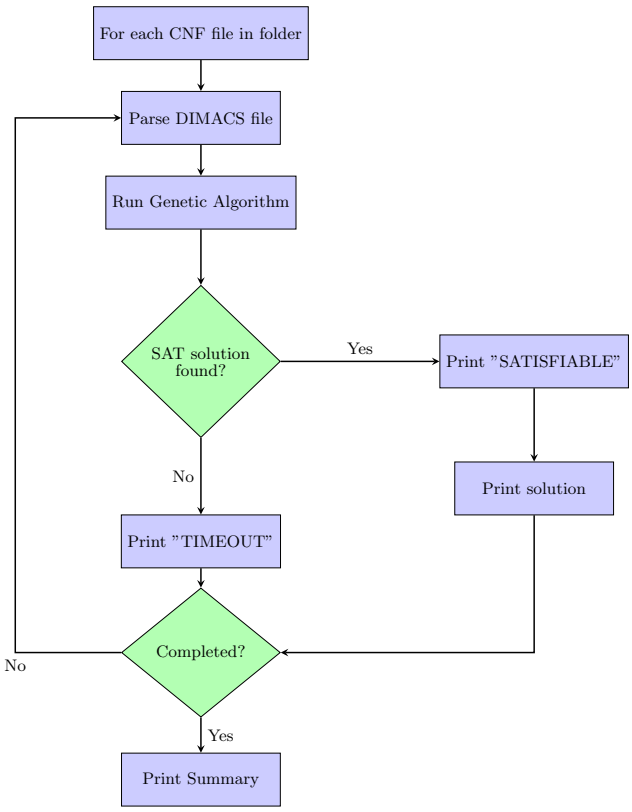


Figure 2: Core SAT Workflow

Algorithm 1 Core SAT Solver Program

INPUT: Command line arguments `args`
OUTPUT: Best SAT solution and performance metrics
$(config, files) \leftarrow$ GenerateConfigFromArgs($args$)
Initialize: $total \leftarrow 0$, $metrics \leftarrow []$, $solutions \leftarrow []$
for each `cnfFile` in `CNF files` **do**
  $satProblem \leftarrow$ ParseDimacs(`cnfFile`)
  $solution \leftarrow$ GeneticAlgorithm(`satProblem`, `config`)
  $(metrics, solutions).append(solution.summary)$
  $total \leftarrow total + 1$
end for
for each index $i = 1$ to N **do**
  let metric $\leftarrow$ metrics[i]
  let solution $\leftarrow$ solutions[i]
  if solution.IsSolved **then**
    PRINT("SATISFIABLE")
    PRINT(solution)
  else
    PRINT("TIMEOUT")
  end if
  PRINT(metric)
end for

4.2 Genetic Algorithm

Introduced in the Introduction section, and detailed in the requirements, we revisit the GA workflow in Figure 3 and Algorithm 2. Inscribed in asterisks (*) are features we will explore after implementing a pure GA solution.

Algorithm 2 Genetic Algorithm for SAT Solver with Local Search* and Elitism*

Require: SAT instance, population size N, max generations G, mutation rate m, elitism rate e
Ensure: Best found satisfying assignment or best approximation
$P \leftarrow$ InitializePopulation(N)
$S^* \leftarrow$ BestIndividual(P)
for generation = 1 **to** G **do**
  $E \leftarrow$ SelectTopIndividuals($P, \lceil e \times N \rceil$) {Elites to carry over}
  $P_{new} \leftarrow E$
  while $|P_{new}| < N$ **do**
    $(parent_1, parent_2) \leftarrow$ SelectParents(P)
    $(child_1, child_2) \leftarrow$ Crossover($parent_1, parent_2$)
    $(child_1, child_2) \leftarrow$ Mutate($child_1, child_2, m$)
    $(child_1, child_2) \leftarrow$ LocalSearch($child_1, child_2$) {If Enabled}
    Add $child_1, child_2$ to P_{new}
  end while
  $P \leftarrow P_{new}$
  $best_current \leftarrow$ BestIndividual(P)
  if Fitness($best_current$) > Fitness(S^*) **then**
    $S^* \leftarrow best_current$
  end if
  if IsSatisfying(S^*) **then**
    return S^*
  end if
end for
return S^*

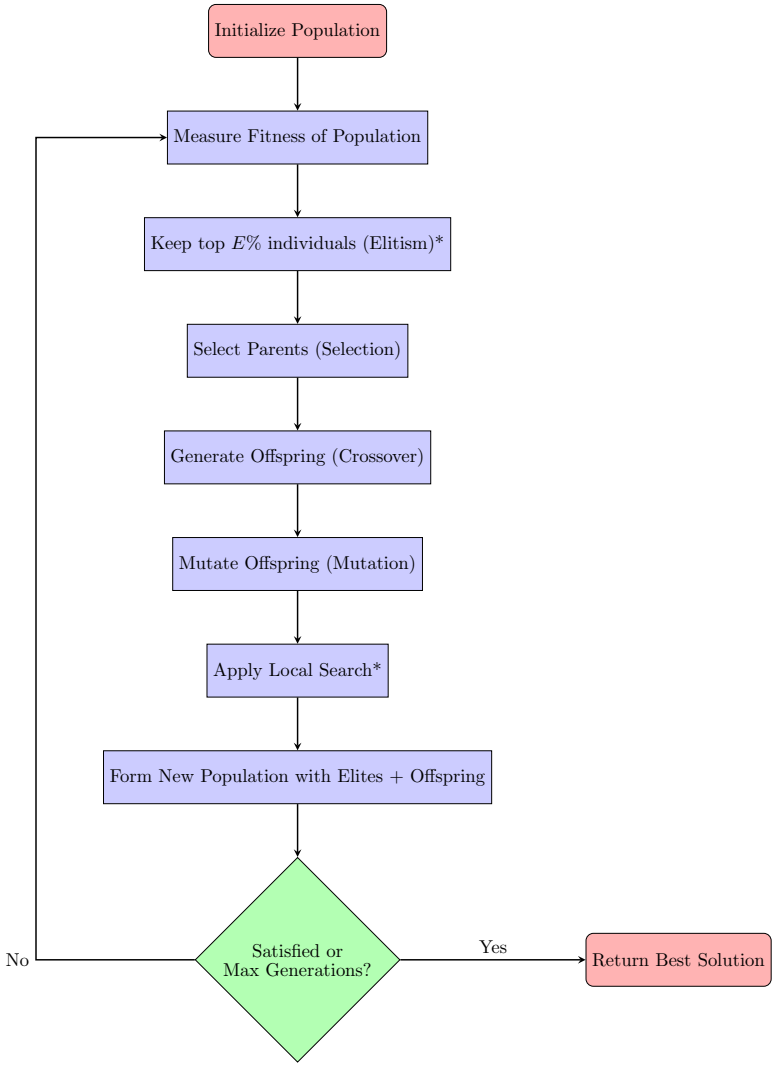


Figure 3: Genetic Algorithm Workflow

4.3 Genetic Operators

Specified in section 3.2, we now provide pseudocode to clearly define the algorithms required.

4.3.1 Population Initialisation

Algorithm 3 Random Population Initialisation

Require: Number of individuals P, number of variables n
Ensure: Initialized population $\mathcal{P}$ of size P
  $\mathcal{P} \leftarrow [\,]$
  for each individual index $i = 1$ to P **do**
    $x \leftarrow [\,]$
    for each variable index $j = 1$ to n **do**
      if `random()` < 0.5 **then**
        $x_j \leftarrow$ true
      else
        $x_j \leftarrow$ false
      end if
    end for
    $\mathcal{P}$.append(x)
  end for
  return $\mathcal{P}$

Algorithm 4 Clause-based Population Initialisation

Require: Number of individuals P, number of variables n, set of clauses C
Ensure: Initialized population $\mathcal{P}$ of size P
  Initialize arrays positive[1..n] $\leftarrow 0$, negative[1..n] $\leftarrow 0$
  for each clause $c \in C$ **do**
    for each literal $l \in c$ **do**
      if $l > 0$ **then**
        positive[l] $\leftarrow$ positive[l] $+ 1$
      else
        negative[$|l|$] $\leftarrow$ negative[$|l|$] $+ 1$
      end if
    end for
  end for
  $\mathcal{P} \leftarrow [\,]$
  for each individual index $i = 1$ to P **do**
    $x \leftarrow [\,]$
    for each variable index $j = 1$ to n **do**
      $t \leftarrow$ positive[j]
      $f \leftarrow$ negative[j]
      $p \leftarrow \frac{t}{t+f+\epsilon}$ {ϵ avoids division by zero}
      if `random()` $< p$ **then**
        $x_j \leftarrow$ true
      else
        $x_j \leftarrow$ false
      end if
    end for
    $\mathcal{P}$.append(x)
  end for
  return $\mathcal{P}$

Algorithm 5 Diversity-Based Population Initialisation

Require: Number of individuals P, number of variables n
Ensure: Initialized population $\mathcal{P}$ of size P

```
P ← [ ]
for each individual index i = 1 to P do
    best_x ← null
    best_score ← −1
    for trial = 1 to T do
        x ← [ ]
        for each variable index j = 1 to n do
            x_j ← random() < 0.5
        end for
        min_distance ← ∞
        for each y ∈ P do
            d ← HammingDistance(x, y)
            if d < min_distance then
                min_distance ← d
            end if
        end for
        if min_distance > best_score then
            best_score ← min_distance
            best_x ← x
        end if
    end for
    P.append(best_x)
end for
return P
```

4.3.2 Fitness

Algorithm 6 Max-SAT Fitness Function

Require: Set of clauses $C = \{C_1, C_2, \ldots, C_m\}$, assignment x
Ensure: Fitness value in range $[0, 1]$

```
satisfied ← 0
for each clause C_i in C do
    if C_i is satisfied by assignment x then
        satisfied ← satisfied + 1
    end if
end for
fitness ← satisfied/|C|
return fitness
```

Algorithm 7 Weighted Max-SAT Fitness Function

Require: SAT instance with clauses $C = \{C_1, \ldots, C_m\}$, assignment x
Ensure: Fitness score in range $[0, 1]$
  Compute frequency of each variable across all clauses
  for each clause C_i **do**
    Initialize weight $w_i \leftarrow 1 + \frac{1}{|C_i|}$
    for each literal $l \in C_i$ **do**
      Add $\frac{1}{\text{frequency}(|l|)+1}$ to w_i
    end for
    Add $0.5 \cdot |\text{pos}(C_i) - \text{neg}(C_i)|$ to w_i
  end for
  Normalize all weights so that $\sum_i w_i = 1$

  function CalculateFitness(assignment x)
    score $\leftarrow 0$
  for each clause C_i **do**
    if C_i is satisfied by x **then**
      score $\leftarrow$ score $+w_i$
    end if
  end for
  return score

Algorithm 8 Amplified Fitness Function

Require: Assignment x, SAT instance with clauses $C = \{C_1, \ldots, C_m\}$, amplification factor $\alpha > 1$
Ensure: Amplified fitness score in range $[0, 1]$
  total $\leftarrow |C|$
  satisfied $\leftarrow$ number of clauses in C satisfied by x
  raw_fitness $\leftarrow \frac{\text{satisfied}}{\text{total}}$
  return raw_fitness$^{\alpha}$

4.3.3 Selection

Algorithm 9 Tournament Selection

Require: Population P with fitness values, Tournament size k
Ensure: Selected individual
  $T \leftarrow$ randomly select k individuals from P
  best $\leftarrow$ individual in T with highest fitness
  return *best*

Algorithm 10 Roulette Wheel Selection

Require: Population P with fitness values
Ensure: Selected individual
  $F \leftarrow$ sum of fitness values of all individuals in P
  $r \leftarrow$ random number in $[0, F)$
  cumulative $\leftarrow 0$
  for each individual i in P **do**
    cumulative $\leftarrow$ *cumulative* $+ fitness(i)$
    if *cumulative* $> r$ **then**
      return i
    end if
  end for

Algorithm 11 Rank Selection

Require: Population P with fitness values
Ensure: Selected individual
  Sort population P in descending order of fitness
  Assign rank-based probabilities (e.g., linear or exponential) to each individual
  Generate a random number $r \in [0, 1)$
  Initialize cumulative probability $c \leftarrow 0$
  for each individual i in sorted population **do**
    $c \leftarrow c + \text{probability}(i)$
    if $r < c$ **then**
      return individual i
    end if
  end for

4.3.4 Crossover

Algorithm 12 N-Point Crossover

Require: Parents P_1, P_2 each of length n, Number of crossover points k where $1 \leq k < n$
Ensure: Two offspring C_1, C_2 created by swapping segments at crossover points
  $points \leftarrow$ select k unique random crossover points, sorted
  $child1 \leftarrow [\,],\ child2 \leftarrow [\,]$
  $toggle \leftarrow$ `False`
  $last \leftarrow 0$
  for $point$ in $points$ **do**
    if $toggle$ **then**
      $child1 \leftarrow child1 + parent2[last : point]$
      $child2 \leftarrow child2 + parent1[last : point]$
    else
      $child1 \leftarrow child1 + parent1[last : point]$
      $child2 \leftarrow child2 + parent2[last : point]$
    end if
    $toggle \leftarrow \neg toggle$
    $last \leftarrow point$
  end for

  return $child1,\ child2$

Algorithm 13 Uniform Crossover

Require: Parents P_1, P_2 each of length n
Ensure: Offspring C_1, C_2 with genes randomly selected from parents
  $length \leftarrow$ number of variables
  $child1, child2 \leftarrow [\,], [\,]$
  for $i \leftarrow 1$ to $length$ **do**
    if random() < 0.5 **then**
      $child1[i] \leftarrow parent1[i]$
      $child2[i] \leftarrow parent2[i]$
    else
      $child1[i] \leftarrow parent2[i]$
      $child2[i] \leftarrow parent1[i]$
    end if
  end for

  return $child1,\ child2$

Algorithm 14 Clause Satisfaction Crossover

Require: Parents P_1, P_2, SAT clauses $\mathcal{C}$
Ensure: Offspring C that prioritize variables satisfying more clauses
  $satisfied1 \leftarrow$ clauses satisfied by $parent1$
  $satisfied2 \leftarrow$ clauses satisfied by $parent2$
  $score1 \leftarrow$ number of clauses each variable satisfies in $parent1$
  $score2 \leftarrow$ same for $parent2$
  $child \leftarrow [\,]$
  for $i \leftarrow 1$ to number of variables **do**
    $total \leftarrow score1[i] + score2[i]$
    if $total = 0$ **then**
      $child[i] \leftarrow$ random value from either parent
    else
      $p \leftarrow score1[i]/total$
      $child[i] \leftarrow parent1[i]$ with probability p, else $parent2[i]$
    end if
  end for

  return $child$

4.3.5 Mutation

Algorithm 15 Bit-Flip Mutation

Require: Individual I with bits $I.bits[1..n]$, mutation rate $\mu \in [0, 1]$
Ensure: Mutated individual I'
  $I' \leftarrow I$
  $r \leftarrow \text{UniformRandom}(0, 1)$
  if $r < \mu$ **then**
    $i \leftarrow \text{RandomInteger}(1, n)$
    $I'.bits[i] \leftarrow 1 - I'.bits[i]$ {Flip bit at position i}
  end if
  return I'

Algorithm 16 N-Bit Mutation

Require: Population P of size M, mutation rate $\mu \in [0, 1]$, bit length n
Ensure: Mutated population P'
  $P' \leftarrow \emptyset$
  $k \leftarrow \lceil \mu \times n \rceil$ {Number of bits to mutate per individual}
  for each individual $I \in P$ **do**
    $I' \leftarrow I$
    $S \leftarrow \text{RandomSample}(\{1..n\}, k)$ {Select k unique random indices}
    for each $j \in S$ **do**
      $I'.bits[j] \leftarrow 1 - I'.bits[j]$ {Flip bit}
    end for
    Add I' to P'
  end for
  return P'

Algorithm 17 Guided Mutation

Require: Individual I, SAT clauses $\mathcal{C}$, function `SatisfiedClausesCount`(I)
Ensure: Mutated individual I' with improved clause satisfaction
  $I' \leftarrow I$
  $V \leftarrow \emptyset$ {Set of variables improving satisfaction if flipped}
  for each variable v in I **do**
    $I_{flipped} \leftarrow \text{Copy}(I)$
    $I_{flipped}.bits[v] \leftarrow 1 - I.bits[v]$ {Flip bit}
    if `SatisfiedClausesCount`($I_{flipped}$) > `SatisfiedClausesCount`(I) **then**
      Add v to V
    end if
  end for
  if $V = \emptyset$ **then**
    return I {No improvement possible}
  end if
  $v^* \leftarrow \text{RandomChoice}(V)$
  $I'.bits[v^*] \leftarrow 1 - I'.bits[v^*]$
  return I'

4.4 Hybrid Methods

4.4.1 Local Search

Algorithm 18 Hill Climbing

Require: Initial solution S, max iterations $maxIter$
Ensure: Best solution found S^*
  $S^* \leftarrow S$
  for iteration = 1 **to** maxIter **do**
    $N \leftarrow \text{GenerateNeighbors}(S^*)$ {All neighbors by flipping one bit}
    $S_{best} \leftarrow S^*$
    for each neighbor $s \in N$ **do**
      if $\text{Fitness}(s) > \text{Fitness}(S_{best})$ **then**
        $S_{best} \leftarrow s$
      end if
    end for
    if $\text{Fitness}(S_{best}) \leq \text{Fitness}(S^*)$ **then**
      BREAK {No improvement, local optimum reached}
    end if
    $S^* \leftarrow S_{best}$
  end for
  return S^*

Algorithm 19 Tabu Search

Require: Initial solution S, max iterations $maxIter$, tabu tenure T
Ensure: Best solution found S^*

```
S* ← S
Initialize tabu list 𝒯 ← ∅
for iteration = 1 to maxIter do
    N ← GenerateNeighbors(S)
    S_best ← null
    for each neighbor s ∈ N do
        if s ∉ 𝒯 and ( S_best = null or Fitness(s) > Fitness(S_best) ) then
            S_best ← s
        end if
    end for
    if Fitness(S_best) > Fitness(S*) then
        S* ← S_best
    end if
    Update tabu list: add move from S to S_best
    Remove oldest move if |𝒯| > T
    S ← S_best
end for
return S*
```

Algorithm 20 Simulated Annealing

Require: Initial solution S, initial temperature T_0, cooling rate α, max iterations $maxIter$
Ensure: Best solution found S^*

```
S* ← S
T ← T_0
for iteration = 1 to maxIter do
    s_new ← RandomNeighbor(S)
    ΔE ← Fitness(s_new) − Fitness(S)
    if ΔE > 0 then
        S ← s_new
        if Fitness(S) > Fitness(S*) then
            S* ← S
        end if
    else
        Generate random number r ← UniformRandom(0, 1)
        p ← e^(ΔE/T)
        if r < p then
            S ← s_new
        end if
    end if
    Update temperature: T ← α × T
    if T is sufficiently small then
        BREAK
    end if
end for
return S*
```

4.4.2 Parallelisation

Algorithm 21 Multiple Threads of Independent Populations

Require: Number of threads T, population size per thread N, max generations G
Ensure: Best solution found across all threads S^*
  Initialize populations $P_1, P_2, \ldots, P_T$ independently
  $S^* \leftarrow$ null
  for generation = 1 **to** G **do**
    for parallel-thread = 1 **to** T **do**
      $P_{thread} \leftarrow$ EvolvePopulation(P_{thread})
      $s_{best} \leftarrow$ BestIndividual(P_{thread})
      if S^* = null OR Fitness(s_{best}) > Fitness(S^*) **then**
        $S^* \leftarrow s_{best}$
      end if
    end for
  end for
  return S^*

4.5 Data Structures

This section outlines the main data structures needed to represent both a SAT problem and a solution. More details, including an Object Orientation design of the application will be specified in the Implementation section.

SatClause

Field	Type	Description
Index	`int`	The index of the clause within the CNF formula.
Literals	`List<int>`	A list of integers representing the literals in the clause. Positive values indicate regular literals, negative values indicate negations.

SatInstance

Field	Type	Description
VariableCount	`int`	Number of variables in the CNF formula.
Clauses	`List<SatClause>`	List of all clauses that define the SAT problem instance.

SatSolution

Field	Type	Description
Assignment	`bool[]`	Boolean array where each entry represents the assignment for a variable (true or false).
Instance	`SatInstance`	The corresponding SAT problem this solution attempts to solve.
Fitness	`double?`	The fitness value assigned to this solution, based on how many clauses it satisfies. Nullable.
Generations	`int`	Number of generations this solution took to evolve (for metrics).
Restarts	`int`	Number of restarts applied during evolution (for metrics).

5 Implementation

The implementation phase translates the specifications and design into functional code, ensuring adherence to the specified requirements while attempting to optimise for performance and maintainability. This section details the core components of the system, including all genetic operators, with a focus on integration and testability. Key challenges are addressed through good modular design, iterative

testing, and performance profiling. The code is well structured for extensibility to accommodate future enhancements.

All code is available in a separate submission or can be found on my github page.[2] I will start by introducing the project structure, showing interfaces used and their implementations and relationships then demonstrate using the application and proceed to perform all execution tests.

5.1 Project Structure

The solution is organised into the following namespaces and classes:

- `SAT.GA` - Core genetic algorithm components
- `SAT.GA.Configuration` - Runtime Configuration
- `SAT.GA.Factories` - Dynamic Operator Creation
- `SAT.GA.Fitness` - Fitness functions
- `SAT.GA.Interfaces` - Genetic Algorithm Contracts
- `SAT.GA.LocalSearch` - Local search implementations
- `SAT.GA.Models` - Problem representation
- `SAT.GA.Operators` - Genetic operators
  - `SAT.GA.Operators.Crossover` - Crossover Operators
  - `SAT.GA.Operators.Mutation` - Mutation Operators
  - `SAT.GA.Operators.Selection` - Selection Operators
- `SAT.GA.Population` - Population Initialisation implementations
- `SAT.GA.Utils` - Utilities such as Parsers and Writers

Details of each interface or class is provided below:

Namespace	Object	Description
SAT.GA	`Program`	Main Executable SAT solver, taking command line arguments of CNF file(s) and runtime parameters
	`GeneticAlgorithm`	Core Genetic Algorithm. Accepts all GA operators polymorphically for custom permutation execution
SAT.GA.Configuration	`GaConfig`	Main Configuration class to define all configurable parameters e.g. PopulationSize
	`GaConfigParser`	Parses command line args to generate GaConfig e.g. -p 100 (population size of 100)
SAT.GA.Factories	`OperatorFactory`	Provides methods to create GA operators dynamically from command line args
SAT.GA.Fitness	`MaxSatFitness`	Max-SAT Fitness Implementation
	`AmplificationFitness`	Amplified Fitness Implementation
	`WeightedMaxSatFitness`	Weighted Fitness Implementation
SAT.GA.Interfaces	`ICrossoverOperator`	Crossover Base Contract
	`IFitnessFunction`	Fitness Base Contract
	`ILocalSearch`	Local Search Base Contract
	`IMutationOperator`	Mutation Base Contract
	`IPopulationGenerator`	Population Generator Base Contract
	`IPopulationGenerator`	Selection Base Contract
SAT.GA.LocalSearch	`HillClimbing`	Hill Climbing Implementation
	`SimulatedAnnealing`	Simulated Annealing Implementation

[2] See: https://github.com/craigiparker/SAT.GA

	`TabuSearch`	Tabu Search Implementation
SAT.GA.Models	`SatClause`	Single CNF Clause
	`SatInstance`	Single SAT Problem instance (i.e. one CNF File)
	`SatSolution`	Single Individual of Population
SAT.GA.Operators	`ClauseBasedCrossover`	Clause Satisfaction Implementation
.Crossover	`NPointCrossover`	N-Point Implementation
	`UniformCrossover`	Uniform Implementation
.Mutation	`BitFlipMutation`	Bit Flip Implementation
	`GuidedMutation`	Guided Implementation
	`NBitMutation`	N-Bit Implementation
.Selection	`RankSelection`	Rank Implementation
	`RouletteWheel`	Roulette-Wheel Implementation
	`TournamentSelection`	Tournament Implementation
SAT.GA.Population	`PopulationGenerator`	Abstract Common Implementation
	`ClauseBasedGenerator`	Initialises population based on clause weighting
	`RandomPopulationGenerator`	Initialises population randomly
	`DiversityBasedGenerator`	Initialises population using diversity
SAT.GA.Utils	`DimacsParser`	Dimacs Format Parser Utility
	`OutputWriter`	Formats for Console so progress can easily be monitored and metrics produced

5.2 Interface Methods

To allow each operation to be easily injected by different permutations of implementations, I have implemented the following interfaces and methods:

Interface	Method
ICrossoverOperator	*// Performs crossover of two parents to produce multiple offspring* `IEnumerable<SatSolution> Crossover(SatSolution parent1, SatSolution parent2)`
IFitnessFunction	*// Calculates an individual's fitness* `double Calculate(SatSolution individual)`
ILocalSearch	*// Uses local search to improve fitness* `void Improve(SatSolution individual, int maxIterations)`
IMutationOperator	*// Mutates an individual per mutation rate* `void Mutate(T individual, double mutationRate)`

IPopulationGenerator	*// Creates a new Individual*
	`SatSolution CreateNewIndividual()`
	// Initialise Population of certain size
	`List<SatSolution> InitialisePopulation(int populationSize)`
	// Allow custom tweaks to a population if certain rules are found
	`void OverrideSolution(SatSolution solution)`

ISelectionOperator	*// Selects new offspring of certain size* `List<T> Select(List<T> population, int selectionSize)`

5.3 Class Diagrams

The following diagrams provide another view of how the SAT.GA solution is composed and how the objects are related.

Starting with the core SAT solver in Figure 4 and the Genetic Algorithm in Figure 5, you can then see how each interface is implemented in Figures 6,7,8,9,10 and 11. Also refer to the pseudocode for implementation details.

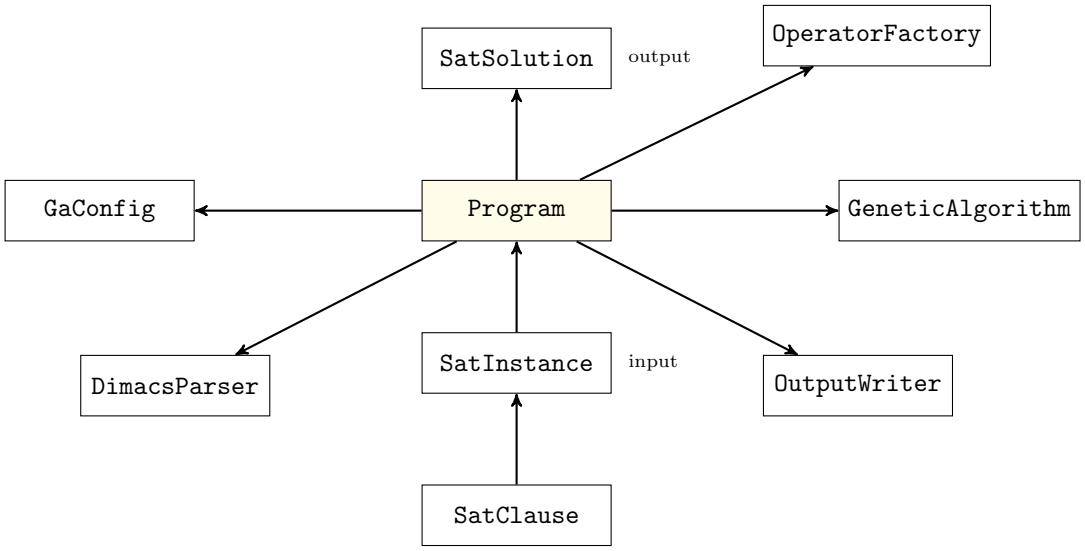


Figure 4: Class Diagram of the Core SAT Solver

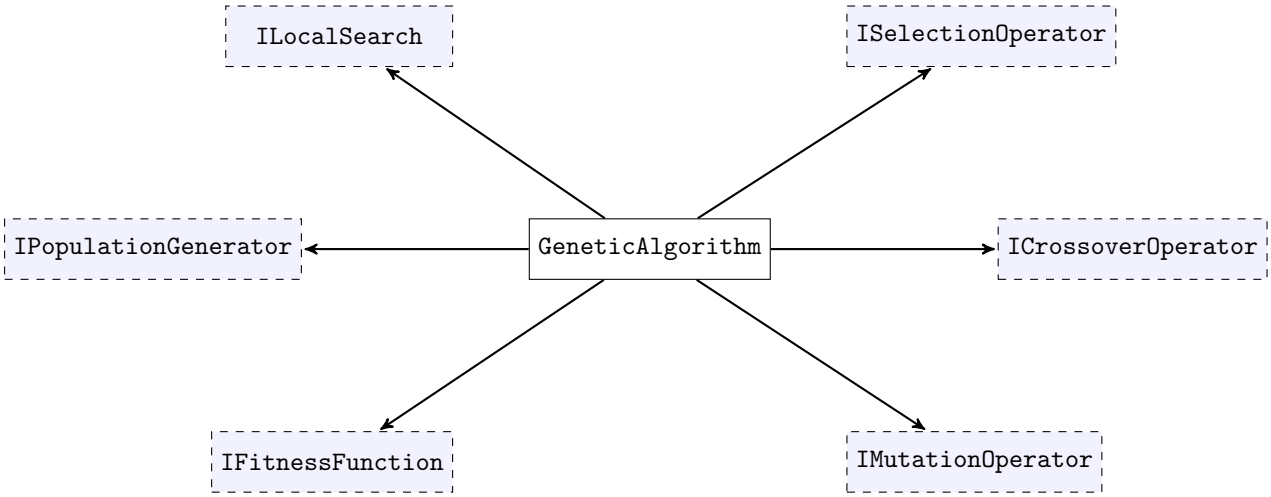


Figure 5: Class Diagram of Genetic Algorithm Components

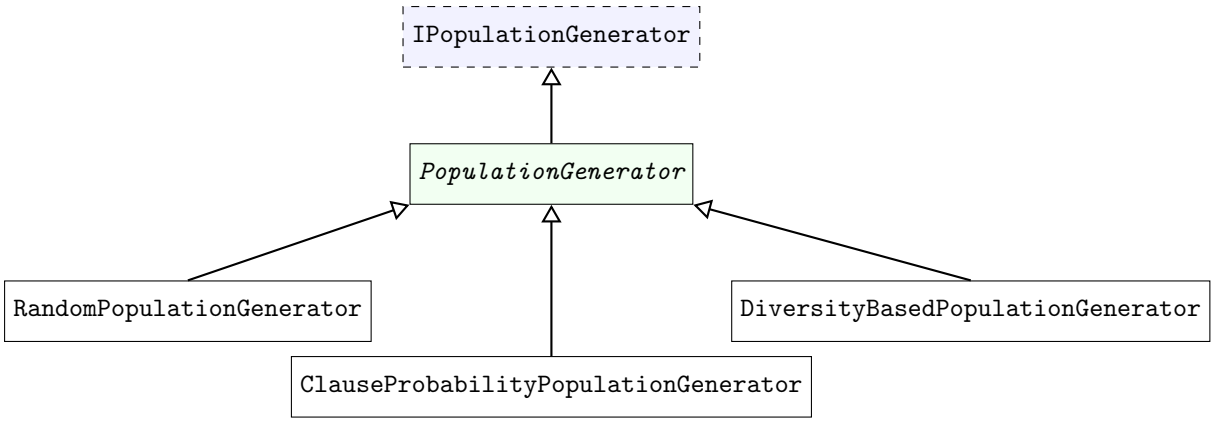


Figure 6: Class Diagram of Population Generators

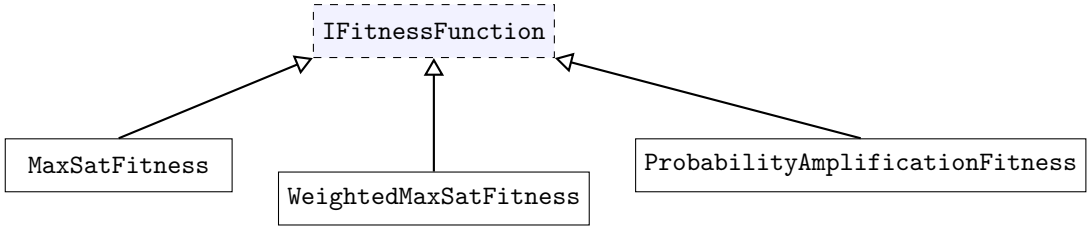


Figure 7: Class Diagram of Fitness Functions

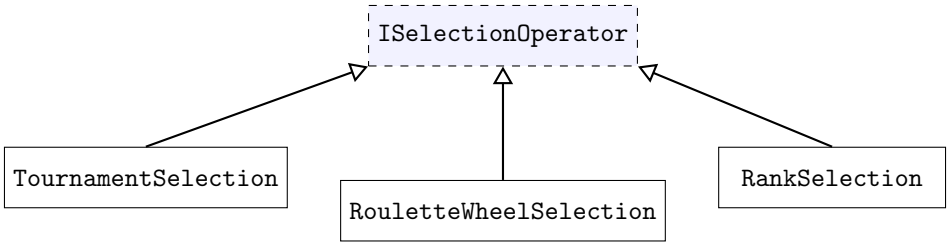


Figure 8: Class Diagram of Selection Operators

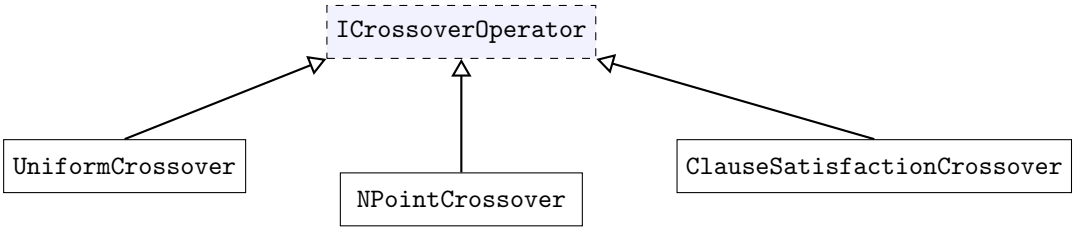


Figure 9: Class Diagram of Crossover Operators

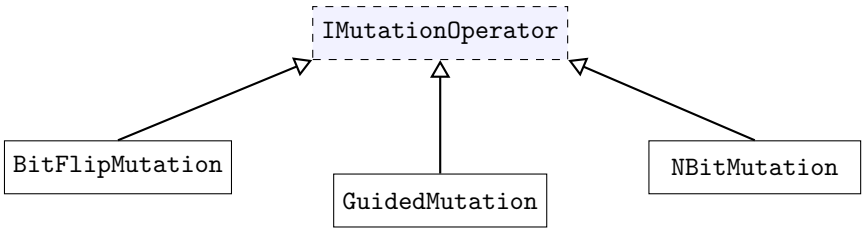


Figure 10: Class Diagram of Mutation Operators

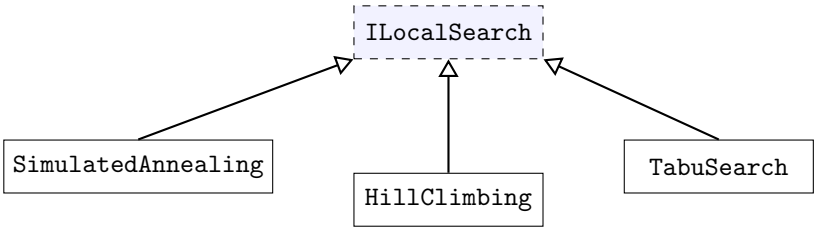


Figure 11: Class Diagram for Local Search

5.4 Demonstration

To see an example run I have prepared a simple set of CNF files to be tested as shown in Figure 12. In this example each problem has 20 variables with 91 clauses.

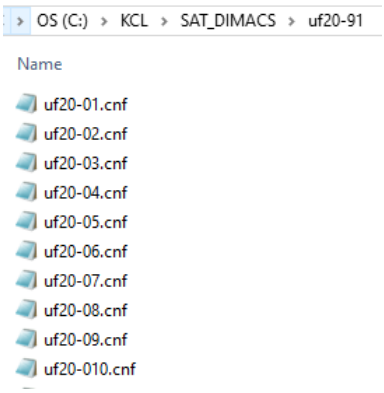


Figure 12: Example Folder

Firstly, I will demonstrate a single run with file **uf20-01.cnf** using all default settings. See Figure 13.

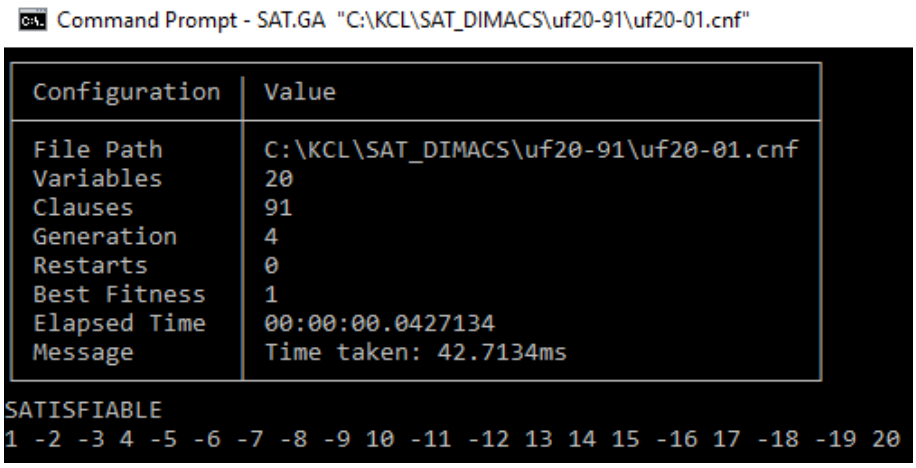


Figure 13: Example Run with No overriding parameters

Now to give a simple example of using different configuration options, I will do a run on the first 2 files in the uf20-91 folder, using a population size of 50 and using Rank selection. See the requirements section for all possible parameters. See Figure 14.

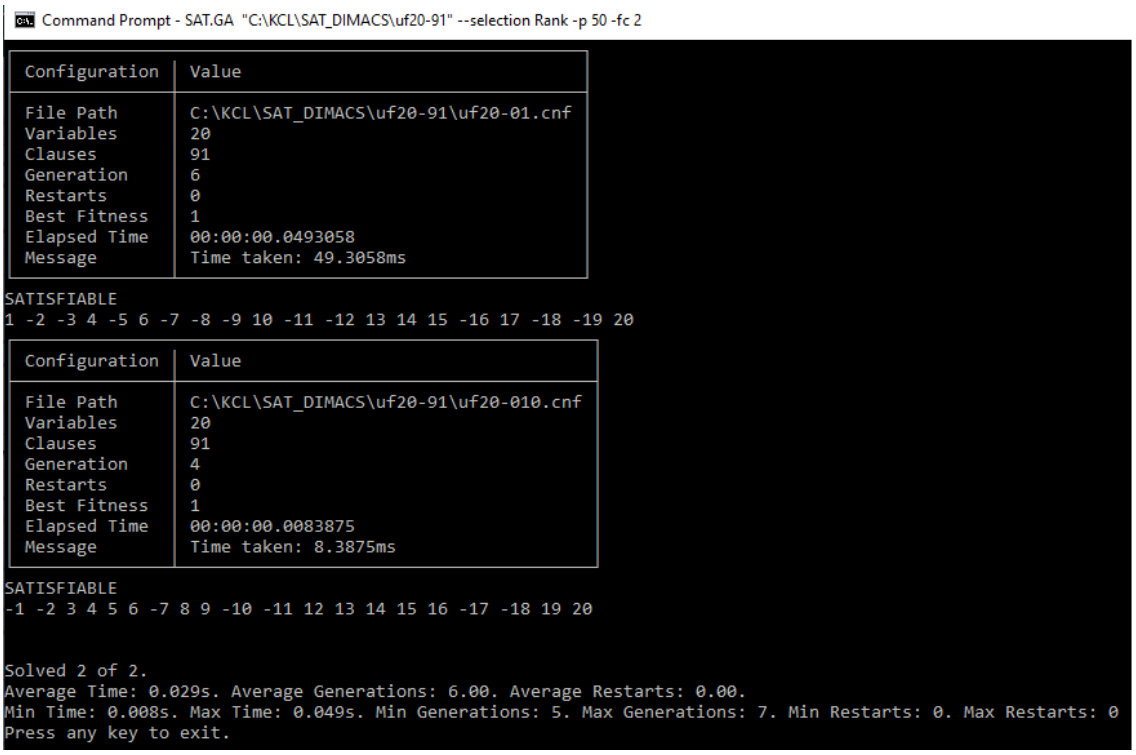


Figure 14: Example Run for 2 files using Rank and population 50

5.5 Execution Plan

Here we critically evaluate which combinations of each operation works best, considering both random and structured instances. Refer to the plan in Section 3.6 under Specifications. We start with pure GA methods, expanding to hybrid options in further tests. All reported results that follows are for the average calculated over all tests for that instance. Individual tests are included in a separate submission with the source code and can also be found on my github page. [3]

[3] See: https://github.com/craigiparker/SAT.GA/blob/master/_Metrics/Metrics.xlsx

5.5.1 Uniform Random Instance - 50 variables, 218 clauses

We start our tests looking at hard uniform random SAT problems. This was signifanctly time-consuming due to the signifcant number of permutations so I restricted this test to 50 SAT problems per test case (of which there are 144 permutations). Since I am using a unit testing framework, I am relying on number of generations as the best way to measure performance. The following parameters are used consistently over all tests, (seen in Table 9).

Parameter	Value
Population Size	100
Elitism Rate	0.1
Max Generations	10,000
Mutation Rate	0.1
Random Seed	2025
Population Generator	Random
Tournament Size	3
Restart Generations	100

Table 9: Test Parameters Configuration

Permutations to be tested are shown below (Table:10). N-Bit mutation cases are expanded to include One-Bit and Two-Bit and N-Point crossover will consider both One-Point and Two-Point.

Selection	Crossover	Mutation	Fitness
Rank	One-Point	Bit-Flip	Max-SAT
Roulette	Two-Point	One-Bit	Weighted
Tournament	Uniform	Two-Bit	Amplified
	Clause	Guided	

Table 10: Test Permutations

Top Ten Combinations After running all permutations, the ten best combination metrics are presented in Table 11. It looks like Guided Mutation features in all top combinations and Tournament selection ranks highly in most cases (6 out of 10). Crossover seems to vary, although the One-Point case gives the best for this test. Weighted fitness appears to offer a slight advantage over Max-SAT, and Amplified only offers additional value when Tournament selection is not used. In all of the top combinations, all the problems were solved successfully. Next we will review each operation individually.

Selection	Crossover	Mutation	Fitness	Generations	Restarts	Time (ms)	% Solved
Tournament	1Point	Guided	Weighted	182	1	683	100%
Tournament	Uniform	Guided	Amplified	186	1	742	100%
Tournament	Uniform	Guided	MaxSat	186	1	742	100%
Tournament	2Point	Guided	Weighted	213	1	776	100%
Rank	1Point	Guided	Amplified	229	1	924	100%
Rank	Uniform	Guided	MaxSat	258	1	1,037	100%
Rank	2Point	Guided	Amplified	297	2	1,314	100%
Roulette	1Point	Guided	MaxSat	306	1	1,307	100%
Tournament	1Point	Guided	Amplified	339	2	1,304	100%
Tournament	1Point	Guided	MaxSat	339	2	1,284	100%

Table 11: Best Pure GA Combinations on Random Instance

Performance comparison of genetic algorithm configurations showing generations needed, restarts required, computation time, and success rate. Tournament selection with 1-point crossover and weighted fitness achieved the fastest execution (182 generations), though all configurations had identical success rates.

Selection The selection results below (Table 12) show a slight performance gain using Tournament over Rank in both accuracy and number of generations. Roulette did not appear to perform as well.

Selection	% Solved	Generations	Restarts	Time (ms)
Tournament	91.5%	1,851	13	5,578
Rank	90.8%	1,930	12	6,906
Roulette	83.4%	2,941	14	10,325

Table 12: Performance comparison of selection operators on random instance

Performance comparison showing Tournament selection achieved both the highest success rate (91.5%) and fastest execution (1,851 generations), while Roulette selection showed surprisingly long computation time and a weak success rate.

Crossover Over all combinations (Table 13) there did not appear to be a clear winner in terms of success, yet Uniform, inline with research showed a slight 10% improvement over one and two point. Clause based crossover didn't appear to offer any gains, perhaps restricting exploration and diversity gains longer term.

Crossover	% Solved	Gens.	Rests.	Time (ms)
Uniform	89.6%	2,063	12	5,282
2-Point	88.9%	2,230	13	5,383
1-Point	89.3%	2,246	13	5,417
Clause	86.4%	2,424	13	5,829

Table 13: Performance comparison of crossover operators on random instance

Performance comparison showing Uniform Crossover achieved both the highest success rate (89.6%) and fastest execution (2,063 generations), while Clause-based crossover showed a slightly long computation time and a weaker success rate.

Mutation The mutation results (Table 14 were probably the most clear-cut. Guided mutation showed a significant improvement of nearly four times over Bit-Flip, and almost achieved a perfect success rate (99%).

Mutation	% Solved	Gens.	Rests.	Time (ms)
Guided	99.0%	580	3	1,722
Bit-Flip	92.0%	2,069	12	5,905
Two-Bit	81.2%	3,056	17	8,183
One-Bit	82.0%	3,258	19	8,650

Table 14: Performance comparison of mutation operators on random instance
Performance comparison showing Guided Mutation as a clear winner, well achieving both the highest success rate (99%) and fastest execution (580 generations).

Fitness The fitness results (Table 15) were quite mixed. Max-SAT a slight performance gain over Amplified, yet Amplified had the highest success rate. This will be addressed in further tests.

Fitness	% Solved	Gens.	Rests.	Time (ms)
Max-Sat	88.8%	2,188	13	6,156
Amplified	90.0%	2,197	13	7,473
Weighted	86.9%	2,337	12	7,636

Table 15: Performance comparison of fitness functions on random instance
Performance comparison showed quite mixed results of varying success and performance.

5.5.2 Structured Instance (Graph-Colouring) - 90 variables, 300 clauses

Our structured analysis begins with a graph colouring test case, having 30 vertices and 60 edges (90 variables, 300 clauses). We omit any detailed explanation of the problem represented, other than to say that it is a structured case and may behave quite different to the random ones. Repeating as for random instances above, we repeat the same tests with exactly the same conditions and parameters but for our structured test case below.

Top Ten Combinations The best structured results are shown in Table 16. Unlike with the Random case, it appears Two-Point and One-Point Crossover stand-out. Guided Mutation still dominates and Tournament selection still appears to perform better. Fitness functions are still a little mixed with Max-SAT performing a bit better than Weighted. In all of the best combinations, success of 100% was achieved.

Selection	Crossover	Mutation	Fitness	Generations	Restarts	Time (ms)	% Solved
Tournament	2Point	Guided	Amplified	174	0	604	100%
Tournament	2Point	Guided	MaxSat	174	0	613	100%
Rank	2Point	Guided	Weighted	215	0	710	100%
Rank	2Point	Guided	MaxSat	224	1	786	100%
Tournament	2Point	Guided	Weighted	260	1	851	100%
Tournament	1Point	Guided	Amplified	278	1	997	100%
Tournament	1Point	Guided	MaxSat	278	1	962	100%
Tournament	1Point	Guided	Weighted	286	1	956	100%
Rank	1Point	Guided	Weighted	311	1	1037	100%
Tournament	Uniform	Guided	Weighted	365	1	1238	100%

Table 16: Best Pure GA Combinations on Structured Instance
A number of differences were found compared with Random instances, notably One and Two point Crossover dominating Uniform.

Selection The selection results (Table 17) ranked similarly to random instances with Tournament selection dominating slightly more than for Random. Roulette was clearly unsuccessful solving only 37% of cases.

Selection	% Solved	Generations	Restarts	Time (ms)
Tournament	81.38%	3605	19	10248
Rank	73.23%	4465	21	15255
Roulette	36.59%	7276	23	49667

Table 17: Performance comparison of selection operators on structured instance
Tournament selection continues to dominate over Rank and Roulette methods.

Crossover Probably the most different outcome when compared to Random instances, One and Two-Point Crossover slightly dominated Uniform, though only slightly. Results are shown in Table 18.

Crossover	% Solved	Generations	Restarts	Time (ms)
2Point	67.71%	4693	20	46426
1Point	66.84%	4784	19	10141
Uniform	63.72%	5054	21	13114
Clause	56.60%	5933	23	30493

Table 18: Performance comparison of crossover operators on structured instance
One and Two-Point Crossover slightly perform better than Uniform Crossover.

Mutation As we saw in the Random case, Guided Mutation continues to dominate by over five times the next best choice. Results are shown in Table 19.

Mutation	% Solved	Generations	Restarts	Time (ms)
Guided	98.95%	907	4	4202
BitFlip	60.91%	5668	22	55883
2Bit	59.20%	5962	23	18514
1Bit	35.94%	7911	34	21781

Table 19: Performance comparison of mutation operators on structured instance
Guided Mutation, as with Random instances, continues to dominate.

Fitness As with Random, Fitness function results are mixed, although this time Weighted performs slightly better on success and just behind Amplified on performance. Results are in Table 20.

Fitness	% Solved	Generations	Restarts	Time (ms)
Amplified	64.49%	4997	22	43183
Weighted	65.01%	5077	17	15773
MaxSat	61.62%	5276	23	16291

Table 20: Performance comparison of fitness functions on structured instance
Mixed results again, slightly different yet comparable with Random instances.

Comparison In summary after the first two tests, I established that Tournament Selection and Guided Mutation consistently outperformed in both Random and Structured instances. I still want to run more tests on Crossover and Fitness to help establish the best overall fit.

5.5.3 Population Initialisation

Now we have a few less permutations to analyse, I will now look into population initialisation algorithms. Here I will keep Tournament Selection and Guided Mutation fixed and focus on the remaining variants of Crossover (One-Point, Two-Point and Uniform) and Fitness (Max-SAT, Weighted and Amplified) while trying to determine the best population initialisation generator (Random,Clause-Based,Diversity-Based). Since our scope has narrowed I now increase the difficulty and the number of instances tested.

Random - 100 variables/430 clauses For this test I will expand the difficulty to 100 variables and 430 clauses with 100 test problems for each permutation. The results (Table 24) show an interesting outcome with One-Point Crossover and Weighted Fitness now dominating. The population generator used shows a slight performance gained (around 10%) using Clause-Based population generation over the other options. Accuracy however is quite similar with a minor improvement using Clause-based.

Structured - 150 variables/545 clauses For this test I will expand the difficulty to 150 variables and 545 clauses (50 vertices and 115 edges) with 100 test problems for each permutation. The results (Table 25) show an interesting outcome with One-Point Crossover and Weighted Fitness now dominating. The population generator used shows a slight performance gained (around 10%) using Clause-Based population generation over the other options. Accuracy, however, is quite similar with a minor improvement using Clause-based. These results compare well with the Random instances.

Crossover	Fitness	Generator	% Solved	Generations	Restarts	Time (ms)
1Point	Weighted	Clause	92%	2,168	12	8,730
1Point	MaxSat	Random	90%	2,790	16	10,326
2Point	MaxSat	Clause	88%	3,065	20	10,355
1Point	MaxSat	Clause	86%	3,103	20	10,743
1Point	MaxSat	Diversity	92%	3,103	18	11,489
Uniform	Weighted	Diversity	88%	3,118	17	10,951
1Point	Weighted	Diversity	84%	3,299	16	12,718
Uniform	Weighted	Random	82%	3,368	18	11,133
Uniform	Amplified	Clause	84%	3,463	23	11,860
1Point	Weighted	Random	90%	3,511	18	13,389
2Point	MaxSat	Random	88%	3,700	22	13,206
Uniform	Weighted	Clause	84%	3,771	22	11,811
Uniform	Amplified	Diversity	80%	3,975	25	14,524
Uniform	Amplified	Random	82%	4,017	25	14,081
2Point	MaxSat	Diversity	84%	4,117	25	14,625
Summary						
		Clause	87%	3,114	19	10,700
		Random	86%	3,477	20	12,427
		Diversity	86%	3,522	20	12,861

Table 21: Population Results for Random instances (100 variables)

Performance comparison showing 1Point crossover with Weighted fitness and Clause generator achieved the best balance of success rate (92%), speed (2,168 generations), and restarts (12). The summary reveals Clause generators required fewer generations while maintaining marginally better success rates (87% vs 86%).

Crossover	Fitness	Generator	% Solved	Generations	Restarts	Time (ms)
1Point	Weighted	Clause	98%	2,130	9	8,093
1Point	MaxSat	Random	96%	2,511	13	9,547
Uniform	Amplified	Random	96%	2,511	13	9,651
2Point	MaxSat	Clause	98%	2,539	13	8,925
1Point	MaxSat	Clause	96%	2,619	14	9,406
Uniform	Amplified	Clause	96%	2,619	14	9,594
1Point	Weighted	Diversity	94%	2,655	12	10,824
1Point	MaxSat	Diversity	96%	2,754	14	10,519
Uniform	Amplified	Diversity	96%	2,754	14	10,748
1Point	Weighted	Random	92%	2,866	12	11,483
2Point	MaxSat	Diversity	94%	3,149	19	11,998
Uniform	Weighted	Clause	88%	3,223	13	11,049
Uniform	Weighted	Diversity	86%	3,600	16	13,061
2Point	MaxSat	Random	96%	3,635	19	13,429
Uniform	Weighted	Random	86%	4,222	18	14,953
Summary						
		Clause	95%	2,626	13	9,413
		Diversity	93%	2,982	15	11,430
		Random	93%	3,149	15	11,813

Table 22: Population Results for Structured instances (150 variables)

Performance comparison showing 1Point crossover with Weighted fitness and Clause generator achieved both the highest success rate (98%) and lowest restarts (9). The summary reveals Clause generators required fewer restarts (13 avg) compared to Diversity and Random (15 avg each), while maintaining superior performance.

Comparison In both Random and Structures cases, Clause-based population generation worked best. One surprise was that One-Point Crossover and Weighted Fitness both dominated. I am now fine-tuning

the model and will utilise this knowledge in the next test cases.

5.5.4 Parameter Tuning

I have now established our preferred defaults (Tournament selection, One-Point Crossover, Weighted Fitness, Guided Mutation and Clause-based population generation). Keeping the defaults fixed I now look to determine which parameter values work best, starting with the restart rate.

Restart Rate To avoid getting stuck in local optima we often need to restart the genetic algorithm. The restart rate considers how many generations we have had without an improvement in fitness. Up until now, we have used a restart rate of 100. For this test we consider two population sizes over random instances of 100 variables and 100 instances. Structured cases were not considered as my assumption is there shouldn't be a significant difference. In both population size cases it became clear that we need some time before restarting but there is no benefit in waiting too long. For both population sizes using 25 as the restart rate seemed optimal, although for a population of 100, a 20 generation restart rate had higher accuracy.

The table 23 below shows the results recorded and a scatter-plot comparing a population of both 50 and 100 performance is seen in Figure 15.

Pop Size	Restart Rate	% Solved	Gen	Restarts	Time (ms)
50	0	12%	8,734	1,242	31,998
50	20	88%	2,478	35	7,901
50	**25**	**94%**	**2,042**	**25**	**6,118**
50	30	94%	2,310	26	6,756
50	35	94%	2,456	26	7,365
50	50	94%	2,503	21	7,483
50	75	88%	2,651	17	8,206
50	100	92%	2,800	15	8,658
50	150	90%	3,021	12	8,924
50	200	88%	3,434	11	10,661
100	20	100%	1,550	22	9,461
100	**25**	**98%**	**1,360**	**17**	**7,965**
100	30	92%	1,668	20	10,090
100	50	94%	1,878	17	10,902
100	100	92%	2,064	12	12,371

Table 23: Restart Rates

Performance comparison showing population size 100 with parameter 20 achieved perfect solve rate (100%). *Bold* rows highlight parameter value 25 configurations, showing excellent performance with population 50 (94% solved, 2,042 generations) and near-perfect with population 100 (98% solved, 1,360 generations).

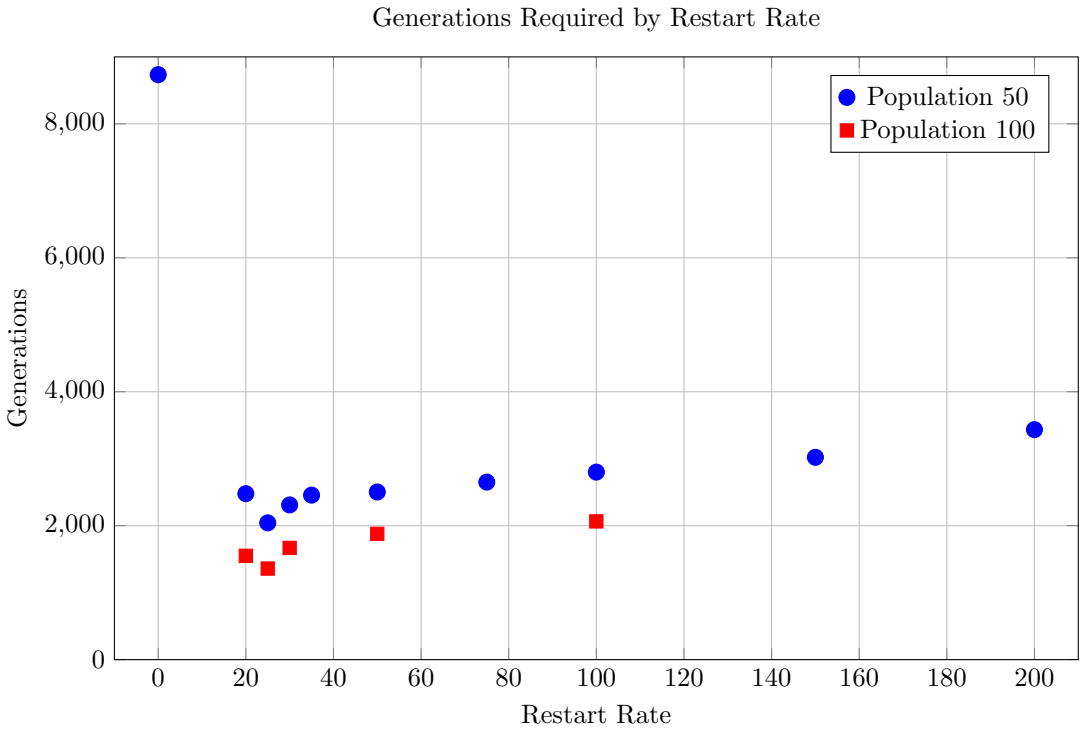


Figure 15: Restart Rate Performance
Scatter plot showing generations required per restart rate. After an initial dip for restart rate of 25, the performance clearly deteriorates the higher the rate.

Population Size - Random Instances Now we try to determine the optimal population size. I consider both 50 and 100 variable SAT problems over a range of population sizes from 20 to 200 (stopped at 200 as couldn't see any improvements). For fair comparison I adjust the maximum number of generations so the operational limit (population size * generations) is consistent. The key metric to check is hence total operations and not generations.

Looking at the results in Table 24 and the scatter-plot in Figure 16 we can see clearly a smaller population scales well better than large ones. Our optimal value was a population of 30 for 100 variables and 25 for 50 variables. This matches what has been said in the literature review. The measure of success also shows the best results.

Population Size - Structured Instances For the structured case we now consider 150 variables to check for consistency with the random test. The results show great consistency with a population size of 30 also being the most optimal, this time even more significantly. Results are seen below in Table 25 and Figure 17.

Var.	Max Gen	Pop. Size	% Solved	Gen	Restarts	Time(ms)	Operations
100	25,000	20	96%	4,970	58	6,061	99,400
100	20,000	25	94%	4,316	51	6,611	107,900
100	**16,667**	**30**	**98%**	**2,537**	**30**	**4,611**	**76,110**
100	14,286	35	96%	2,395	29	4,900	83,825
100	12,500	40	92%	2,678	33	6,344	107,120
100	10,000	50	94%	2,042	25	6,118	102,100
100	5,000	100	92%	1,190	15	7,074	119,000
100	3,333	150	86%	1,054	14	9,612	158,100
100	2,500	200	84%	904	12	11,172	180,800
50	25,000	20	100%	379	6.4	243	7,580
50	**20,000**	**25**	**100%**	**268**	**4.7**	**220**	**6,700**
50	16,667	30	100%	246	4.4	238	7,380
50	14,286	35	100%	290	5.3	318	10,150
50	12,500	40	100%	245	4.4	306	9,800
50	10,000	50	100%	177	3.16	285	8,850
50	5,000	100	100%	94	1.6	305	9,400
50	3,333	150	98%	193	3.8	969	28,950
50	2,500	200	98%	203	4.2	1,363	40,600

Table 24: Population Size Performance - Random Instances

Performance metrics showing performance between different population sizes. The optimal parameter range appears around 30 for both configurations.

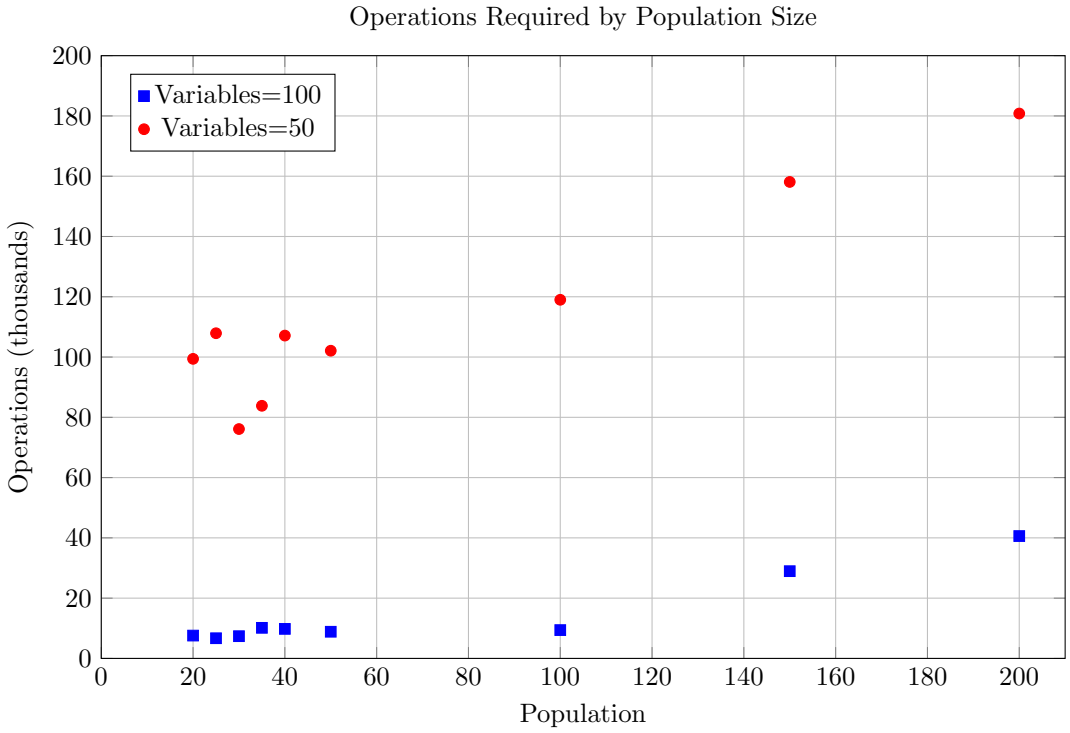


Figure 16: Operations per population size - Random Instances

Scatter plot showing operations required versus population size. The optimal parameter range (20-40) yields the lowest operation counts for both configurations.

Var.	Max Gen	Pop. Size	% Solved	Gen	Restarts	Time(ms)	Operations
150	18,750	20	84%	7,188	65	18,749	143,760
150	15,000	25	96%	4,338	41	14,550	108,450
150	**12,500**	**30**	**98%**	**2,302**	**21**	**8,808**	**69,060**
150	10,714	35	96%	3,738	36	18,950	130,830
150	9,375	40	92%	2,830	27	15,655	113,200
150	7,500	50	96%	2,272	21	16,010	113,600
150	3,750	100	98%	1,148	11	15,910	114,800
150	2,500	150	86%	934	9	19,394	140,100
150	1,875	200	90%	632	6	17,008	126,400

Table 25: Population Size Performance - Structured Instances

Performance metrics showing optimal performance when population size is30 (98% solved, 69,060 operations). Higher parameter values generally reduce generations but increase operational costs.

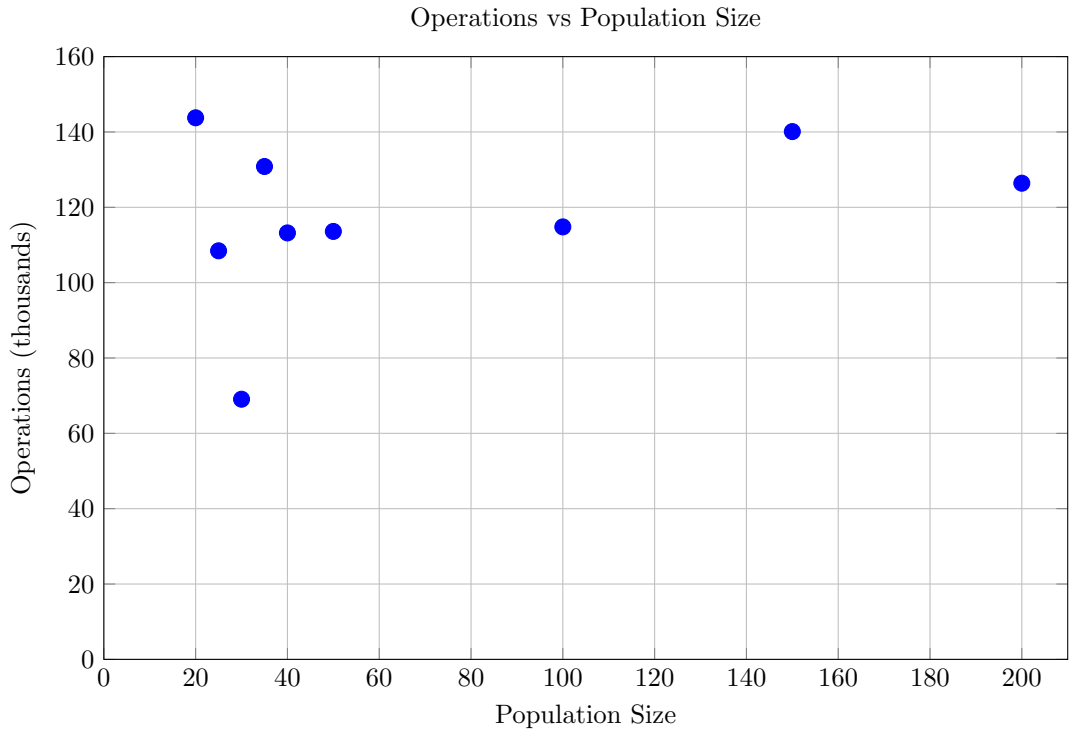


Figure 17: Operations per population size - Structured Instances
Scatter plot showing operational costs versus population size. Higher parameters reduce generations but don't consistently lower operational costs.

Elitism Rate Next I am looking at the elitism rate. This is simply a measure of how much of the fittest population we retain between generations. Research indicated a small percentage is useful but not too much as we still want the ability to explore the search space. The results of this test are shown in Table 26 and a scatter-plot showing the relationship in Figure 18.

It seems obvious that lower values of elitism work best and gradually deteriorate as the rate increases. The optimal value in the test show a rate of 10% works best. This is seen in both the least number of generations and accuracy rates.

Variables	Elitism	% Solved	Gen.	Restarts	Time(ms)
100	0.00	94%	2,554	28	4,730
100	0.05	94%	2,903	34	5,348
100	**0.10**	**98%**	**2,455**	**29**	**4,237**
100	0.15	98%	3,138	37	5,566
100	0.175	88%	3,725	45	6,640
100	0.20	86%	4,206	50	7,275
50	0.00	100%	266	4.4	273
50	0.05	100%	311	5.6	317
50	**0.10**	**100%**	**246**	**4.4**	**243**
50	0.15	100%	260	4.5	266
50	0.175	100%	384	7.0	372
50	0.20	100%	489	8.9	453

Table 26: Elitism Rate Performance

Performance comparison showing a 10% Elitism Rate performs best on both 50 and 100 variables with the highest accuracy and least number of generations.

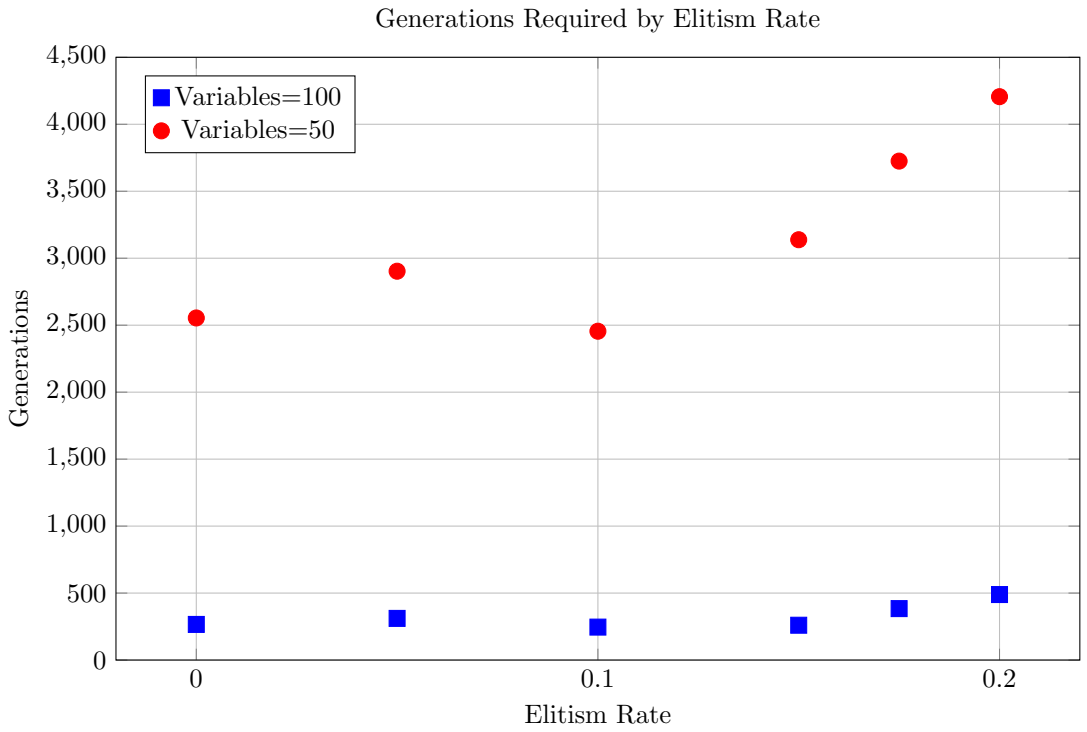


Figure 18: Elitism Rate Performance

Scatter plot showing generations versus elitism rate. Both configurations show best performance at parameter=0.10, with higher values increasing generation counts.

Tournament Size Since we have been using Tournament Selection in most of our test cases, it is worth determining the most optimal size for each tournament. Past research has indicated that a value of between 3 and 5 works best. Since we are testing with our optimal population size (30), the scope of testing is limited to a small range of values. The results are seen in Table 27 and shown on a graph in Figure 19.

For both 50 and 100 variable test cases a tournament size of 3 worked best, measured in both accuracy and performance.

Variables	T-Size	% Solved	Gen	Restarts	Time (ms)
100	2	90%	3,480	37	6,183
100	**3**	**98%**	**2,455**	**29**	**4,237**
100	4	92%	2,829	36	5,245
100	5	88%	3,509	47	6,456
100	6	94%	3,117	43	5,534
50	2	100%	406	6.7	383
50	**3**	**100%**	**246**	**4.4**	**243**
50	4	100%	284	5.3	288
50	5	100%	210	3.9	211
50	6	100%	242	4.7	238
50	7	100%	319	6.7	317

Table 27: Tournament Size Performance

Performance comparison showing both 50 and 100 variables achieved the best performance with a tournament size of 3.

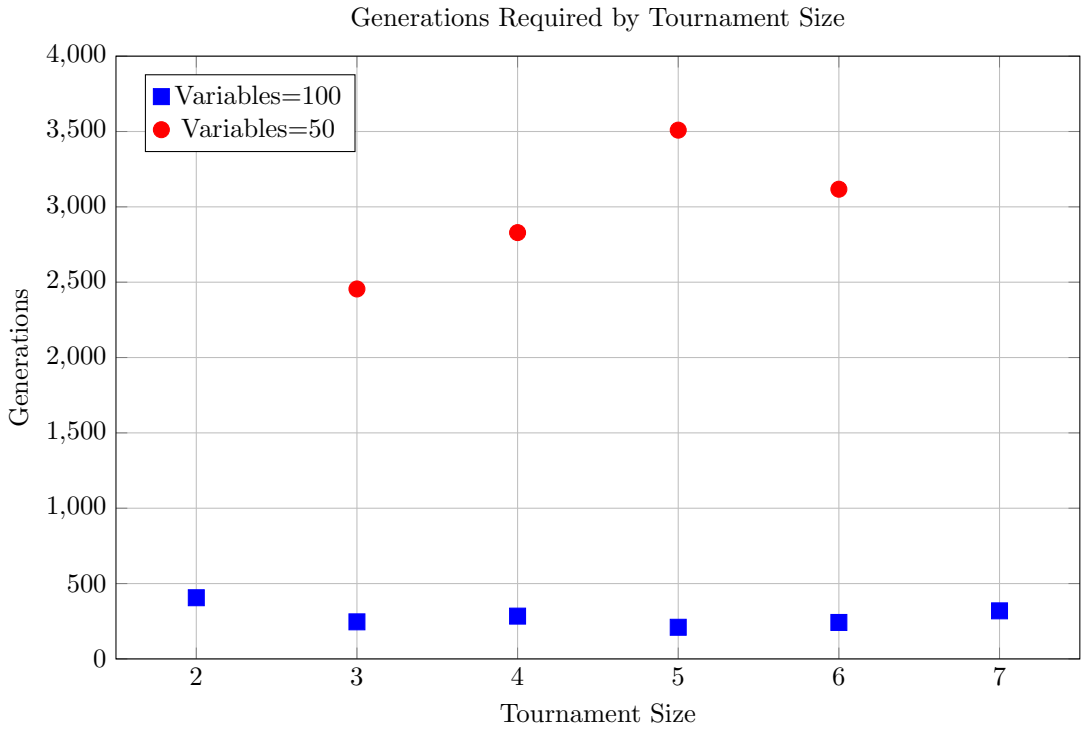


Figure 19: Tournament Size Performance

Scatter plot showing generations required versus tournament size. Best performance (least generations) happens at a size of 3, before slowly deteriorating.

5.5.5 Local Search

Local search has been shown to significantly improve performance compared to a pure genetic algorithm alone.I tried the three local search implementations and used the pure GA implementation (no local search) as a benchmark. We use the parameter of iterations to measure how deep per each generation we search before continuing to the next generation.

Overall, the results were disappointing (see Table 28) and I expected something better. I was able to force a better performance in terms of number of generations by using extreme values for each method. If we factor in the number of operations per generation however, we can see a clear deterioration in performance, as seen by the time taken.

Although not the main focus of this report, I would have liked the local search to add value. My implementation performed a local search on every new child in each generation. I may have been able

to get better results if this had been selected better with the use of some heuristics. I suspect that the algorithms that work well in practice are engineered differently and focus on local search first and genetic algorithms second.

Search	Iterations	% Solved	Gen	Restarts	Time(ms)
None	0	98%	2,537	30	9,602
Hil lClimbing	1	92%	3,493	42	13,863
Hill Climbing	30	96%	3,678	46	17,725
Hill Climbing	100	94%	2,608	36	18,617
Hill Climbing	500	100%	1,600	27	36,067
Tabu	1	96%	3,758	46	22,376
Tabu	30	96%	2,938	40	58,232
Tabu	100	96%	2,163	37	144,006
Simulated Annealing	1	94%	2,718	31	10,923
Simulated Annealing	30	98%	2,527	30	12,383
Simulated Annealing	100	98%	2,648	30	17,938

Table 28: Local Search Performance

Performance comparison of search algorithms showing local search can improve the performance in accuracy in terms of generation cost, but once the operational cost per generation is considered no performance gain can be measured, as seen by the execution time.

5.5.6 Parallel Strategies

One big advantage of genetic algorithms is the ability for each generation to be run in parallel. I explore this strategy and determine how many threads of execution is optimal. Each thread will run independently of each other and when the first thread finds a solution we stop. In this test I am evaluating over 1000 SAT problems of 100 variables and 430 clauses. I am also allowing up to 20000 generations to fully test the differences. In this test I am not using any random seed but the large number of test cases should be a fair indication of performance.

For this test I am relying on execution time as a better benchmark than generations alone as I expect in parallel fewer generations will always be needed. For reference my computer has 4 cores and 8 logical processors. I used the command line argument *–thread-count* to test.

At this point I was worried that the console output between threads may introduce resource contention so I introduced a new command line argument *–hide* to hide any cross thread progress and only show the result when done. The results of the solver are shown below in raw format. The first test case is using standard console output and the second without. Table 29 summarizes the average of each run and is graphed in Figure 20.

The results come inline with my expectations and in fact the best performance happened to match when the number of threads used matched my computer logical processor count of 8. This is good news and indicates that potentially optimal GA SAT solvers that are run on high performance computers with a large number of processors can achieve significantly faster results on large, hard SAT problems.

Increasing the thread count beyond 8 showed a sudden drop off in performance in terms of execution time, but since more threads were working and the generation limit per thread was constant, the number of generations needed naturally fell as expected, and is not a good indicator of success. The success rate naturally increased also as a result. The overhead of managing too many threads was the obvious contributing factor.

```
# 32 Threads
# 1
Solved 1000 of 1000.
Average Time: 3.103s. Average Generations: 318.46. Average Restarts: 3.24.
Min Time: 0.095s. Max Time: 118.074s. Min Generations: 14. Max Generations: 7559.
Min Restarts: 0. Max Restarts: 85
# 2
Solved 998 of 1000.
```

```
Average Time: 3.193s. Average Generations: 306.89. Average Restarts: 3.10.
Min Time: 0.123s. Max Time: 164.399s. Min Generations: 17. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 240

# 16 Threads
# 1
Solved 1000 of 1000.
Average Time: 3.527s. Average Generations: 348.58. Average Restarts: 3.63.
Min Time: 0.093s. Max Time: 145.972s. Min Generations: 17. Max Generations: 12758.
Min Restarts: 0. Max Restarts: 163
# 2
Solved 1000 of 1000.
Average Time: 1.257s. Average Generations: 299.14. Average Restarts: 3.00.
Min Time: 0.045s. Max Time: 61.932s. Min Generations: 18. Max Generations: 13255.
Min Restarts: 0. Max Restarts: 171

# 8 Threads
#1
Solved 996 of 1000.
Average Time: 1.371s. Average Generations: 560.97. Average Restarts: 6.20.
Min Time: 0.053s. Max Time: 49.050s. Min Generations: 17. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 248
#2
Solved 997 of 1000.
Average Time: 1.238s. Average Generations: 514.62. Average Restarts: 5.60.
Min Time: 0.039s. Max Time: 47.280s. Min Generations: 14. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 247

# 4 Threads
#1
Solved 998 of 1000.
Average Time: 1.615s. Average Generations: 890.55. Average Restarts: 10.16.
Min Time: 0.036s. Max Time: 35.870s. Min Generations: 17. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 245
#2
Solved 995 of 1000.
Average Time: 1.639s. Average Generations: 935.64. Average Restarts: 10.75.
Min Time: 0.027s. Max Time: 35.206s. Min Generations: 15. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 267

# 2 Threads
# 1
Solved 985 of 1000.
Average Time: 2.842s. Average Generations: 1708.62. Average Restarts: 20.30.
Min Time: 0.032s. Max Time: 33.826s. Min Generations: 16. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 256
# 2
Solved 981 of 1000.
Average Time: 3.274s. Average Generations: 2009.13. Average Restarts: 23.90.
Min Time: 0.028s. Max Time: 33.414s. Min Generations: 15. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 266

# 1 Thread
# 1
Solved 954 of 1000.
Average Time: 4.844s. Average Generations: 3027.69. Average Restarts: 36.48.
Min Time: 0.043s. Max Time: 33.029s. Min Generations: 0. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 261
```

```
# 2
Solved 961 of 1000.
Average Time: 4.760s. Average Generations: 3039.01. Average Restarts: 36.60.
Min Time: 0.033s. Max Time: 32.519s. Min Generations: 0. Max Generations: 20000.
Min Restarts: 0. Max Restarts: 263
```

Threads	% Solved	Generations	Restarts	Time(ms)
32	99.9	312.5	3.17	3,148
16	100.0	324.0	3.32	2,392
8	**99.65**	**538.0**	**5.90**	**1,304.5**
4	99.65	913.5	10.46	1,627
2	98.3	1,858.5	22.10	3,058
1	95.75	3,033.5	36.54	4,802

Table 29: Parallel Execution Performance

Performance metrics showing 8 threads achieve the fastest computation time (1,304.5ms), matching the number of logical units on the test machine.

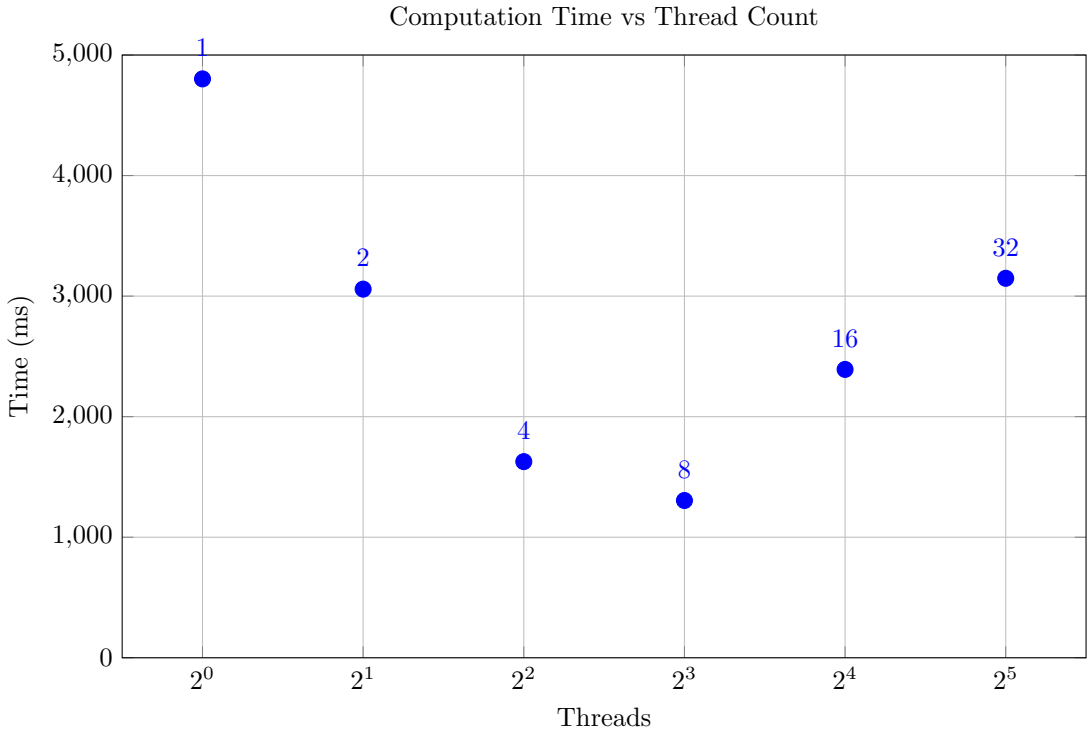


Figure 20: Parallel Execution Performance

Scatter plot showing non-linear relationship between thread count and computation time. Performance increases as we increase the number of threads until we reach the optimal point at 8 threads (1,304.5ms). Performance degrades with both too few and too many threads due to under-utilisation and overhead respectively.

6 Evalation

After following the execution plan I now have the most optimal SAT solver from the tests covered. A summary of these parameters and settings is shown in Table 30. Note that for evaluation purposes I have also allowed up to 20,000 generations to evaluate the full range of results. I also use all the problems available in each category. Typically the smaller problems have 1000 problems and the larger ones 100. All tables show average results. In previous testing I used a testing framework and wasn't strictly optimised for performance. For evaluation I have built a release build and ensured that no other software

is running on the computer. The key metric I am now using to measure performance is execution time in milliseconds, also as it makes it easier to compare with other benchmarks.

In this evaluation plan I will test all of the hard random instances, from 20 to 250 variables, evaluating how well the algorithm performs as the problem size grows. I also do the same tests for our structured test case, from 90 to 300 variables. I start with evaluating both the standard and parallel models, and then summarise and compare the results against MiniSAT and WalkSAT algorithms. Full evaluation results can be found on my github page. [4]

Parameter	Value
Population Size	30
Elitism Rate	0.1
Tournament Size	3
Restart Generations	25
Population Generator	Clause
Fitness Function	Weighted
Selection Operator	Tournament
Crossover Operator	One-Point
Mutation Operator	Guided
Thread Count	1 & 8
Max Generations	20,000

Table 30: Evaluation Parameters Configuration

6.1 Hard Random Instances - 20 to 250 variables

I begin by summarising all hard random instances and then proceed to break down the performance for each different sized problem. The algorithm performed well over the smaller problems, able to solve 956 of the 1000 problems with 100 variables with a single thread and 998 using 8 threads, within the maximum allowed number of generations. 8 threads performed very well on 100 variables and was able to complete within 0.78 seconds on average. The single thread case was comparably slower as expected, completing in 2.3 seconds on average. A summary is given in Table 31.

The trend shown in Figure 21 shows that performance of 8 threads outperforms a single thread up to a problem size of 175 then the benefits of concurrency begin to reduce. Both implementations perform very well up to 125 variables and then computational time appears to change exponentially. I suspect this may be due to most testing done around the 100 variable range but also due to the NP-Hard nature of SAT.

For completeness I also demonstrate the performance in terms of number of generations (Figure 22) and success rates (Figure 23). While not the metric used for this evaluation it is interesting to see a similar exponential trend with generational cost and a sudden drop in success after 125 variables.

Next we will break down further metrics per problem size and evaluate the variance needed in solving each one.

[4] See: https://github.com/craigiparker/SAT.GA/tree/master/_Metrics/evaluation

Threads	Size	Success (%)	Generations	Restarts	Time (ms)
1	25	100.0	15	0.23	3
1	50	100.0	242	4.27	48
1	75	99.0	1,083	15.76	291
1	100	95.6	2,983	36.21	2,334
1	125	84.0	6,973	72.00	3,037
1	150	70.0	10,566	95.12	11,757
1	175	37.0	15,309	123.00	9,726
1	200	20.0	17,770	127.00	12,654
1	225	13.0	18,503	121.00	15,534
1	250	11.0	18,702	112.37	38,168
8	25	100.0	3	0.01	2
8	50	100.0	36	0.30	16
8	75	100.0	113	1.10	71
8	100	99.8	539	5.98	780
8	125	100.0	1,590	16.00	1,359
8	150	89.0	4,028	35.61	8,482
8	175	82.0	7,661	61.00	9,013
8	200	57.0	11,992	85.00	16,355
8	225	43.0	13,859	90.10	21,696
8	250	28.0	15,873	94.62	57,162

Table 31: Hard Random Instance Performance - 20 to 250 variables

Complete performance data showing the impact of problem size and thread count on success rate, generations required, restarts, and computation time. The 8-thread configuration maintains higher success rates with lower computation times until very large problem sizes (200+).

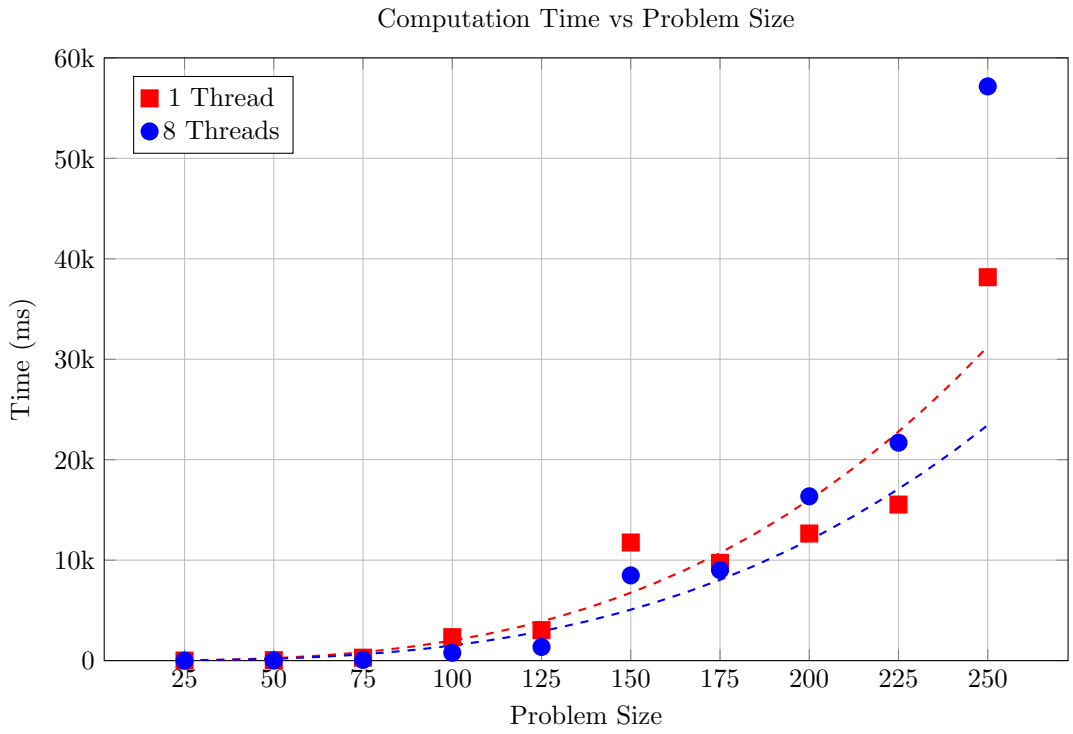


Figure 21: Hard Random Instance Performance - 20 to 250 variables

Scatter plot showing computation time versus problem size for 1-thread and 8-thread configurations. Both follow cubic trends (dashed lines), but 8 threads maintain consistent advantage until a size of 200.

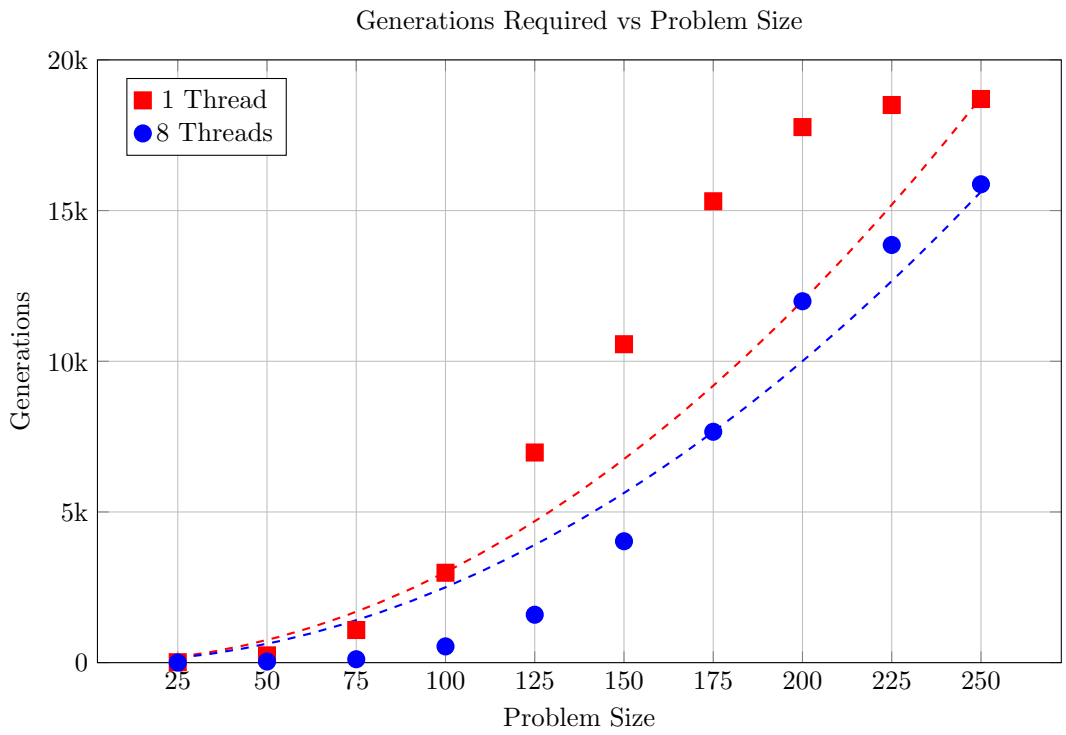


Figure 22: Hard Random Instance Generational Cost - 20 to 250 variables

Scatter plot showing generations required versus problem size. 8-thread configuration (blue circles) requires significantly fewer generations than single-threaded at all problem sizes. Both follow quadratic trends (dashed lines), with 8 threads maintaining a consistent 3-5x reduction in generations needed.

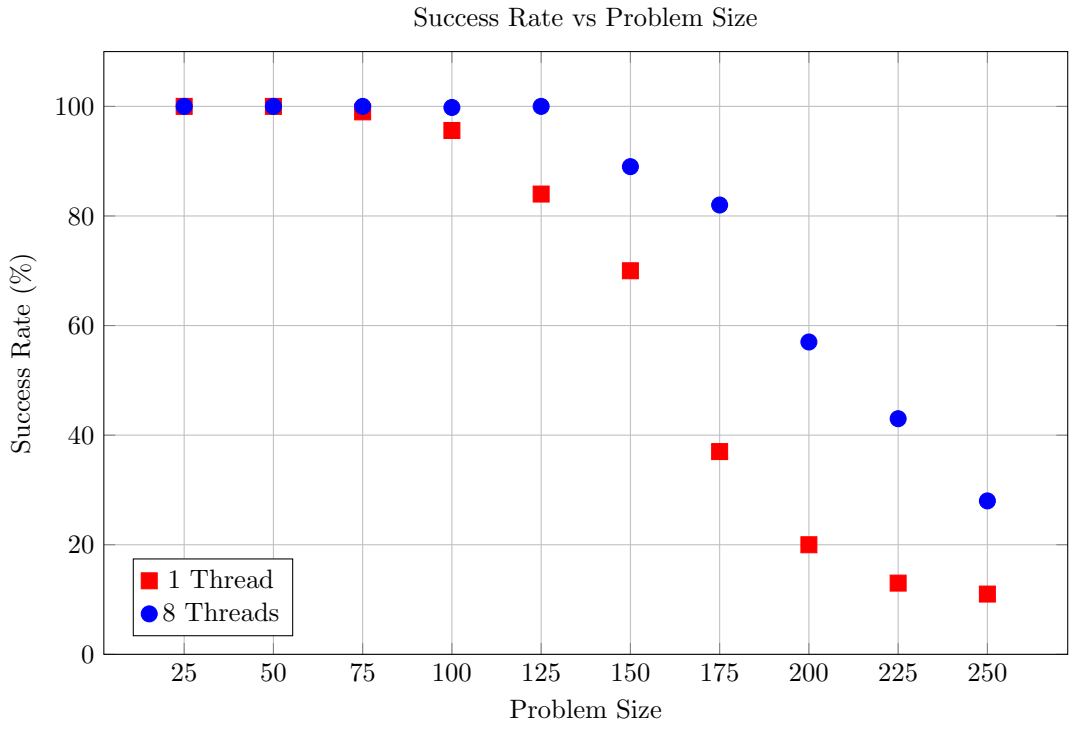


Figure 23: Hard Random Instance Success Rate - 20 to 250 variables

Success rate comparison showing 8 threads (blue) maintain higher reliability, especially at larger sizes (+37% at size=200). Both configurations degrade but 8 threads fail more gracefully.

6.1.1 Random 20

I begin with our easiest random evaluation test with 20 variables and 91 clauses. Performance was very good with a median of only 1.28ms for single thread and 1.21ms for the 8-thread case. I managed to solve all cases under 42ms for 8-thread 119ms for single-threaded and 90% of all cases under 6ms.

Threshold(ms)	**1 Thread**	**Bucket**	**8 Thread**	**Bucket**
< 0.5	67	67	57	57
< 1	385	318	416	359
< 1.5	576	191	573	157
< 2	679	103	636	63
< 2.5	737	58	726	90
< 3	786	49	790	64
< 3.5	808	22	844	54
< 4	833	25	869	25
< 4.5	855	22	899	30
< 5	869	14	916	17
< 5.5	890	21	934	18
< 6	900	10	944	10
< 6.5	905	5	956	12
< 7	910	5	965	9
< 7.5	924	14	974	9
< 8	926	2	982	8
< 8.5	932	6	983	1
< 9	938	6	985	2
< 9.5	942	4	987	2
< 10	946	4	987	0
All	1000	54	1000	13

Table 32: Performance Data by Threshold - Random 20 variables

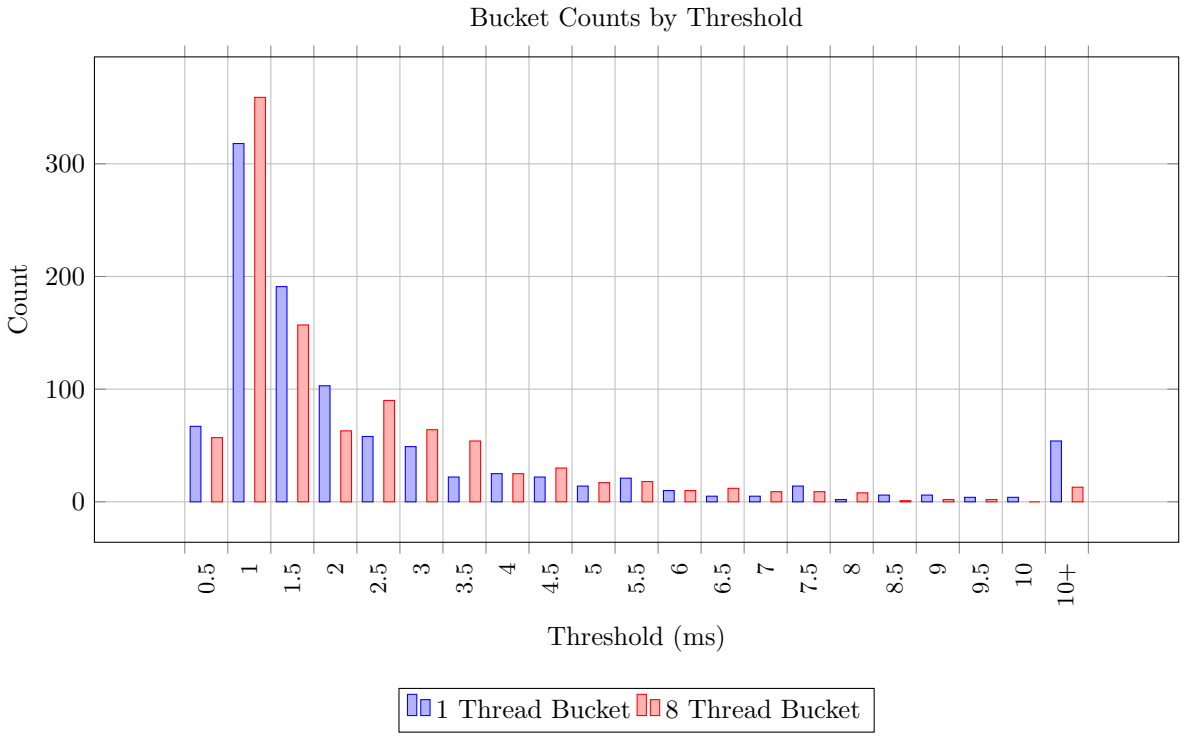


Figure 24: Comparison of bucket counts between 1 Thread and 8 Thread configurations for random 20 variables.

Metric(ms)	**1 Thread**	**8 Thread**
Median	1.28	1.21
Mean	2.93	2.13
Standard Deviation	5.93	2.70
Minimum	0.17	0.23
Maximum	118.95	42.46

Table 33: Performance Metrics Comparison (Random 20 variables)

6.1.2 Random 50

Next we look at an also fairly trivial test with 50 variables and 218 clauses. Performance was also very good with a median of only 16.65ms for single thread and 8.52ms for the 8-thread case. I managed to solve all cases under 328ms for 8-thread and 3096ms for single-threaded and 90% of all cases under 35ms.

Threshold(ms)	1 Thread	Bucket	8 Thread	Bucket
< 5	186	186	107	107
< 10	380	194	598	491
< 15	464	84	778	180
< 20	546	82	835	57
< 25	613	67	861	26
< 30	653	40	880	19
< 35	684	31	903	23
< 40	710	26	925	22
< 45	743	33	931	6
< 50	764	21	940	9
< 55	786	22	949	9
< 60	802	16	953	4
< 65	816	14	957	4
< 70	835	19	958	1
< 75	846	11	964	6
< 80	858	12	969	5
< 85	868	10	972	3
< 90	880	12	977	5
< 95	887	7	979	2
< 100	894	7	980	1
All	1000	106	1000	20

Table 34: Latency Distribution by Threshold - Random 50 variables

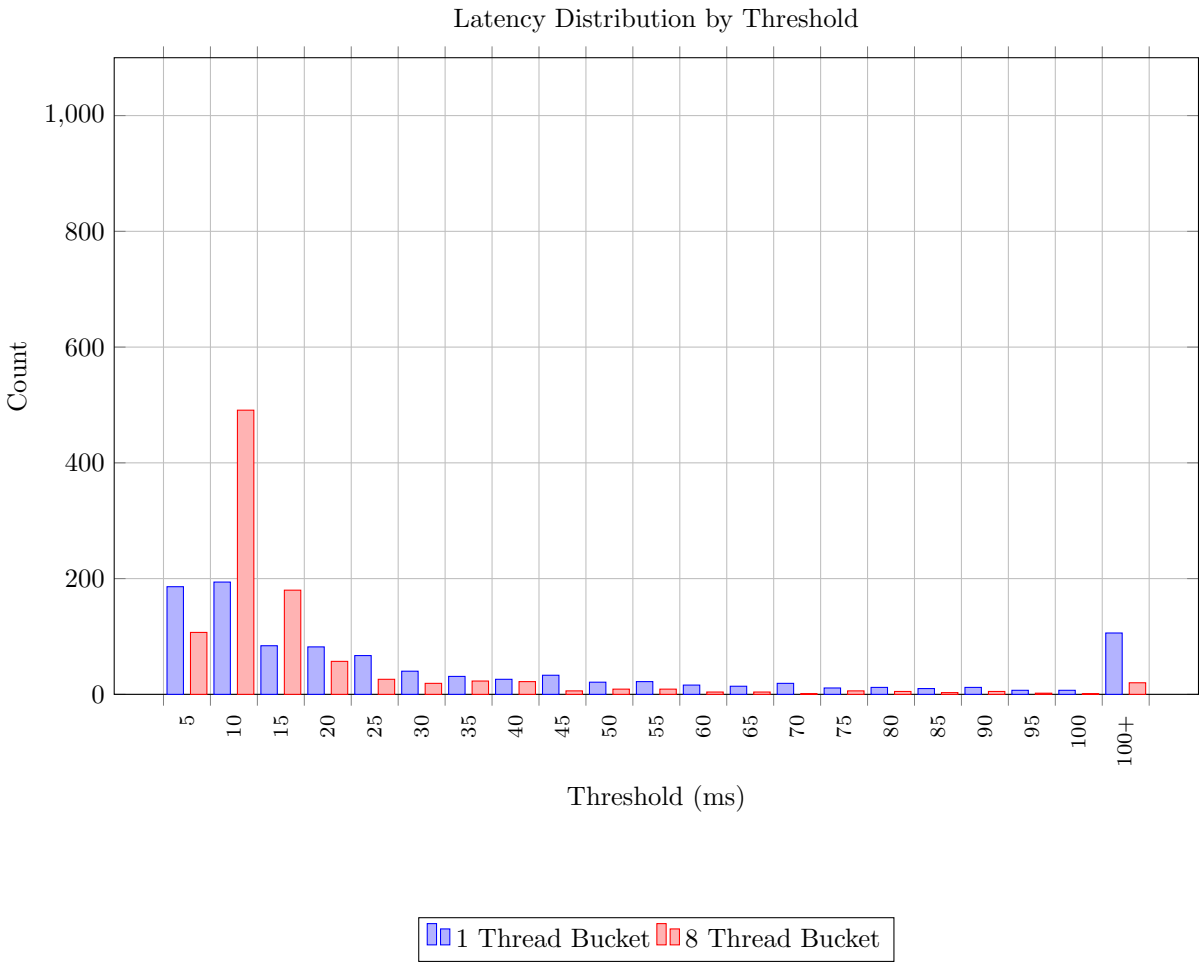


Figure 25: Comparison of latency bucket counts between 1 Thread and 8 Thread configurations for random 50 variables. The 8 Thread configuration shows significantly more requests completing under 10ms.

Metric (ms)	1 Thread	8 Thread
Median	16.65	8.52
Mean	47.74	16.46
Std. Deviation	130.23	26.07
Minimum	1.85	3.03
Maximum	3096.10	328.72

Table 35: Performance Metrics Comparison (Random 50 variables)

6.1.3 Random 75

The difficultly starts to increase with 75 variables and 325 clauses. Performance is still good with a median of only 78.84ms for single thread and 26.79ms for the 8-thread case. I managed to solve all cases under 885ms for 8-thread and 5151ms for single-threaded and 90% of all cases under 175ms.

Threshold(ms)	1 Thread	Bucket	8 Thread	Bucket
< 25	21	21	45	45
< 50	38	17	71	26
< 75	48	10	79	8
< 100	53	5	83	4
< 125	60	7	85	2
< 150	67	7	87	2
< 175	72	5	90	3
< 200	74	2	90	0
< 225	75	1	93	3
< 250	76	1	95	2
< 275	77	1	96	1
< 300	77	0	97	1
< 325	77	0	97	0
< 350	78	1	97	0
< 375	79	1	97	0
< 400	80	1	97	0
< 425	80	0	97	0
< 450	80	0	98	1
< 475	82	2	98	0
< 500	82	0	98	0
All	100	18	100	2

Table 36: Latency Distribution by Threshold (Random 75 variables)

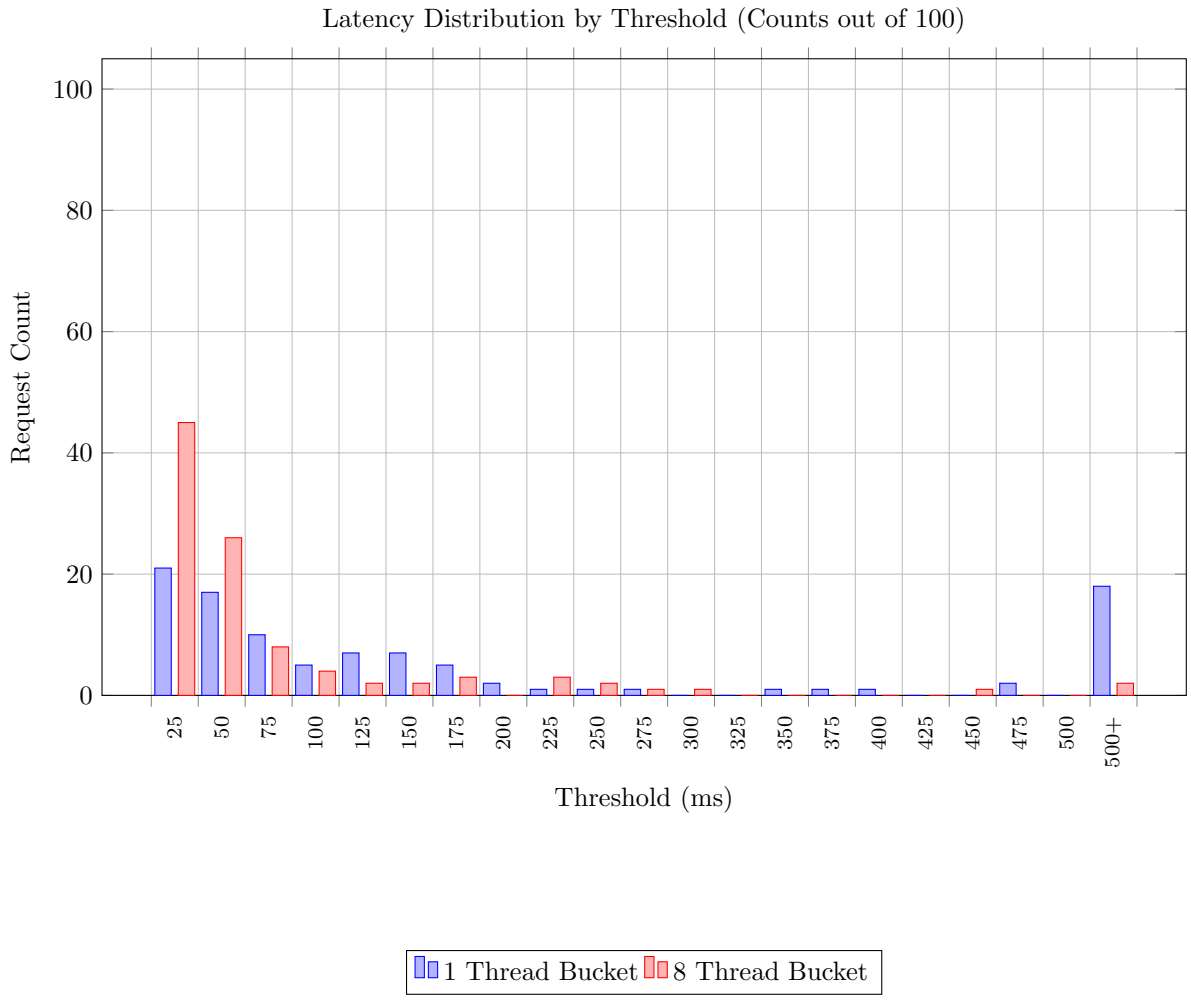


Figure 26: Comparison of latency distributions between thread configurations for random 75 variables. The 8-thread configuration shows significantly more requests completing under 50ms, while the 1-thread configuration has more gradual completion. The values represent new requests completing within each latency band.

Metric(ms)	**1 Thread**	**8 Thread**
Median	78.84	26.79
Mean	291.07	70.74
Std. Deviation	623.70	123.18
Minimum	5.14	7.94
Maximum	5151.13	884.75

Table 37: Performance Metrics Comparison (Random 75 variables)

6.1.4 Random 100

The difficultly increases again with 100 variables and 430 clauses. Most performance testing we have done to date is with this test so the results are quite interesting. Performance is again good with a median of only 730.17ms for single thread and 218.67ms for the 8-thread case. I managed to solve 70% of all cases under 500ms for the 8-thread case and 1750ms for the single case. I quietly was aiming to achieve under one second for this benchmark on average and so this result is quite pleasing.

Threshold(ms)	**1 Thread**	**Bucket**	**8 Thread**	**Bucket**
< 250	244	244	541	541
< 500	391	147	717	176
< 750	503	112	796	79
< 1000	574	71	850	54
< 1250	622	48	880	30
< 1500	657	35	901	21
< 1750	682	25	910	9
< 2000	712	30	923	13
< 2250	729	17	932	9
< 2500	751	22	938	6
< 2750	766	15	941	3
< 3000	781	15	945	4
< 3250	796	15	953	8
< 3500	810	14	961	8
< 3750	821	11	963	2
< 4000	835	14	969	6
< 4250	840	5	969	0
< 4500	847	7	970	1
< 4750	854	7	971	1
< 5000	865	11	975	4
All	1000	135	1000	25

Table 38: Latency Distribution by Threshold (Random 100 variables)

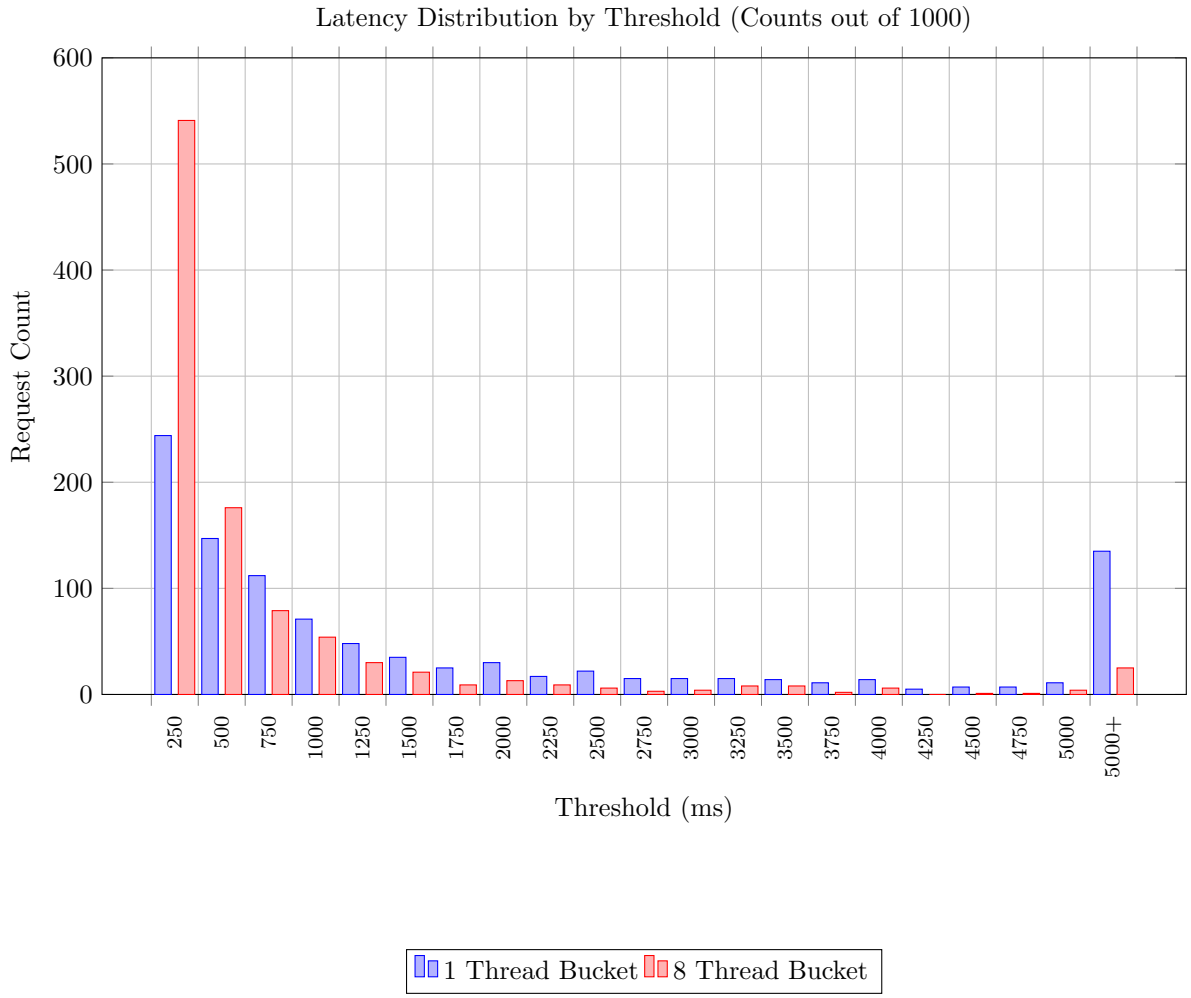


Figure 27: Latency distribution comparison for random 100 variables, showing 8-thread configuration completes 2-3× more requests at lower thresholds.

Metric(ms)	**1 Thread**	**8 Thread**
Median	730.17	218.67
Mean	2334.16	779.50
Std. Deviation	3730.87	2257.87
Minimum	19.98	25.76
Maximum	16748.05	28962.49

Table 39: Performance Metrics Comparison (Random 100 variables)

6.1.5 Random 125

Performance is still decent with an increased difficulty of 125 variables and 538 clauses, as well as accuracy. We still achieve sub 1 second performance for 8-threads with a median of 449.82ms and 1734.46 for a single thread. 60% of tests were completed under 0.75 seconds for 8-thread and 2.5 seconds for a single thread. 80% of tests were managed under 1.75 seconds.

Threshold(ms)	1 Thread	Bucket	8 Thread	Bucket
< 250	12	12	39	39
< 500	26	14	53	14
< 750	33	7	61	8
< 1000	38	5	68	7
< 1250	44	6	71	3
< 1500	47	3	77	6
< 1750	50	3	81	4
< 2000	54	4	83	2
< 2250	57	3	86	3
< 2500	59	2	88	2
< 2750	61	2	89	1
< 3000	64	3	90	1
< 3250	64	0	90	0
< 3500	66	2	90	0
< 3750	69	3	91	1
< 4000	70	1	93	2
< 4250	72	2	93	0
< 4500	75	3	93	0
< 4750	78	3	93	0
< 5000	78	0	93	0
All	100	22	100	7

Table 40: Latency Distribution by Threshold (Random 125 variables)

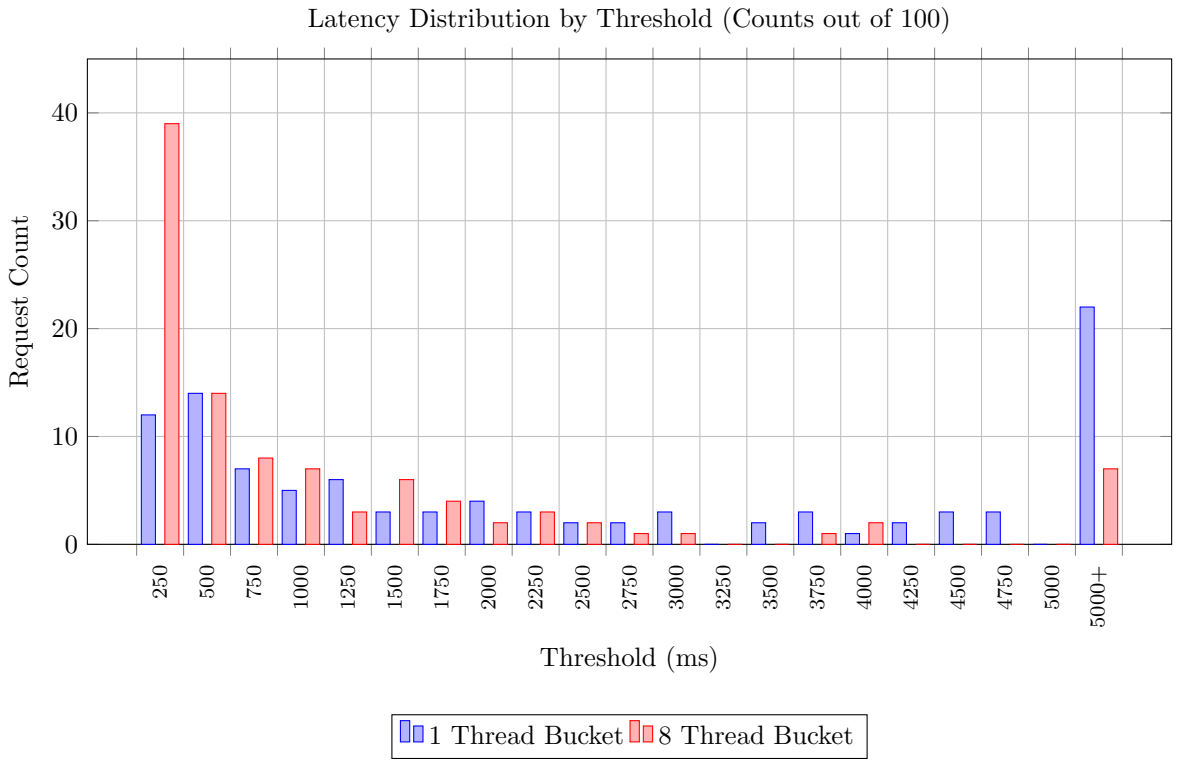


Figure 28: Latency distribution for random 125 variables showing 8-thread's 3× advantage in lowest bucket (under 250ms). Both configurations reach 100% completion, but with different latency profiles.

Metric(ms)	1 Thread	8 Thread
Median	1734.46	449.82
Mean	3037.16	1359.39
Std. Deviation	3100.36	2537.27
Minimum	26.80	31.42
Maximum	9496.92	14448.62

Table 41: Performance Metrics Comparison (Random 125 variables)

6.1.6 Random 150

We saw in our earlier summary that some accuracy starts falling at 150 variables and 645 clauses. Performance also starts to weaken to a median of 2.1 seconds for 8-threads and 10.4 seconds for a single thread. We can successfully solve 71% of problems within 5 seconds on 8-threads but only 32% for the single case. Some

Performance is still decent with an increased difficulty of 125 variables and 538 clauses, as well as accuracy. We still achieve sub 1 second performance for 8-threads with a median of 449.82ms and 1734.46 for a single thread. 60% of tests were completed under 0.75 seconds for 8-thread and 2.5 seconds for a single thread. 80% of tests were managed under 1.75 seconds.

Threshold(ms)	1 Thread	Bucket	8 Thread	Bucket
< 1000	10	10	37	37
< 2000	19	9	49	12
< 3000	21	2	61	12
< 4000	27	6	68	7
< 5000	32	5	71	3
< 6000	36	4	74	3
< 7000	41	5	76	2
< 8000	43	2	77	1
< 9000	47	4	78	1
< 10000	48	1	79	1
< 11000	53	5	80	1
< 12000	56	3	80	0
< 13000	58	2	80	0
< 14000	61	3	81	1
< 15000	61	0	81	0
< 16000	62	1	82	1
< 17000	62	0	82	0
< 18000	62	0	82	0
< 19000	63	1	82	0
< 20000	66	3	82	0
All	100	34	100	18

Table 42: Latency Distribution by Threshold (Random 150 variables)

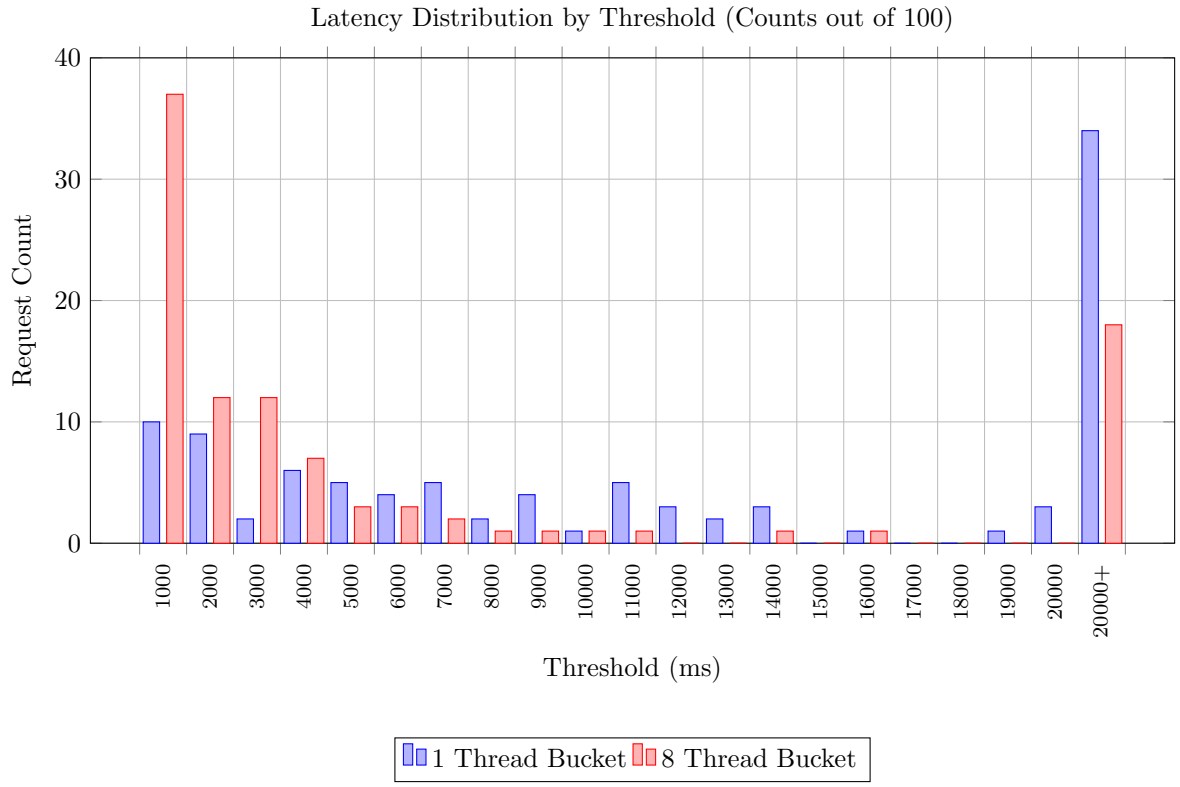


Figure 29: Random 150 variables - 8-thread shows 3.7× better performance under 1s. While both complete all requests, 8-thread achieves this with significantly better low-latency performance.

Metric(ms)	**1 Thread**	**8 Thread**
Median	10,416.89	2,096.82
Mean	11,757.33	8,481.60
Std. Deviation	8,602.30	13,684.33
Minimum	185.00	103.88
Maximum	23,580.40	43,188.59

Table 43: Performance Metrics Comparison (Random 150 variables)

6.1.7 Random 175

As accuracy falls so does performance. At 175 variables and 753 clauses we now weaken to a median of 5.6 seconds for 8-threads and 12.4 seconds for a single thread. We can successfully solve 71% of problems within 5 seconds on 8-threads but only 32% for the single case. Of note is our maximum performance times being more severe on the parallel case, likely due to the number of cases we couldn't solve in the maximum number of generations (18% unsolved in time). With a median under 6 seconds I would say that the performance is acceptable but would have liked better.

Threshold	1 Thread	Bucket	8 Thread	Bucket
<1s	7	7	19	19
<2s	13	6	32	13
<3s	16	3	37	5
<4s	18	2	41	4
<5s	19	1	47	6
<6s	25	6	51	4
<7s	27	2	53	2
<8s	29	2	57	4
<9s	31	2	62	5
<10s	33	2	65	3
<11s	35	2	67	2
<12s	37	2	68	1
<13s	89	52	68	0
<14s	100	11	71	3
<15s	100	0	74	3
<16s	100	0	75	1
<17s	100	0	77	2
<18s	100	0	78	1
<19s	100	0	78	0
<20s	100	0	79	1
All	100	0	100	21

Table 44: Latency Distribution by Threshold (Random 175 variables)

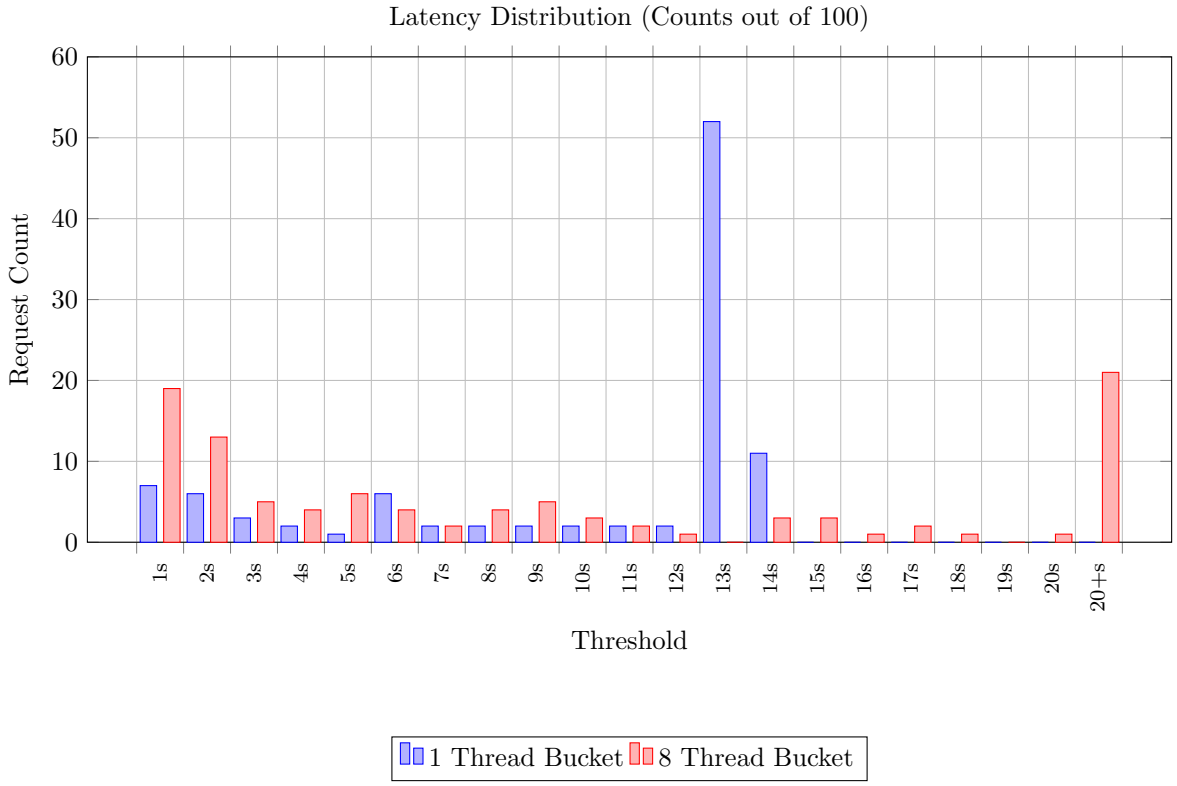


Figure 30: Random 175 variables, 8-thread shows better early performance, while 1-thread has 52% completions in the 12-13s bucket.

Metric(ms)	1 Thread	8 Thread
Median	12,387.27	5,654.57
Mean	9,725.94	9,012.66
Std. Deviation	4,438.61	8,543.96
Minimum	300.23	112.58
Maximum	13,632.73	24,446.48

Table 45: Performance Metrics Comparison - Random 175 variables

6.1.8 Random 200 to Random 250

Once 200 variables were reached, single thread execution began to outperform, although in both cases accuracy and execution times fell significantly. For 200 variables and 860 clauses only 57% of cases were solved in time with a median time of 20 seconds for 8-thread and 14 seconds for single thread (with only 20% solved in time).

This once again got worse for 225 variables and 960 clauses, requiring a median of 16.6 seconds and 30.5 seconds for 1 and 8 threads respectively. Accuracy also fell with only 13% and 43% solved respectively.

Finally our worse results came for 250 variables and 1065 clauses with only 11% and 28% solved at a time of 40.6 and 7 seconds respectively.

Threshold	1 Thread	Bucket	8 Thread	Bucket
<1s	3	3	8	8
<2s	3	0	15	7
<3s	5	2	20	5
<4s	6	1	26	6
<5s	6	0	27	1
<6s	9	3	27	0
<7s	10	1	28	1
<8s	12	2	32	4
<9s	15	3	37	5
<10s	17	2	40	3
<11s	18	1	42	2
<12s	19	1	42	0
<13s	20	1	44	2
<14s	45	25	45	1
<15s	96	51	46	1
<16s	100	4	48	2
<17s	100	0	49	1
<18s	100	0	49	0
<19s	100	0	49	0
<20s	100	0	49	0
All	100	0	100	51

Table 46: Latency Distribution by Threshold (Random 200 variables)

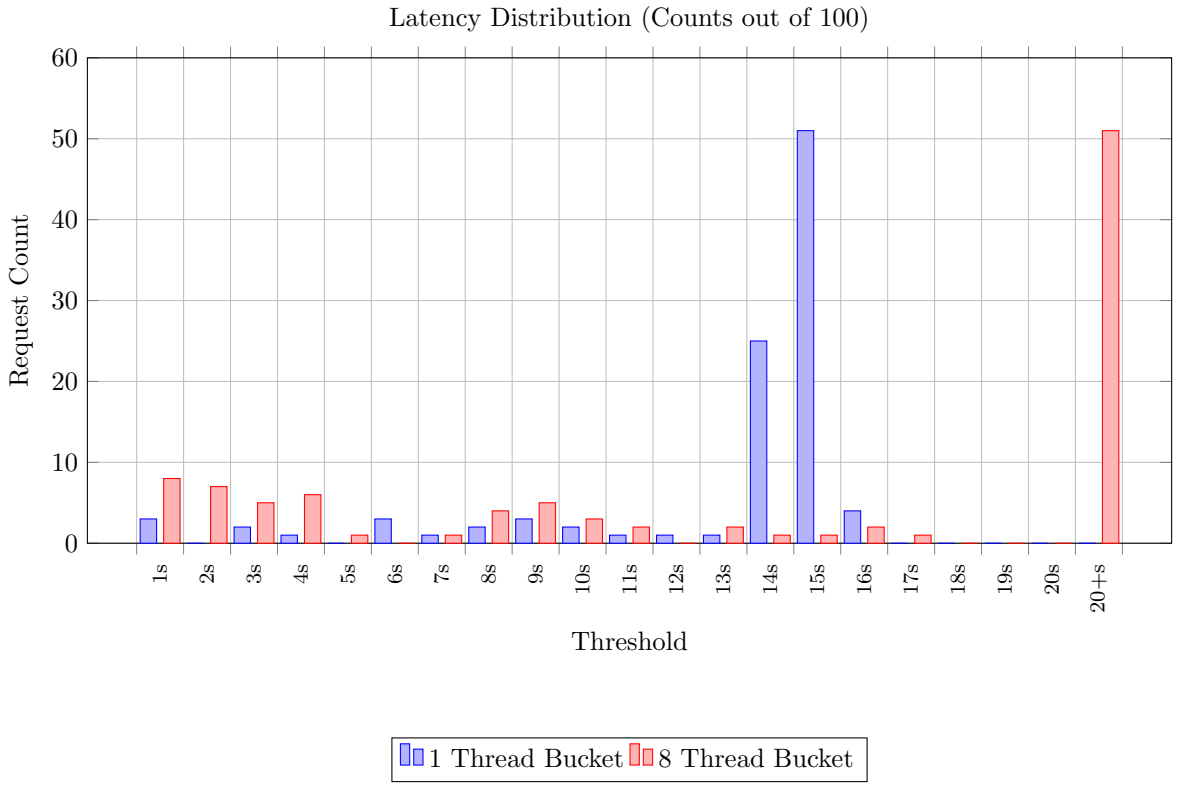


Figure 31: Random 200 variables, 1-thread shows dramatic 51% completion at 14-15s, while 8-thread has steady progress with 51% in final bucket.

Metric(ms)	**1 Thread**	**8 Thread**
Median	14,042.72	20,057.92
Mean	12,653.52	16,355.39
Std. Deviation	3,577.84	11,022.38
Minimum	185.5	175.32
Maximum	15,216.03	29,510.53

Table 47: Performance Metrics Comparison - Random 200 variables

Threshold	1 Thread	Bucket	8 Thread	Bucket
<2.5s	4	4	14	14
<5s	7	3	22	8
<7.5s	8	1	25	3
<10s	9	1	27	2
<12.5s	9	0	31	4
<15s	11	2	33	2
<17.5s	93	82	35	2
<20s	100	7	36	1
<22.5s	100	0	39	3
<25s	100	0	39	0
<27.5s	100	0	39	0
<30s	100	0	43	4
<32.5s	100	0	99	56
<35s	100	0	100	1

Table 48: Latency Distribution by Threshold (Random 225 variables)

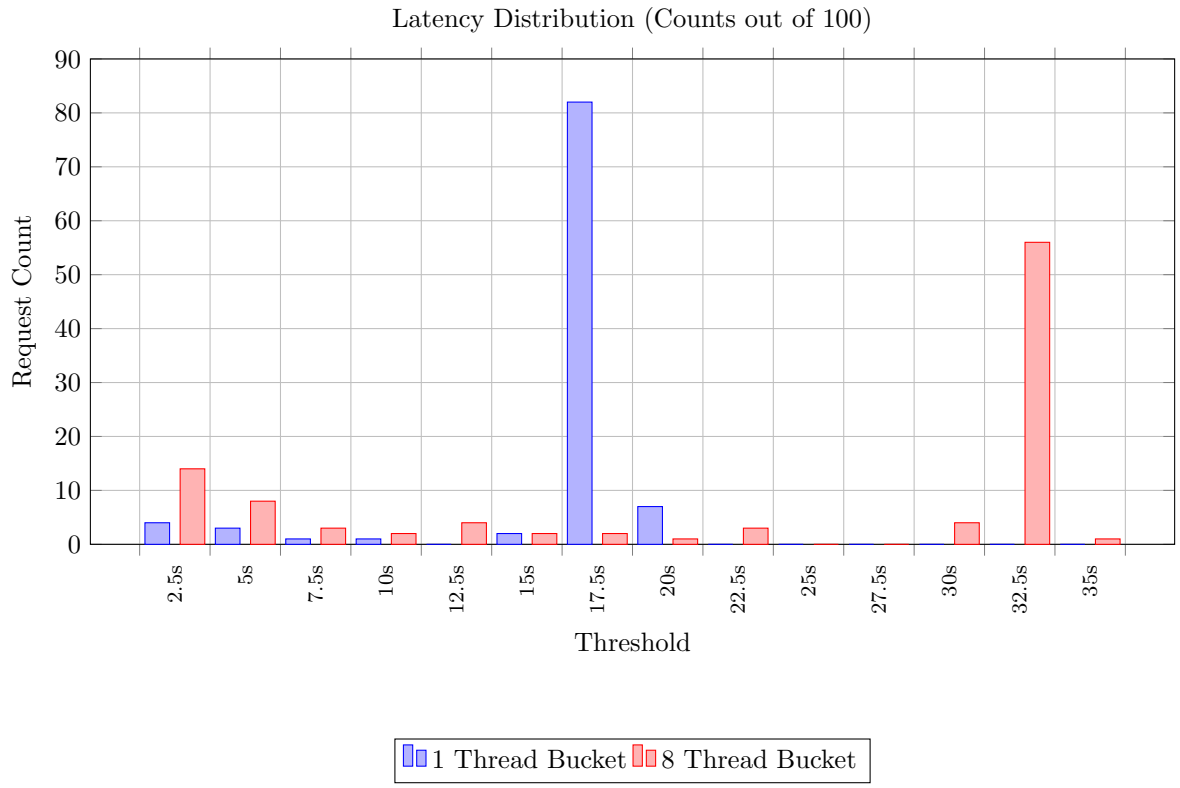


Figure 32: Random 225 variables - Dramatic completion spikes: 1-thread at 15-17.5s (82%), 8-thread at 30-32.5s (56%). 8-thread shows better early performance.

Metric(ms)	1 Thread	8 Thread
Median	16,647.79	30,546.58
Mean	15,534.11	21,696.02
Std. Deviation	3,906.14	12,460.59
Minimum	229.59	104.98
Maximum	18,047.67	34,000.10

Table 49: Performance Metrics Comparison - Random 225 variables

Threshold	1 Thread	Bucket	8 Thread	Bucket
<2.5s	0	0	3	3
<5s	2	2	5	2
<7.5s	5	3	6	1
<10s	5	0	10	4
<12.5s	5	0	13	3
<15s	6	1	16	3
<17.5s	6	0	17	1
<20s	6	0	18	1
<22.5s	8	2	18	0
<25s	8	0	22	4
<27.5s	8	0	23	1
<30s	8	0	23	0
<32.5s	9	1	23	0
<35s	10	1	23	0
<37.5s	11	1	24	1
<40s	26	15	24	0
<42.5s	95	69	24	0
<45s	99	4	24	0
<47.5s	100	1	24	0
<50s	100	0	24	0
All	100	0	100	76

Table 50: Latency Distribution by Threshold (Random 250 variables)

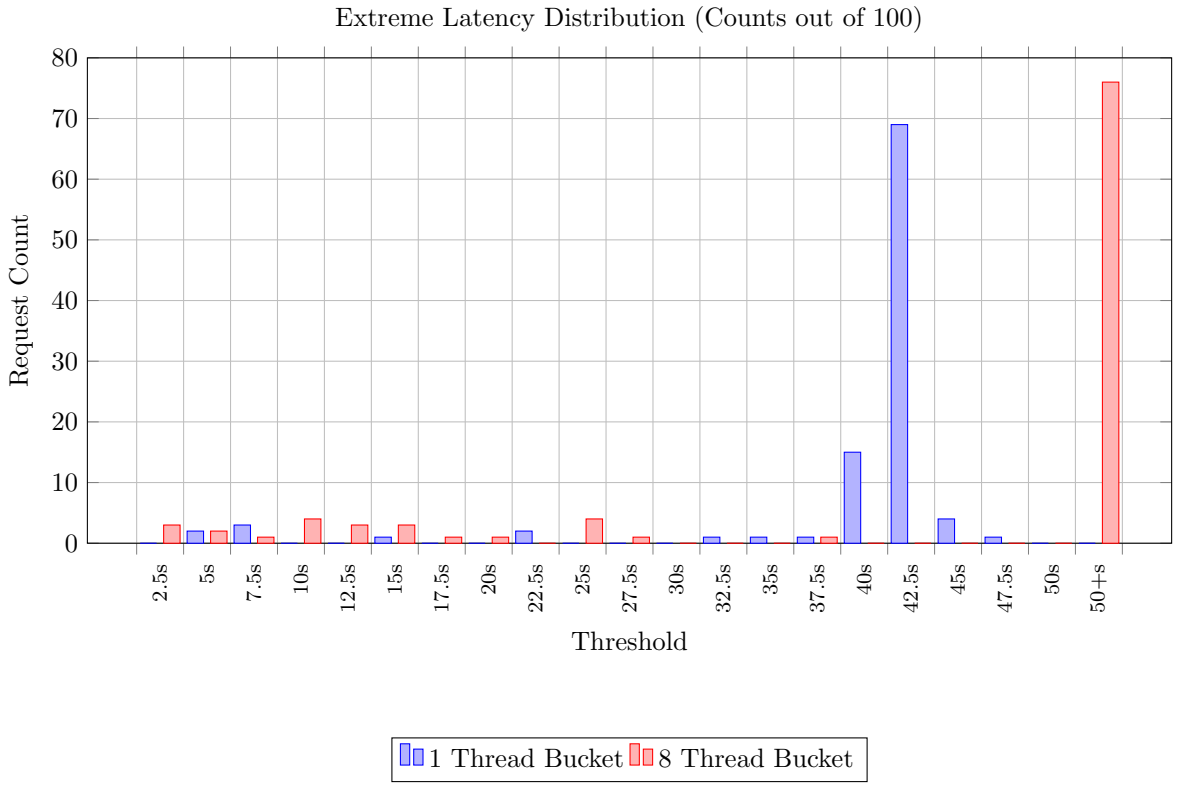


Figure 33: Random 250 variables - Dramatic patterns: 1-thread has 69% completions at 40-42.5s, while 8-thread completes 76% of requests in final processing stage.

Metric(ms)	1 Thread	8 Thread
Median	40,581.99	7,023.83
Mean	38,168.09	57,161.99
Std. Deviation	8,533.06	25,349.21
Minimum	4,049.55	478.07
Maximum	45,576.38	74,575.08

Table 51: Performance Metrics Comparison - random 250 variables

6.2 Structured Instances - 90 to 300 variables

As for random, I begin by summarising all structured instances and then proceed to break down the performance for each different sized problem. The algorithm performed well over the smaller problems, able to solve 965 of the 1000 problems with 150 variables with a single thread and 1000 using 8 threads, within the maximum allowed number of generations. 8 threads performed very well on 150 variables and was able to complete within 0.69 seconds on average. The single thread case was comparably slower as expected, completing in 2.6 seconds on average. A summary is given in Table 52.

The trend shown in Figure 34 shows that performance of 8 threads consistently outperformed a single thread for all problem sizes. Both implementations perform very well up to 150 variables and then computational time appears to change exponentially, inline with the random case. 8-threads was able to remain quite accurate up to 225 variables (88%), completing in 6.8 seconds on average, whereas 1-thread could only achieve 37% accuracy in 7.1 seconds. At 300 variables the success rate and performance suffered significantly with only 4% and 11% achieved for 1 and 8 threads respectively. At this point I decided it was not worth trying additional problem sizes.

For completeness I also demonstrate the performance in terms of number of generations (Figure 35) and success rates (Figure 36). While not the metric used for this evaluation it is interesting to see a similar exponential trend with generational cost and a sudden drop in success at 225 variables.

As for random, I will break down further metrics per problem size and evaluate the variance needed in solving each one.

Threads	Size	Success (%)	Generations	Restarts	Time (ms)
1	90	100.0%	235	2.90	106
1	150	96.5%	4,072	38.70	2,597
1	225	37.0%	15,782	111.46	7,141
1	300	4.0%	19,655	105.00	20,627
8	90	100.0%	44	0.80	54
8	150	100.0%	559	4.58	688
8	225	88.0%	7,957	55.65	6,814
8	300	11.0%	18,583	110.00	11,610

Table 52: Structured Instances Performance Summary

Performance metrics showing 8-thread configuration consistently outperforms single-threaded execution. At size=225, 8 threads achieve 88% success in 6,814ms vs 37% in 7,141ms with 1 thread.

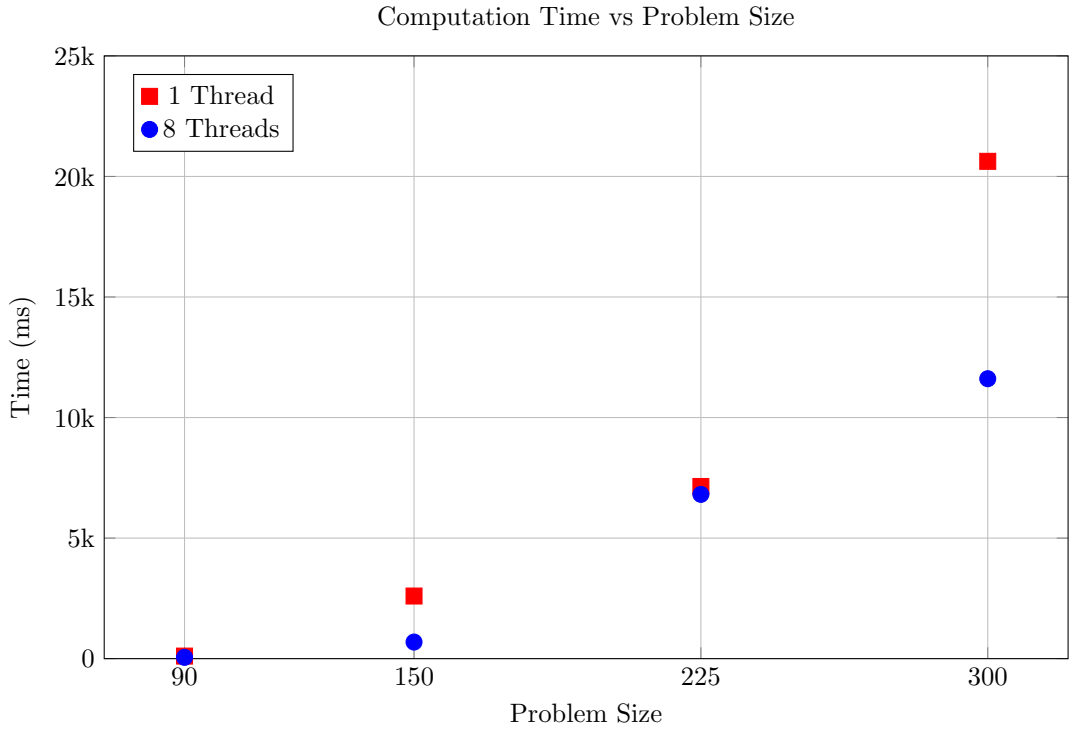


Figure 34: Computation time comparison for structured instances, showing 8 threads maintain consistent performance advantage across all problem sizes.

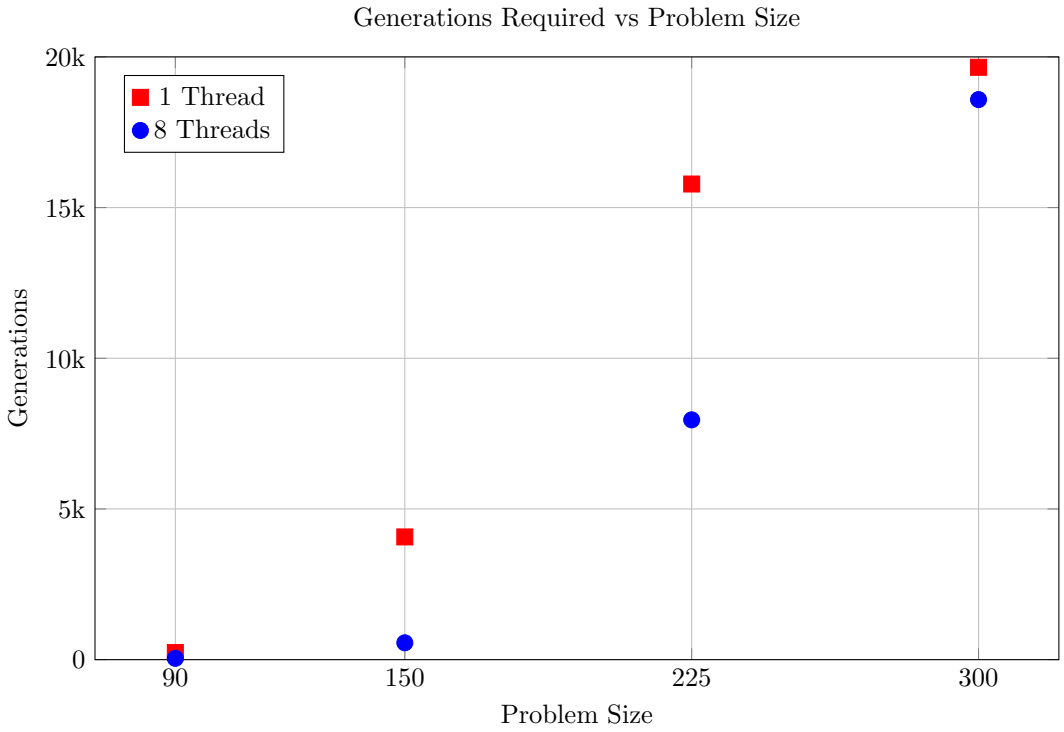


Figure 35: Structured Instances - Generation Comparison

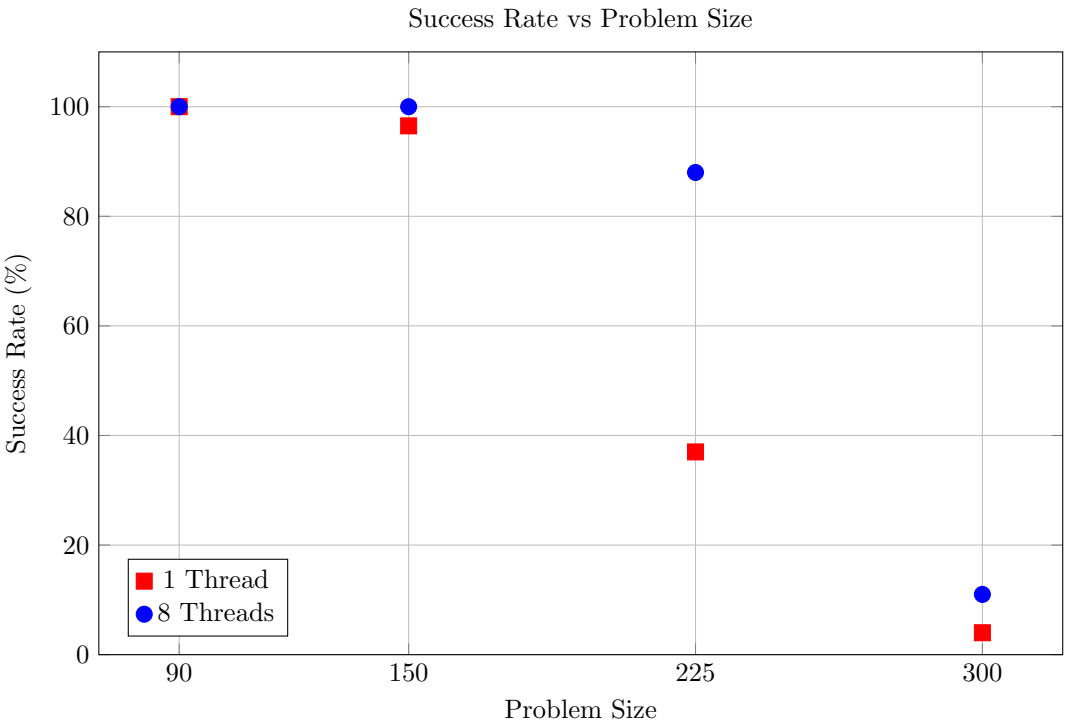


Figure 36: Structured Instances: Success rate comparison showing 8 threads (blue) maintain higher reliability than 1 thread (red) as problem size increases. At the threshold (size=200) 8 threads show a significant advantage (+51% at size=225).

6.2.1 Structured 90

I begin with our easiest structured evaluation test with 90 variables and 300 clauses. Performance was very good with a median of only 61.94ms for single thread and 37.54ms for the 8-thread case. I managed

to solve 90% of cases under 100ms for 8-thread and 240ms for single-threaded and both cases achieved 100% accuracy.

Threshold(ms)	1 Thread	1 Thread Bucket	8 Thread	8 Thread Bucket
<20	16	16	1	1
<40	34	18	60	59
<60	46	12	80	20
<80	56	10	87	7
<100	61	5	91	4
<120	70	9	93	2
<140	74	4	94	1
<160	78	4	95	1
<180	81	3	96	1
<200	85	4	97	1
<220	87	2	97	0
<240	90	3	97	0
<260	91	1	97	0
<280	91	0	98	1
<300	93	2	99	1
<320	93	0	99	0
<340	93	0	99	0
<360	94	1	100	1
<380	97	3	100	0
<400	98	1	100	0
All	100	2	100	0

Table 53: Latency Distribution by Threshold (Structured 90 variables)

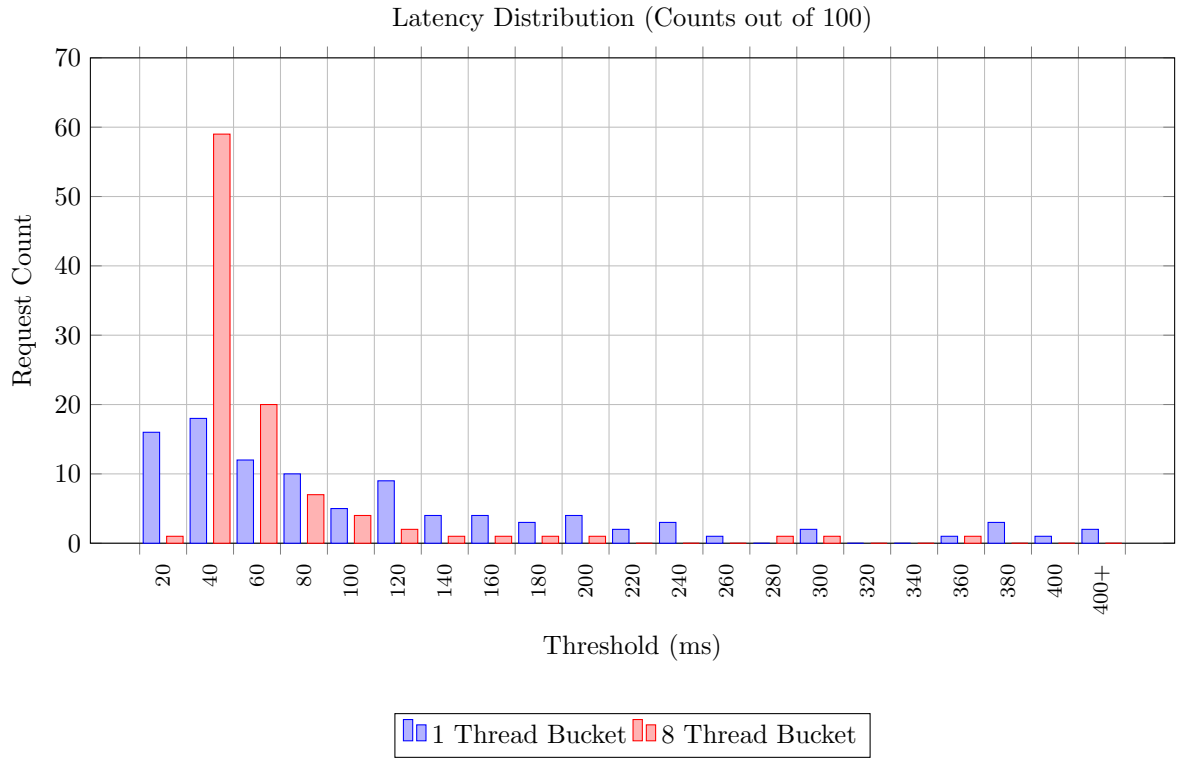


Figure 37: Structured 90 variables, 8-thread shows dramatic 59 completions in 20-40ms range, while 1-thread has more gradual distribution.

Metric(ms)	1 Thread	8 Thread
Median	61.94	37.54
Mean	106.00	54.41
Std. Deviation	109.03	53.12
Minimum	9.61	15.09
Maximum	608.85	351.38

Table 54: Performance Metrics Comparison for Structured 90 variables

6.2.2 Structured 150

Next we look at a more challenging benchmark test with 150 variables and 545 clauses. Performance was also good with a median of 1,467ms for single thread and 372ms for the 8-thread case. I managed to solve all 80% of cases under 1 second for 8-thread but required 4 seconds for the single-threaded case. Accuracy was still not really a problem with 100% accuracy and 96.5% accuracy for 8 and 1 threads respectively.

Threshold (ms)	1 Thread	1 Thread Bucket	8 Thread	8 Thread Bucket
<250	127	127	374	374
<500	239	112	584	210
<750	328	89	722	138
<1000	402	74	801	79
<1250	468	66	856	55
<1500	508	40	896	40
<1750	566	58	911	15
<2000	616	50	932	21
<2250	644	28	949	17
<2500	668	24	957	8
<2750	693	25	965	8
<3000	714	21	971	6
<3250	732	18	979	8
<3500	752	20	984	5
<3750	777	25	985	1
<4000	798	21	986	1
<4250	818	20	987	1
<4500	829	11	989	2
<4750	841	12	989	0
<5000	849	8	990	1
All	999	150	999	9

Table 55: Latency Distribution by Threshold (Structured 150 variables)

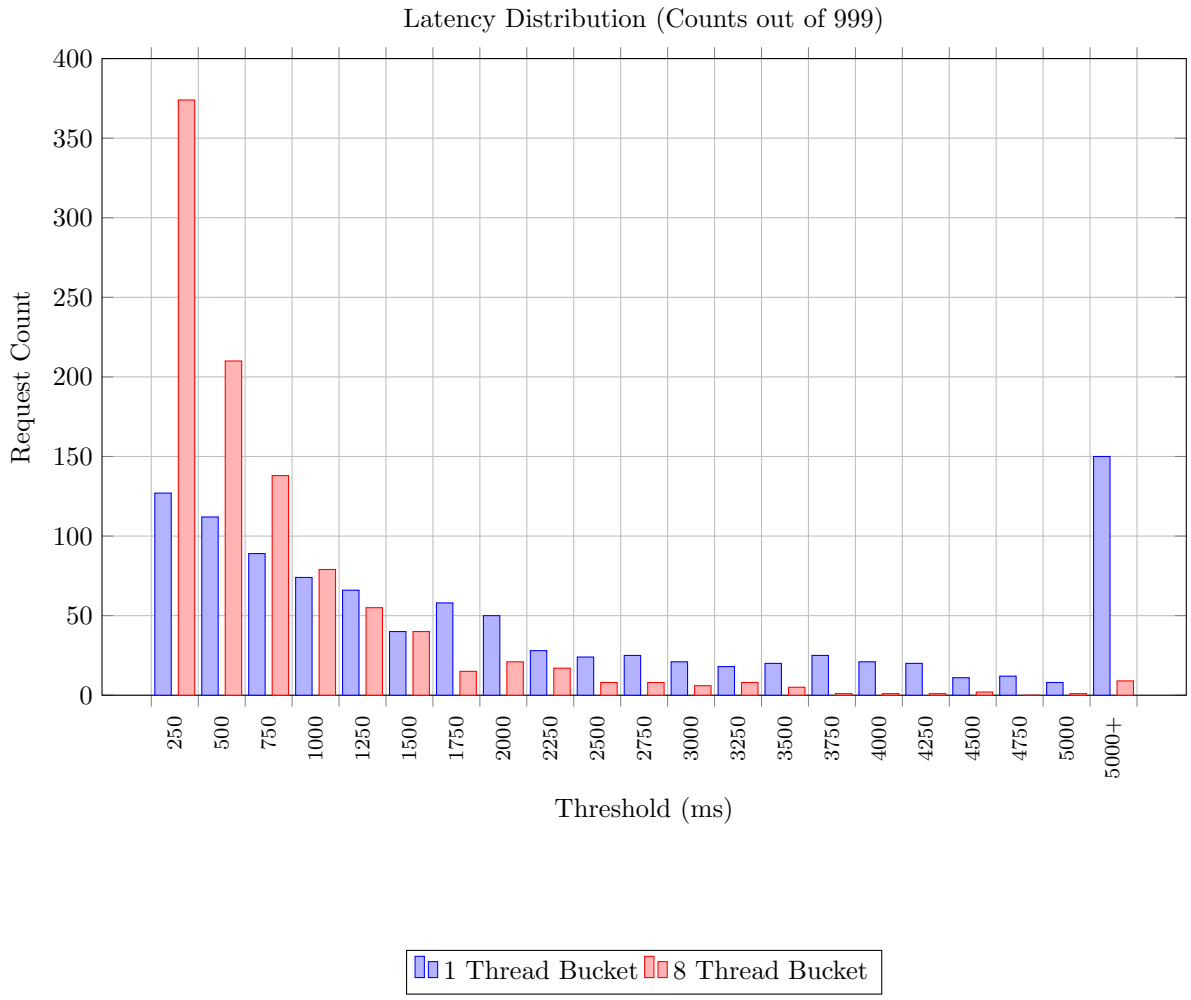


Figure 38: Structured 150 variables: 8-thread shows 3× more completions under 250ms. Both configurations complete all requests with different latency profiles.

Metric(ms)	1 Thread	8 Thread
Median	1426.92	371.59
Mean	2596.83	688.24
Std. Deviation	3094.19	949.41
Minimum	41.73	53.11
Maximum	14 278.23	10 751.03

Table 56: Performance Metrics Comparison (Structured 150 variables)

6.2.3 Structured 225

Like with random, as accuracy falls so does performance. At 225 variables and 840 clauses we now weaken to a median of 4.4 seconds for 8-threads and 8.7 seconds for a single thread. We can successfully solve 50% of problems within 4 seconds on 8-threads but only 23% for the single case. As with random, our maximum performance times are more severe on the parallel case, likely due to the number of cases we couldn't solve in the maximum number of generations (12% unsolved in time). With a median under 5 seconds I would say that the performance is acceptable but would have liked better.

Threshold (ms)	1 Thread	1 Thread Bucket	8 Thread	8 Thread Bucket
<500	1	1	9	9
<1000	4	3	17	8
<1500	6	2	23	6
<2000	8	2	30	7
<2500	11	3	35	5
<3000	13	2	40	5
<3500	18	5	46	6
<4000	23	5	49	3
<4500	26	3	51	2
<5000	28	2	55	4
<5500	29	1	55	0
<6000	31	2	58	3
<6500	31	0	61	3
<7000	33	2	63	2
<7500	33	0	63	0
<8000	33	0	63	0
<8500	40	7	65	2
<9000	68	28	66	1
<9500	91	23	66	0
<10000	96	5	68	2
All	100	4	100	32

Table 57: Latency Distribution by Threshold (Structured 225 variables)

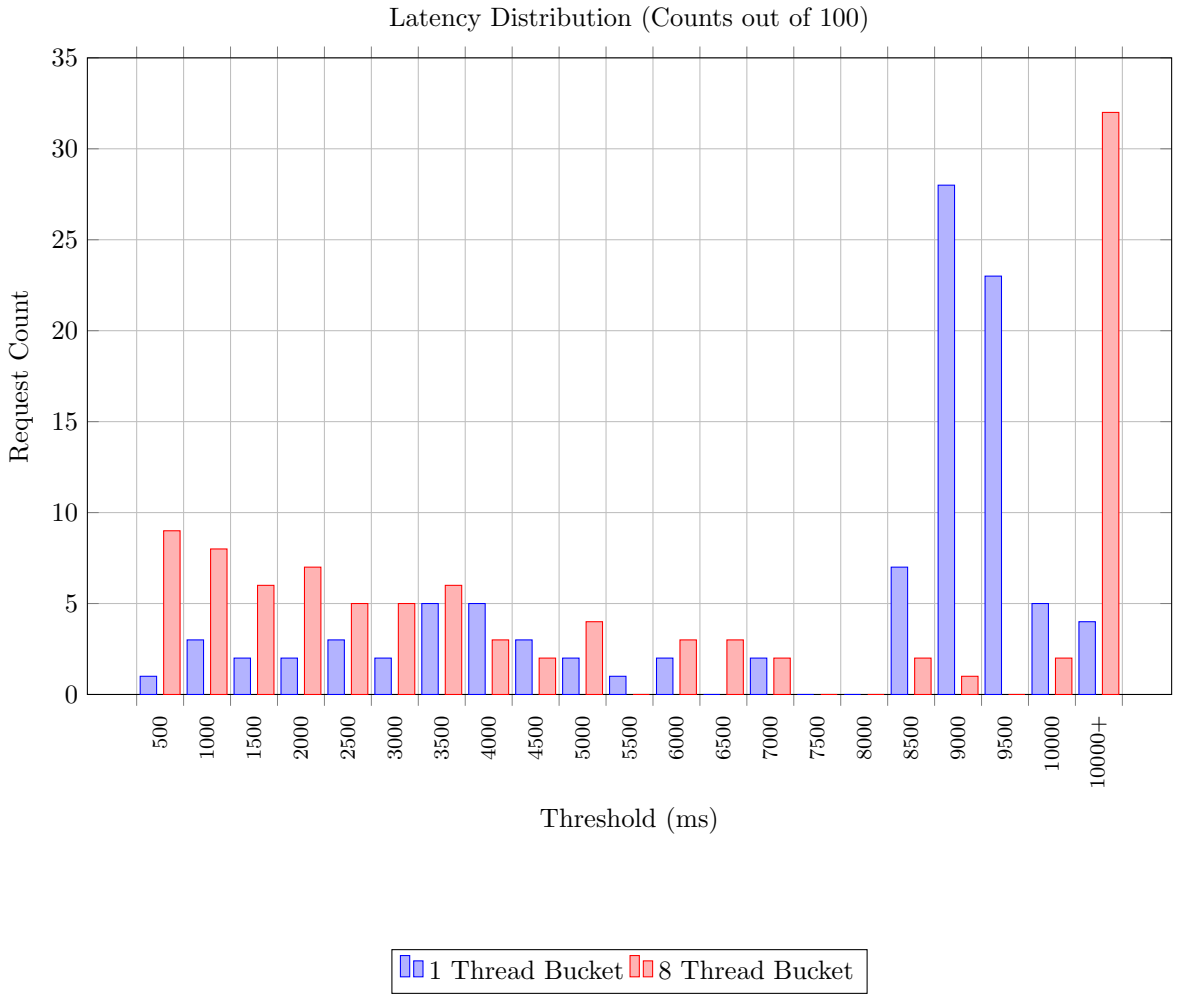


Figure 39: Structured 225 variables: Dual-phase pattern: 8-thread leads early (9× better under 500ms), 1-thread catches up later with 28% spike at 8-9s.

Metric(ms)	1 Thread	8 Thread
Median	8747.58	4413.57
Mean	7140.70	6813.95
Std. Deviation	2877.01	6005.59
Minimum	233.21	165.61
Maximum	10 500.16	17 665.38

Table 58: Performance Metrics Comparison (Structured 225 variables)

6.2.4 Structured 300

At a 11% success rate for 300 variables and 1117 constraints we wont be winning any SAT solver awards in a hurry. The metrics are severely distorted by the number of failures and I only managed 10 successes out of 100 under 10 seconds.

Threshold(ms)	1 Thread	Bucket	8 Thread	Bucket
<5s	2	2	4	4
<10s	3	1	7	3
<15s	100	97	10	3
<20s	100	0	10	0
<25s	100	0	100	90

Table 59: Bimodal Latency Distribution (Structured 300 variables)

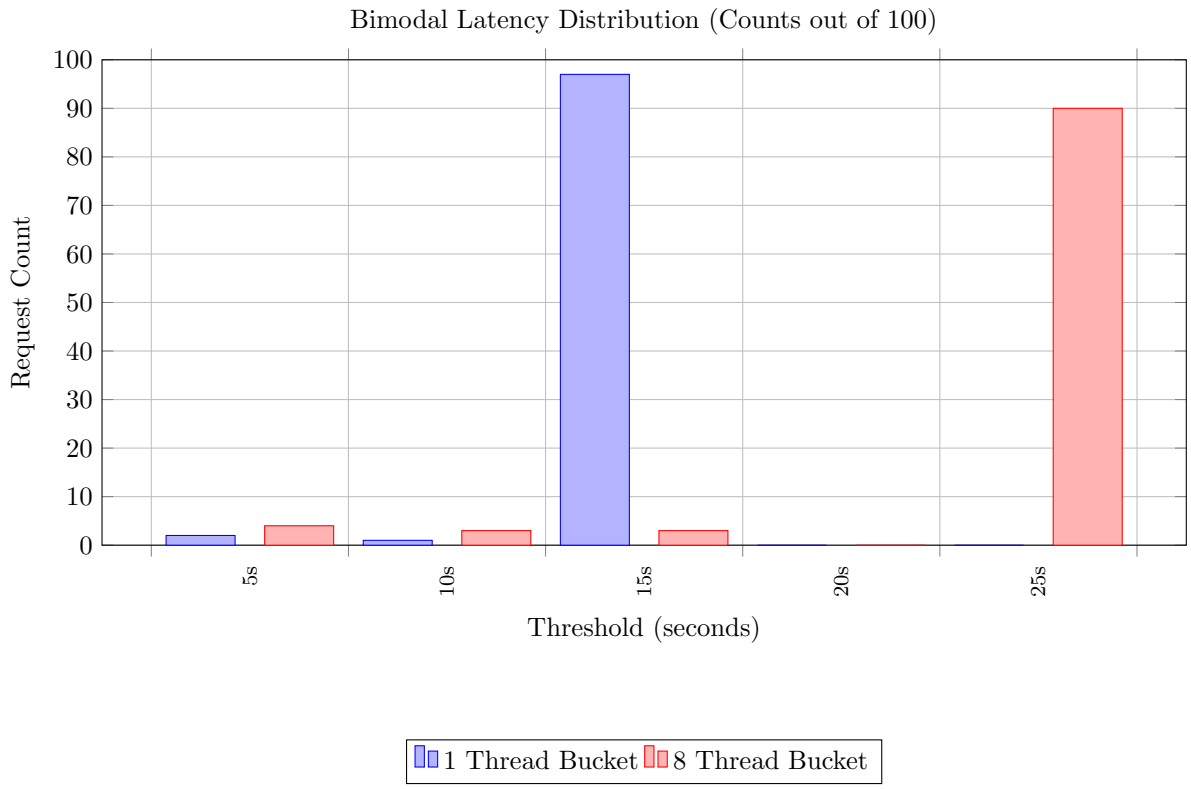


Figure 40: Structured 300 variables shows very limited success and in 90% of cases reaches the maximum generations allowed.

Metric(ms)	1 Thread	8 Thread
Median	11,742.50	22,016.28
Mean	11,609.58	20,626.76
Std. Deviation	1,282.02	4,806.77
Minimum	2,328.15	1,033.77
Maximum	12,642.72	23,388.00

Table 60: Performance Metrics Comparison (Structured 300 variables)

6.3 Comparison

Random and structured instances behaved similarly and achieved comparable results. The algorithm I developed works equally well over random and structured cases (that we evaluated), yet performs poorly as we increase the search space required. The parallel strategy worked well in small to medium problems but didn't manage to handle larger problems well. My suspicion is that most execution testing focused on this range and this is where the success is evident.

The following table gives a full summary of the median times needed for both implementations, followed by a logarithmic graph to demonstrate the pattern. Performance patterns are very similar in both random and structured cases, and the graph shows we performed a little better on structured instances.

MiniSAT and WalkSAT were also included in running the same tests, admittedly at a much higher performance. In all cases they were able to solve each problem successfully, except for one instance when walkSAT timed out on the structured 300 variable case. The results are interesting, highlighting that the stochastic solver (WalkSAT) works better over random instances and the complete solver (MiniSAT) works best over more structured instances. Although my intention of this study was not necessarily to compete with well-known standards, its nice to see my solver performs relatively well, trending inline with these standards, albeit probably ten times slower.

Type	Size	1 Thread	8 Thread	MiniSAT	WalkSAT
Random	20.00	1.28	1.21	0.20	0.09
Random	50.00	16.65	8.52	1.34	0.72
Random	75.00	78.84	26.79	4.21	2.13
Random	100.00	730.17	218.67	37.59	18.65
Random	125.00	1,734.46	449.82	48.40	34.88
Random	150.00	10,416.89	2,096.82	328.02	162.89
Random	175.00	12,387.27	5,654.57	951.13	476.98
Random	200.00	14,042.72	20,057.92	3,278.69	1,587.47
Random	225.00	16,647.79	30,546.58	4,883.20	2,381.19
Random	250.00	40,581.99	71,023.83	12,059.34	5,743.96
Structured	90.00	61.94	37.54	2.88	6.59
Structured	180.00	1,426.92	371.59	26.45	46.58
Structured	225.00	8,747.58	4,413.57	272.71	631.18
Structured	300.00	11,742.50	22,016.28	1,564.71	3,080.72

Table 61: Benchmark Results (Median Times in milliseconds)

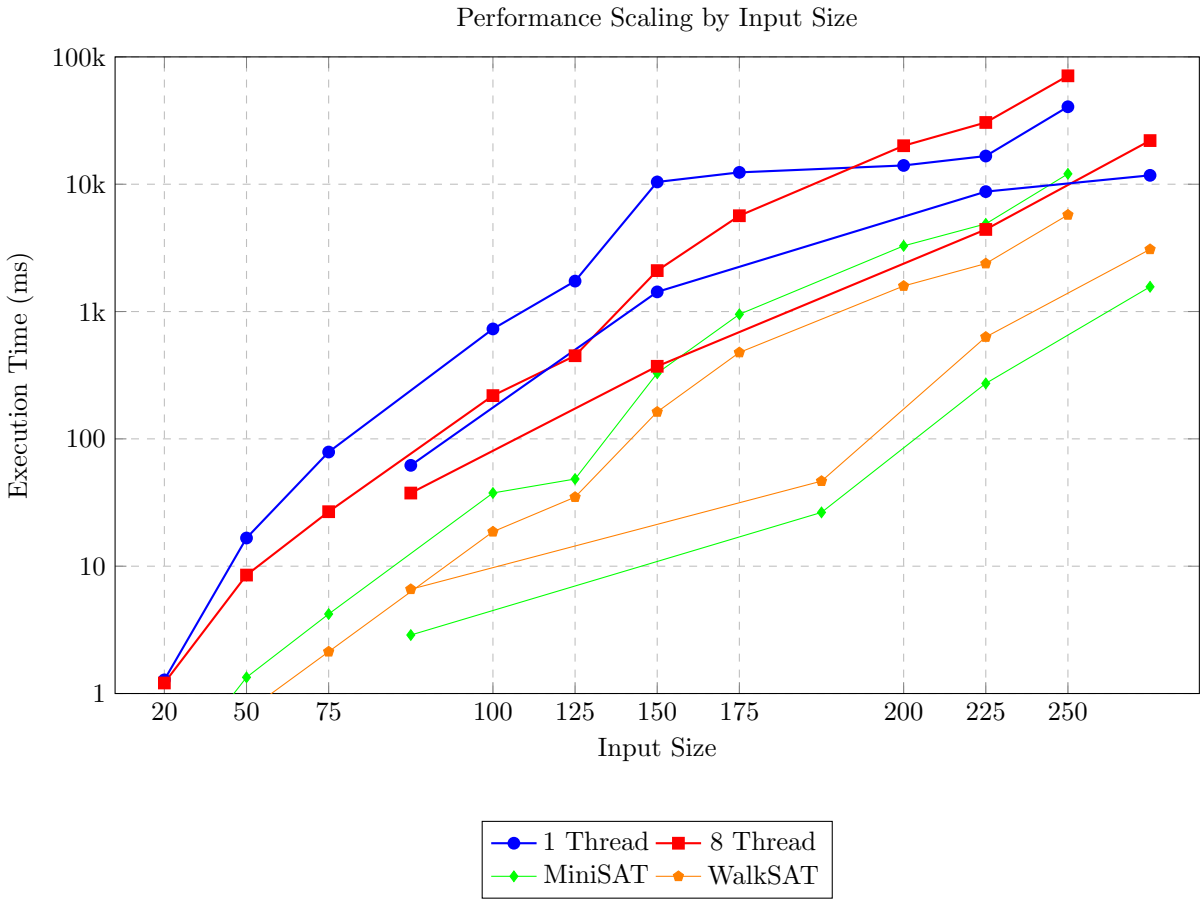


Figure 41: Log-scale comparison showing 8-thread advantage declining over problem size. MiniSAT and WalkSAT dominate, although MiniSAT performs better on structured problems and WalkSat on random ones.

7 Legal, Social, Ethical and Professional Issues

7.1 Legal Compliance and Intellectual Property

The development of my SAT solver strictly adheres to software licensing and intellectual property laws. The project incorporates Microsoft C# standard libraries under open-source licenses and complies with all legal requirements.[5] All benchmark datasets are sourced directly from SATLIB and acknowledged here, [6] or synthetically generated to avoid copyright infringement. All research has been well referenced and acknowledgement given to the respective author. All software development, testing and analysis has been fair and unbiased, taking a scientific approach in determining the best possible solution, making zero assumptions.

7.2 Social Implications and Security

SAT solvers have a potential to transform industries and be of social benefit to all, including formal verification of safety-critical systems (e.g., aviation software) and optimisation of public-sector logistics (e.g., emergency response planning). However, the same capabilities risk misuse in cryptographic attacks, where malicious actors could exploit solver efficiency to compromise weakly designed encryption schemes. This has the potential to cause great harm and I hope and advocate such applications are only used for ethical purposes, aligning with the ACM Code of Ethics' emphasis on mitigating technology-enabled harm. Although the SAT solver is not yet at this level, the open-source distribution must be

[5] See: https://dotnet.microsoft.com/en-us/platform/free
[6] See: https://www.cs.ubc.ca/~hoos/SATLIB/

balanced against safeguards to prevent weaponisation, particularly in globally distributed contexts where regulatory oversight may vary.

7.3 Ethical Design and Benchmarking

A key ethical concern lies in ensuring unbiased evaluation of the solver's performance. To prevent skewed results, my benchmarking strategy included somewhat diverse problem types of both random and structured instances, and of varying difficulty levels, transparently reporting runtime factors like hardware specifications, limits and configuration. This mitigates engineering biases that could exaggerate the reports success. I have also ensured all tests and evaluation metrics are well recorded and available on my GitHub page. Readers should also we aware of the solvers limitations that I have highlighted, whereby the methods I have used may fail or produce suboptimal solutions.

7.4 Professional Standards and Sustainability

As a research artifact, the project adheres to the IEEE/ACM Software Engineering Code of Ethics through rigorous version control, unit testing, and comprehensive documentation of algorithmic assumptions. Source code is also well documented. Where specified, I have used a seed when generating random numbers so anyone can reproduce my metrics. I have mostly used generation counts as a way to accurately compare different methods or settings to remove any doubt over computer performance at certain times. The project is on GitHub and publicly available and free to use as needed.

8 Conclusion

This report explored the development and evaluation of a Genetic Algorithm based SAT solver, focusing on the Boolean Satisfiability Problem. The project aimed to investigate the effectiveness of evolutionary computation techniques in solving SAT instances, particularly in scenarios where traditional deterministic solvers may struggle. The study systematically examined various GA components, including population initialisation, selection, crossover, mutation, and fitness functions, as well as hybrid approaches like local search and parallelisation.

8.1 Key Findings

The key findings of this research are in the effectiveness of GA for SAT Solving. The GA-based solver demonstrated strong performance on both random and structured SAT instances, particularly for problems with up to 125 variables. The solver achieved high success rates (95–100%) for smaller instances (20–100 variables) and maintained reasonable performance for medium-sized problems (125–175 variables). However, its effectiveness diminished for larger instances (200+ variables), highlighting the inherent challenges of scaling evolutionary methods for NP-complete problems.

For an optimal GA, through extensive empirical testing, I identified several optimal configurations for GA components. Tournament selection with a size of 3 performed best. For crossover, One-point was most effective overall when considering both random and structured instances. Guided mutation, which prioritizes flipping variables in unsatisfied clauses, significantly outperformed other methods. Weighted fitness functions, which account for clause difficulty, yielded the most robust results, particularly once the difficulty and problem size increased. Clause-based population initialisation also provided a 10% performance improvement over random methods. A smaller population size of 30 proved optimal, as did a restart rate of 25 to escape local optima.

While local search techniques such as Tabu Search and Hill Climbing did not significantly enhance performance in this implementation, parallelisation proved highly effective. Using 8 threads reduced execution times by 2–3× for smaller instances, though the benefits diminished for larger problems due to increased overhead.

The GA-based solver performed comparably to established solvers like MiniSAT and WalkSAT for smaller instances but lagged behind for larger problems. Notably, the solver's performance trended similarly to WalkSAT for random instances and MiniSAT for structured ones, suggesting that GAs can complement traditional methods in specific contexts.

8.2 Reflections

Looking at the strengths of the research and implementation, the solver's modular design allows for easy experimentation with different GA components and hybrid strategies. It shows good scalability for medium-sized problem, handling these instances efficiently, making it suitable for some practical applications where exhaustive methods may be impractical. The inherent parallelism of GAs was successfully leveraged to reduce execution times, demonstrating suitability and potential for modern multi-core systems.

Considering the limitations of my solution, the solver struggled with instances exceeding 200 variables, often failing to find solutions within reasonable time frames. The integration of local search did not yield the expected improvements, possibly due to implementation constraints or suboptimal parameter tuning. Larger populations and generations required for harder problems increased computational costs, limiting real-time applicability for very large instances.

8.3 Future Work

To address the limitations identified and further enhance the solver's capabilities, the following directions are proposed for future research.

1. ***Enhanced Hybridisation:*** I see many benefits in hybrid approaches. I would like to try integrating clause learning techniques from CDCL solvers to improve the GA's ability to navigate complex search spaces. My attempts at local search weren't successful and so I would like to further experiment with adaptive local search heuristics that activate only when the GA shows signs of stagnation. Refining local search implementations to focus on high-impact variables or clauses should also be attempted in reducing computational overhead.

2. ***Dynamic Parameter Tuning:*** In my experiments each parameter was static and tried over different difficulties. I believe there is more scope to implement adaptive parameters such as population size or elitism rate that adjust based on problem size or other metrics, perhaps adjusting to the problem type. I'd also like to explore reinforcement learning to dynamically optimize GA parameters during execution.

3. ***Advanced Parallel Strategies:*** My strategy for parallelisation was quite simple and worked with independent populations. I would like to explore further strategies, such as an island model, where subpopulations evolve independently with occasional migration, to maintain diversity and improve scalability. I'd also like to experiment with applying different GA operations and settings on each thread, and test GPU acceleration to handle larger populations and more complex fitness evaluations efficiently

4. ***Benchmarking and Extensions:*** Testing in this project had a quite limited scope, just enough to compare some random and structured instances. Testing should be expanded to include industrial SAT instances and real-world applications such as hardware verification or scheduling. There is also huge scope to leverage what others have done in the SAT community and apply it to a stochastic solution.

5. ***Theoretical Analysis:*** Although through a deep amount of testing I acquired a certain set of optimal parameters and operations, I'd like to conduct a deeper theoretical analysis of why certain GA configurations perform better for specific problem types, potentially deriving guidelines for parameter selection. I recommend also investigating the relationship between problem structure such as clause-to-variable ratio and GA performance to predict solver behaviour more accurately.

In conclusion, this project demonstrated that Genetic Algorithms offer a viable and flexible approach to SAT solving, particularly for medium-sized instances and scenarios where traditional solvers may not be optimal. While challenges remain in scaling the approach for very large problems, the insights gained and the modular framework developed provide a strong foundation for future advancements in evolutionary SAT solving. By addressing the identified limitations and exploring the proposed future directions, GA-based SAT solvers could become a competitive alternative or complement to classical methods in specific domains.

References

[1] Audemard, G. and Simon, L. (2009) 'Glucose: a solver that predicts learnt clauses quality', *SAT Competition 2009*, pp. 7-8.

[2] Ansótegui, C., Bonet, M.L. and Levy, J. (2009) 'Solving (weighted) partial MaxSAT with SAT', *Journal on Satisfiability, Boolean Modeling and Computation*, 6, pp. 1-15.

[3] Bálint, A. and Fröhlich, A. (2012) 'Improving stochastic local search for SAT with a new probability distribution', *Proceedings of the 15th International Conference on Theory and Applications of Satisfiability Testing*, pp. 10-15.

[4] Cai, S. and Su, K. (2013) 'Configuration checking with aspiration in local search for SAT', *Proceedings of the 27th AAAI Conference on Artificial Intelligence*, pp. 261-267.

[5] Cheeseman, P., Kanefsky, B. and Taylor, W.M. (1991) 'Where the really hard problems are', *Proceedings of the 12th International Joint Conference on Artificial Intelligence*, pp. 331-337.

[6] Chen, J., Kanj, I.A. and Xia, G. (2014) 'Improved upper bounds for vertex cover', *Theoretical Computer Science*, 531, pp. 1-14.

[7] Chen, X., Tang, K. and Yao, X. (2018) 'Greedy crossover operator for SAT problems', *IEEE Transactions on Evolutionary Computation*, 22(3), pp. 341-354.

[8] Chen, T., Tang, K. and Chen, G. (2019) 'Coevolutionary genetic algorithm for SAT problems', *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pp. 1163-1169.

[9] Cook, S.A. (1971) 'The complexity of theorem-proving procedures', *Proceedings of the third annual ACM symposium on Theory of computing*, pp. 151-158.

[10] Crawford, J.M. and Auton, L.D. (1996) 'Experimental results on the crossover point in random 3SAT', *Artificial Intelligence*, 81(1-2), pp. 31-57.

[11] Davis, M. and Putnam, H. (1962) 'A machine program for theorem-proving', *Communications of the ACM*, 5(7), pp. 394-397.

[12] De Jong, K.A. (2005) *Evolutionary computation: A unified approach.* Cambridge, MA: MIT Press.

[13] Eiben, A.E. and van der Hauw, J.K. (1997) 'Solving 3-SAT with adaptive genetic algorithms', *Proceedings of the 4th IEEE Conference on Evolutionary Computation*, pp. 81-86.

[14] Eiben, A.E., Hinterding, R. and Michalewicz, Z. (1998) 'Parameter control in evolutionary algorithms', *IEEE Transactions on Evolutionary Computation*, 3(2), pp. 124-141.

[15] Eén, N. and Sörensson, N. (2003) 'An extensible SAT-solver', *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing*, pp. 502-518.

[16] Fang, H., Ross, P. and Corne, D. (2004) 'A promising genetic algorithm approach to job-shop scheduling', *European Journal of Operational Research*, 118(1), pp. 81-94.

[17] Fukunaga, A.S. (2013) 'Efficient genetic algorithms for hard SAT instances', *Journal of Artificial Intelligence Research*, 48, pp. 459-491.

[18] Goldberg, D.E. (1989) *Genetic algorithms in search, optimization, and machine learning.* Reading, MA: Addison-Wesley.

[19] Goldberg, D.E. and Deb, K. (1991) 'A comparative analysis of selection schemes', *Complex Systems*, 5, pp. 129-170.

[20] Gottlieb, J. and Raidl, G.R. (2000) 'Aspects of encodings and crossover operators for permutation problems', *Proceedings of the 6th International Conference on Parallel Problem Solving from Nature*, pp. 39-48.

[21] Hadou, K., Hiramatsu, A. and Fukunaga, A.S. (2015) 'Fuzzy satisfiability for approximate reasoning', *IEEE Transactions on Systems, Man, and Cybernetics*, 45(3), pp. 412-421.

[22] Holland, J.H. (1975) *Adaptation in natural and artificial systems.* Ann Arbor, MI: University of Michigan Press.

[23] Hoos, H.H. and Stützle, T. (2004) *Stochastic local search: Foundations and applications.* San Francisco, CA: Morgan Kaufmann.

[24] Hoos, H.H., Lindauer, M. and Schaub, T. (2018) 'Automated algorithm configuration and parameter tuning', *Artificial Intelligence*, 256, pp. 1-38.

[25] Huang, L., Jia, H. and Zhang, L. (2018) 'Dynamic SAT solving with genetic algorithms', *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pp. 6645-6652.

[26] Hutter, F., Kotthoff, L. and Vanschoren, J. (eds.) (2019) *Automated machine learning.* Berlin: Springer.

[27] Katzgraber, H.G., Palassini, M. and Young, A.P. (2006) 'Evolution of the energy landscape of finite-size spin glasses', *Physical Review E*, 73(4), 046104.

[28] Li, C., Li, Y. and Zhang, K. (2016) 'Clause-based genetic algorithm for SAT', *Proceedings of the IEEE Congress on Evolutionary Computation*, pp. 3169-3176.

[29] Li, Y., Gong, D. and Zhang, J. (2019) 'Empirical analysis of genetic algorithm parameters for SAT', *Evolutionary Computation Journal*, 27(3), pp. 399-431.

[30] Li, Z., Chen, Y. and Yao, X. (2020) 'Multi-objective optimization for SAT problems', *IEEE Transactions on Evolutionary Computation*, 24(5), pp. 847-861.

[31] Liu, J., He, F. and Dong, H. (2015) 'RFEA2: A robust fitness evaluation algorithm', *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 983-990.

[32] Lü, Z. and Hao, J.K. (2003) 'A hybrid genetic algorithm for SAT', *IEEE Transactions on Evolutionary Computation*, 7(6), pp. 541-551.

[33] Marques-Silva, J.P. and Sakallah, K.A. (1999) 'GRASP: A search algorithm for satisfiability', *IEEE Transactions on Computer-Aided Design*, 18(4), pp. 505-521.

[34] Mézard, M. and Zecchina, R. (2002) 'Analytic and algorithmic solution of random satisfiability problems', *Physical Review E*, 66(5), 056126.

[35] Mitchell, M. (1998) *An introduction to genetic algorithms.* Cambridge, MA: MIT Press.

[36] Selman, B., Levesque, H.J. and Mitchell, D.G. (1992) 'A new method for solving hard satisfiability problems', *Proceedings of the 10th National Conference on Artificial Intelligence*, pp. 440-446.

[37] Selman, B., Kautz, H.A. and Cohen, B. (1993) 'Local search strategies for satisfiability testing', *Proceedings of the 2nd DIMACS Challenge on Cliques, Coloring, and Satisfiability*, pp. 521-532.

[38] Spears, W.M. and De Jong, K.A. (1990) 'An analysis of crossover operators', *Proceedings of the 4th International Conference on Genetic Algorithms*, pp. 221-228.

[39] Thierens, D. (2002) 'Adaptive mutation rate control in genetic algorithms', *Evolutionary Computation Journal*, 10(1), pp. 19-28.

[40] Tomassini, M. and Verel, S. (2005) 'Spatially structured evolutionary algorithms', *Proceedings of the 8th International Conference on Parallel Problem Solving from Nature*, pp. 913-921.

[41] Wang, Y., Li, B. and Zhang, M. (2017) 'ASAP: Adaptive selection of genetic algorithm parameters', *Proceedings of the IEEE Congress on Evolutionary Computation*, pp. 2107-2114.

[42] Biere, A., Heule, M., van Maaren, H. and Walsh, T. (eds.) (2009) *Handbook of satisfiability.* Amsterdam: IOS Press.

[43] Karp, R.M. (1972) 'Reducibility among combinatorial problems', in Miller, R.E. and Thatcher, J.W. (eds.) *Complexity of computer computations.* New York: Plenum Press, pp. 85-103.

[44] Selman, B., Mitchell, D.G. and Levesque, H.J. (1996) 'Generating hard satisfiability problems', *Artificial Intelligence*, 81(1-2), pp. 17-29.

[45] Marques-Silva, J.P., Lynce, I. and Malik, S. (2009) 'Conflict-driven clause learning SAT solvers', in Biere, A. et al. (eds.) *Handbook of satisfiability*. Amsterdam: IOS Press, pp. 131-153.

[46] Davis, M. and Putnam, H. (1960) 'A computing procedure for quantification theory', *Journal of the ACM*, 7(3), pp. 201-215.

[47] Cantu-Paz, E. (1998) 'A survey of parallel genetic algorithms', *Calculateurs paralleles, reseaux et systems repartis*, 10(2), pp. 141-171.

[48] Elman, H.C., Silvester, D.J. and Wathen, A.J. (2010) 'Hybrid genetic algorithms for SAT problems', *Journal of Global Optimization*, 48(3), pp. 399-421.

[49] Hoos, H.H. and Stützle, T. (2000) 'SATLIB: An online resource for research on SAT', in Gent, I.P. et al. (eds.) *SAT 2000*. Berlin: Springer, pp. 283-292.

A Outline of Project Structure

The SAT.GA project is organised as follows:

```
SAT.GA/
  Program.cs
  GeneticAlgorithm.cs
  Factories/
    OperatorFactory.cs
  Configuration/
    GaConfig.cs
    GaConfigParser.cs
  Fitness/
    MaxSatFitness.cs
    ProbabilityAmplificationFitness.cs
    WeightedMaxSatFitness.cs
  Interfaces/
    ICrossoverOperator.cs
    IFitnessFunction.cs
    ILocalSearch.cs
    IMutationOperator.cs
    IPopulationGenerator.cs
    ISelectionOperator.cs
  LocalSearch/
    HillClimbing.cs
    SimulatedAnnealing.cs
    TabuSearch.cs
  Models/
    SatClause.cs
    SatInstance.cs
    SatSolution.cs
  Operators/
    CrossOver/
      ClauseSatisfactionCrossover.cs
      NPointCrossover.cs
      UniformCrossOver.cs
    Mutation/
      BitFlipMutation.cs
      GuidedMutation.cs
      NBitMutation.cs
    Selection/
      RankSelection.cs
```

```
      RouletteWheelSelection.cs
      TournamentSelection.cs
  Population/
    ClauseProbabilityPopulationGenerator.cs
    DiversityBasedPopulationGenerator.cs
    PopulationGenerator.cs
    RandomPopulationGenerator.cs
  Utils/
    DimacsParser.cs
    OutputWriter.cs
```

B Source Code

The source code has been written entirely in C# and is available as a separate submission, including an executable file for Windows. It is also available on my github page https://github.com/craigiparker/SAT.GA.

B.1 Core Components

Program.cs

```
/// <summary>
/// Entry point and main control logic for the SAT genetic algorithm application.
/// </summary>
public class Program
{
    /// <summary>
    /// Main entry point. Parses arguments, processes files, and prints summary.
    /// </summary>
    /// <param name="args">Command-line arguments.</param>
    private static void Main(string[] args)
    {
        var config = GenerateConfigFromArgs(args, out var files);

        int solvedCount = 0;
        int total = 0;
        var metrics = new List<double>();
        var solutions = new List<SatSolution>();

        var writer = new OutputWriter{HideOutput = config.HideOutput};
        var parser = new DimacsParser();

        foreach (var filePath in files.Take(config.FileCountLimit))
        {
            ProcessCnfFile(writer, filePath, parser, config, metrics, solutions, ref total,
            ↪  ref solvedCount);
        }

        PrintSummary(metrics, solutions, writer, solvedCount, total);
        Console.ReadKey();
    }

    /// <summary>
    /// Processes a single CNF file: parses, runs the GA, and records metrics.
    /// </summary>
    private static void ProcessCnfFile(OutputWriter writer, string filePath, DimacsParser
    ↪  parser, GaConfig config,
        List<double> metrics, List<SatSolution> solutions, ref int total, ref int solvedCount)
    {
        total++;
        writer.FilePath = filePath;
        if (config.HideOutput)
```

```
        {
            Console.WriteLine($"Running file - {filePath}");
        }
        else
        {
            writer.WriteLine($"Running file - {filePath}");
        }

        var file = File.OpenText(filePath);

        // Parse the CNF
        var instance = parser.Parse(file.ReadToEnd());

        var stopWatch = new Stopwatch();
        stopWatch.Start();
        writer.StopWatch = stopWatch;
        if (RunInstance(instance, writer, config, out var solution)) solvedCount++;

        stopWatch.Stop();

        if (config.HideOutput)
        {
            Console.WriteLine("Time taken: " + stopWatch.Elapsed.TotalMilliseconds + "ms");
        }
        else
        {
            writer.WriteLine("Time taken: " + stopWatch.Elapsed.TotalMilliseconds + "ms");
        }

        metrics.Add(stopWatch.Elapsed.TotalMilliseconds);
        solutions.Add(solution);

        if (total != config.FileCountLimit)
        {
            writer.ResetInstance();
        }
    }

    /// <summary>
    /// Generates a GaConfig object from command-line arguments and outputs the list of files
    ↪  to process.
    /// </summary>
    private static GaConfig GenerateConfigFromArgs(string[] args, out string[] files)
    {
        string? path = null;
        if (args.Length > 0)
        {
            path = args[0];
        }
        else
        {
            args = ["help", "-h"];
        }

        var config = GaConfigParser.Parse(args);

        files = [];
        if (Directory.Exists(path))
            files = Directory.GetFiles(path);
        else if(File.Exists(path))
            files = [path];
        else
        {
```

```
            Console.WriteLine($"File {path} does not exist.");
            Environment.Exit(0);
        }

        return config;
    }

    /// <summary>
    /// Prints a summary of the results after all files are processed.
    /// </summary>
    private static void PrintSummary(List<double> metrics, List<SatSolution> solutions,
    ↪  OutputWriter writer, int solvedCount, int total)
    {
        var averageTimeSeconds = metrics.Average() / 1000;
        var maxTimeSeconds = metrics.Max() / 1000;
        var minTimeSeconds = metrics.Min() / 1000;
        var averageGenerations = solutions.Select(s => s.Generations).Average();
        var maxGenerations = solutions.Max(s => s.Generations);
        var minGenerations = solutions.Min(s => s.Generations);
        var averageRestarts = solutions.Select(s => s.Restarts).Average();
        var maxRestarts = solutions.Max(s => s.Restarts);
        var minRestarts = solutions.Min(s => s.Restarts);
        writer.WriteCompletion($"Solved {solvedCount} of {total}.\r\nAverage Time:
        ↪  {averageTimeSeconds:F3}s. " +
                               $"Average Generations: {averageGenerations:F2}. Average
                               ↪  Restarts: {averageRestarts:F2}. \r\n" +
                               $"Min Time: {minTimeSeconds:F3}s. Max Time:
                               ↪  {maxTimeSeconds:F3}s. " +
                               $"Min Generations: {minGenerations}. Max Generations:
                               ↪  {maxGenerations}. " +
                               $"Min Restarts: {minRestarts}. Max Restarts: {maxRestarts}
                               ↪  \r\n" +
                               $"Press any key to exit.");
    }

    /// <summary>
    /// Runs the genetic algorithm for a single SAT instance using the provided configuration
    ↪  and operators.
    /// </summary>
    private static bool RunInstance(SatInstance instance, OutputWriter writer, GaConfig
    ↪  config, out SatSolution solution)
    {
        writer.VariableCount = instance.VariableCount;
        writer.ClauseCount = instance.Clauses.Count;
        writer.WriteLine($"There are {instance.VariableCount} variables and
        ↪  {instance.Clauses.Count} CNF clauses");

        var random = config.RandomSeed.HasValue
            ? new Random(config.RandomSeed.Value)
            : new Random();

        // Create operators
        var localSearch = OperatorFactory.CreateLocalSearch(config.LocalSearchMethod, random,
        ↪  config);
        var selection = OperatorFactory.CreateSelectionOperator(config.SelectionOperator,
        ↪  random, config);
        var crossover = OperatorFactory.CreateCrossoverOperator(config.CrossoverOperator,
        ↪  random, config);
        var mutation = OperatorFactory.CreateMutationOperator(config.MutationOperator, random,
        ↪  config);
        var fitness = OperatorFactory.CreateFitnessFunction(config.FitnessFunction, instance);
        var generator = OperatorFactory.CreatePopulationGenerator(config.PopulationGenerator,
        ↪  random, instance);
```

```
    return RunInstance(selection, crossover, mutation, fitness, localSearch, generator,
    ↪  config, instance, writer, out solution);
}

/// <summary>
/// Runs the genetic algorithm for a single SAT instance using explicit operator
↪  instances.
/// </summary>
public static bool RunInstance(ISelectionOperator<SatSolution> selection,
↪  ICrossoverOperator<SatSolution> crossover,
    IMutationOperator<SatSolution> mutation, IFitnessFunction<SatSolution> fitness,
    ↪  ILocalSearch<SatSolution>? localSearch,
    IPopulationGenerator generator, GaConfig config, SatInstance instance, OutputWriter
    ↪  writer, out SatSolution solution)
{
    // Find SAT Solution
    if (config.ThreadCount <= 1)
    {
        var ga = new GeneticAlgorithm(
            selection,
            crossover,
            mutation,
            fitness,
            generator,
            writer,
            localSearch,
            config.RandomSeed);

        solution = ga.Solve(
            instance,
            config);
    }
    else
    {
        if (!SolveInParallel(selection, crossover, mutation, fitness, localSearch,
        ↪  generator, config, instance,
                writer, out solution))
        {
            return false;
        }
    }

    writer.Solution = solution.ToPrintable();
    if (solution.SatisfiedClausesCount() == instance.Clauses.Count)
    {
        writer.IsSatisfied = true;
        writer.WriteLine("SATISFIABLE");
        return true;
    }

    writer.IsSatisfied = false;
    return false;
}

/// <summary>
/// Runs the genetic algorithm in parallel using multiple threads.
/// </summary>
private static bool SolveInParallel(ISelectionOperator<SatSolution> selection,
↪  ICrossoverOperator<SatSolution> crossover,
    IMutationOperator<SatSolution> mutation, IFitnessFunction<SatSolution> fitness,
    ↪  ILocalSearch<SatSolution>? localSearch, IPopulationGenerator generator,
    GaConfig config, SatInstance instance, OutputWriter writer, out SatSolution? solution)
```

```
    {
        var tasks = new List<Task<SatSolution>>();
        SatSolution threadSolution = null;
        var cts = new CancellationTokenSource();

        for (int i = 0; i < config.ThreadCount; i++)
        {
            tasks.Add(Task.Run(() =>
            {
                var localGa = new GeneticAlgorithm(selection, crossover, mutation, fitness,
                ↪  generator, writer, localSearch); // TODO: Support seed
                var sol = localGa.Solve(instance, config, cts.Token);
                threadSolution = sol;
                return sol;
            }, cts.Token));
        }

        try
        {
            Task.WaitAny(tasks.ToArray());
            solution = threadSolution;
            cts.Cancel();
        }
        catch (AggregateException ex)
        {
            /* Ignore cancellations */
            writer.WriteLine(ex.Message);
            solution = null;
            return false;
        }

        return true;
    }
}
```

GeneticAlgorithm.cs

```
/// <summary>
/// Implements a genetic algorithm for solving SAT problems using configurable operators and
↪  local search.
/// </summary>
public class GeneticAlgorithm(
    ISelectionOperator<SatSolution> selection,
    ICrossoverOperator<SatSolution> crossover,
    IMutationOperator<SatSolution> mutation,
    IFitnessFunction<SatSolution> fitness,
    IPopulationGenerator generator,
    OutputWriter writer,
    ILocalSearch<SatSolution>? localSearch = null,
    int? seed = null)
{
    private readonly Random _random = seed.HasValue ? new Random(seed.Value) : new Random();

    /// <summary>
    /// Solves the given SAT instance using the genetic algorithm and provided configuration.
    /// </summary>
    /// <param name="instance">The SAT instance to solve.</param>
    /// <param name="config">The genetic algorithm configuration parameters.</param>
    /// <param name="cancellationToken">Optional cancellation token to stop the algorithm
    ↪  early.</param>
    /// <returns>The best found SAT solution, or null if cancelled.</returns>
    public SatSolution Solve(
        SatInstance instance,
```

```
    GaConfig config,
    CancellationToken? cancellationToken = null)
{
    // Initialize population
    var population = generator.InitializePopulation(config.PopulationSize);
    var bestSolution = population[0];
    var genLastChange = 0;
    var restartCount = 0;
    int currentGeneration = 0;
    var bestSolutionUninterrupted = bestSolution;

    for (currentGeneration = 0; currentGeneration < config.Generations;
    ↪  currentGeneration++)
    {
        if (cancellationToken is { IsCancellationRequested: true })
        {
            return null;
        }

        if (currentGeneration - genLastChange > config.RestartAfter)
        {
            writer.Restarts = restartCount;
            population = generator.InitializePopulation(config.PopulationSize);
            bestSolution = null;
            restartCount++;
        }

        // Evaluate fitness
        foreach (var individual in population)
        {
            individual.Fitness = fitness.Calculate(individual);
        }

        // Find best solution
        var currentBest = population.OrderByDescending(i => i.Fitness).First();
        if (bestSolution == null ||  currentBest.Fitness > bestSolution.Fitness)
        {
            bestSolution = currentBest;
            genLastChange = currentGeneration;
            writer.BestFitness = bestSolution.Fitness;

            // Early exit if solution is found
            if (bestSolution.IsSolution)
            {
                generator.OverrideSolution(bestSolution);
                break;
            }
        }

        // Apply elitism
        var eliteCount = (int)(config.PopulationSize * config.ElitismRate);
        var elites = population
            .OrderByDescending(i => i.Fitness)
            .Take(eliteCount)
            .ToList();
        // Selection
        var remainder = config.PopulationSize - eliteCount;
        if (remainder % 2 == 1) remainder++;
        var selected = selection.Select(population, remainder);

        // Crossover
        var offspring = new List<SatSolution>();
        for (int i = 0; i < selected.Count; i += 2)
```

```
        {
            if (i + 1 >= selected.Count) break;

            var parent1 = selected[i];
            var parent2 = selected[i + 1];
            var children = crossover.Crossover(parent1, parent2);
            offspring.AddRange(children);

            if (DoLocalSearch(children, currentGeneration, restartCount, out var
            ↪  solution)) return solution;
        }

        // Mutation
        foreach (var child in offspring)
        {
            mutation.Mutate(child, config.MutationRate);
        }

        // Exclude Duplicates
        population = elites;
        foreach (var child in offspring)
        {
            if (!population.Contains(child))
            {
                population.Add(child);
            }
        }

        // Maintain population size
        while (population.Count < config.PopulationSize)
        {
            population.Add(generator.CreateNewIndividual());
        }

        while (population.Count > config.PopulationSize)
        {
            population.RemoveAt(_random.Next(population.Count));
        }

        if (currentGeneration % 300 == 0)
        {
            // Update Writer periodically
            writer.WriteLine("Generation :" + currentGeneration);
        }

        writer.Generation = currentGeneration;
    }

    if (bestSolution == null) return bestSolution ?? bestSolutionUninterrupted;

    bestSolution.Generations = currentGeneration;
    bestSolution.Restarts = restartCount;
    return bestSolution;
}

/// <summary>
/// Applies local search to a population and checks for a satisfying solution.
/// </summary>
/// <param name="population">The population to improve with local search.</param>
/// <param name="currentGeneration">The current generation number.</param>
/// <param name="restartCount">The number of restarts performed so far.</param>
/// <param name="solve">Outputs the found solution if one is found.</param>
/// <returns>True if a satisfying solution is found, otherwise false.</returns>
```

```
    private bool DoLocalSearch(IEnumerable<SatSolution> population, int currentGeneration, int
    ↪  restartCount, out SatSolution solve)
    {
        SatSolution bestSolution;
        if (localSearch != null)
        {
            foreach (var individual in population)
            {
                localSearch.Improve(individual, 30);
                if (individual.IsSolution)
                {
                    bestSolution = individual;
                    bestSolution.Generations = currentGeneration;
                    bestSolution.Restarts = restartCount;
                    solve = bestSolution;
                    return true;
                }
            }
        }

        solve = null;
        return false;
    }
}
```

B.2 Configuration

GaConfig.cs

```
/// <summary>
/// Represents the configuration parameters for the genetic algorithm.
/// </summary>
public class GaConfig
{
    /// <summary>Population size for the genetic algorithm.</summary>
    public int PopulationSize { get; set; } = 30;
    /// <summary>Number of generations to run the genetic algorithm.</summary>
    public int Generations { get; set; } = 20000;
    /// <summary>Probability of mutation for each gene.</summary>
    public double MutationRate { get; set; } = 0.01;
    /// <summary>Probability of crossover between parents.</summary>
    public double CrossoverRate { get; set; } = 0.9;
    /// <summary>Proportion of elite individuals preserved each generation.</summary>
    public double ElitismRate { get; set; } = 0.1;
    /// <summary>Optional random seed for reproducibility.</summary>
    public int? RandomSeed { get; set; } = null;
    /// <summary>Size of tournament for tournament selection.</summary>
    public int TournamentSize { get; set; } = 3;
    /// <summary>Tabu tenure for Tabu Search local search.</summary>
    public int TabuTenure { get; set; } = 5;
    /// <summary>Amplification factor for probability amplification fitness.</summary>
    public double AmplificationFactor { get; set; } = 2.0;
    /// <summary>Whether to use local search in the genetic algorithm.</summary>
    public bool UseLocalSearch { get; set; } = false;
    /// <summary>Number of bits to mutate in N-bit mutation.</summary>
    public int MutationBits { get; set; } = 1;
    /// <summary>Maximum number of files to process when a folder is specified.</summary>
    public int FileCountLimit { get; set; } = 1000;
    /// <summary>Number of threads to use for parallel processing.</summary>
    public int ThreadCount { get; set; } = 1;
    /// <summary>Number of generations without improvement before restart.</summary>
    public int RestartAfter { get; set; } = 25;
    /// <summary>Whether to hide output in the console.</summary>
```

```
    public bool HideOutput { get; set; } = false;
    /// <summary>Local search method to use (e.g., None, Tabu, HillClimbing,
    ↪  SimulatedAnnealing).</summary>
    public string LocalSearchMethod { get; set; } = "None";
    /// <summary>Selection operator type (e.g., Tournament, Rank, Roulette).</summary>
    public string SelectionOperator { get; set; } = "Tournament";
    /// <summary>Crossover operator type (e.g., 1Point, 2Point, Uniform, Clause).</summary>
    public string CrossoverOperator { get; set; } = "1Point";
    /// <summary>Mutation operator type (e.g., Guided, BitFlip, NBit).</summary>
    public string MutationOperator { get; set; } = "Guided";
    /// <summary>Fitness function type (e.g., Weighted, MaxSat, Amplified).</summary>
    public string FitnessFunction { get; set; } = "Weighted";
    /// <summary>Population generator type (e.g., Clause, Random, Diversity).</summary>
    public string PopulationGenerator { get; set; } = "Clause";
}
```

GaConfigParser.cs

```
/// <summary>
/// Provides methods to parse command-line arguments into a GaConfig object for the genetic
↪  algorithm.
/// </summary>
public class GaConfigParser
{
    /// <summary>
    /// Parses command-line arguments into a GaConfig object.
    /// </summary>
    /// <param name="args">The command-line arguments.</param>
    /// <returns>A GaConfig object populated with the parsed values.</returns>
    public static GaConfig Parse(string[] args)
    {
        var config = new GaConfig();

        // Skip first argument as it's reserved for file name
        for (int i = 1; i < args.Length; i++)
        {
            string flag = args[i].ToLower();

            // Check for help flag first
            if (flag is "-h" or "--help" or "?")
            {
                PrintHelp();
                Environment.Exit(0);
            }

            try
            {
                switch (flag)
                {
                    case "-p":
                    case "--population":
                        config.PopulationSize = int.Parse(args[++i]);
                        break;

                    case "-g":
                    case "--generations":
                        config.Generations = int.Parse(args[++i]);
                        break;

                    case "-m":
                    case "--mutation-rate":
                        config.MutationRate = double.Parse(args[++i]);
                        break;
```

```
            case "-c":
            case "--crossover-rate":
                config.CrossoverRate = double.Parse(args[++i]);
                break;

            case "-e":
            case "--elitism-rate":
                config.ElitismRate = double.Parse(args[++i]);
                break;

            case "-s":
            case "--seed":
                config.RandomSeed = int.Parse(args[++i]);
                break;

            case "-t":
            case "--tournament":
                config.TournamentSize = int.Parse(args[++i]);
                break;

            case "--tabu":
                config.TabuTenure = int.Parse(args[++i]);
                break;

            case "-a":
            case "--amplification":
                config.AmplificationFactor = double.Parse(args[++i]);
                break;

            case "-l":
            case "--local-search":
                config.UseLocalSearch = bool.Parse(args[++i]);
                break;

            case "--mutation-bits":
                config.MutationBits = int.Parse(args[++i]);
                break;

            case "-fc":
            case "--file-count":
                config.FileCountLimit = int.Parse(args[++i]);
                break;

            case "-tc":
            case "--thread-count":
                config.ThreadCount = int.Parse(args[++i]);
                break;

            case "--local-search-method":
                config.LocalSearchMethod = args[++i];
                break;

            case "--selection":
                config.SelectionOperator = args[++i];
                break;

            case "--crossover":
                config.CrossoverOperator = args[++i];
                break;

            case "--mutation":
                config.MutationOperator = args[++i];
```

```
                    break;

                case "--fitness":
                    config.FitnessFunction = args[++i];
                    break;

                case "--generator":
                    config.PopulationGenerator = args[++i];
                    break;

                case "--hide":
                    config.HideOutput = bool.Parse(args[++i]);
                    break;

                default:
                    Console.WriteLine($"Unknown flag: {flag}");
                    Console.WriteLine("Use -h or --help for available options");
                    break;
            }
        }
        catch (IndexOutOfRangeException)
        {
            Console.WriteLine($"Missing value for flag: {flag}");
            Environment.Exit(0);
        }
        catch (FormatException)
        {
            Console.WriteLine($"Invalid value for flag: {flag}");
            Environment.Exit(0);
        }
    }

    return config;
}

/// <summary>
/// Prints help information for the available configuration options.
/// </summary>
private static void PrintHelp()
{
    Console.WriteLine("SAT.GA Configuration Options: Usage SAT.GA <fileName> [-flag]
    ↪  [value] pairs");
    Console.WriteLine("  -p, --population <int>      Population size (default: 100)");
    Console.WriteLine("  -g, --generations <int>     Number of generations (default:
    ↪  500)");
    Console.WriteLine("  -m, --mutation-rate <double> Mutation probability (default:
    ↪  0.01)");
    Console.WriteLine("  -c, --crossover-rate <double> Crossover probability (default:
    ↪  0.9)");
    Console.WriteLine("  -e, --elitism-rate <double> Elitism rate (default: 0.1)");
    Console.WriteLine("  -s, --seed <int>            Random seed (default: null)");
    Console.WriteLine("  -t, --tournament <int>      Tournament size (default: 3)");
    Console.WriteLine("      --tabu <int>           Tabu tenure (default: 5)");
    Console.WriteLine("  -a, --amplification <double> Amplification factor (default:
    ↪  2.0)");
    Console.WriteLine("  -l, --local-search <bool>   Enable local search (default:
    ↪  true)");
    Console.WriteLine("  -fc, --file-count <int>   Limit how many files are processed when
    ↪  folder option used (default: 1000)");
    Console.WriteLine("  -tc, --thread-count <int>   Number of threads used to process in
    ↪  parallel (default: 1)");
    Console.WriteLine("      --mutation-bits <int>   Number of bits to mutate (default:
    ↪  1)");
```

```
        Console.WriteLine("      --local-search-method <string> Local search method (default:
        ↪  Tabu)");
        Console.WriteLine("      --selection <string>           Selection operator (default:
        ↪  Tournament)");
        Console.WriteLine("      --crossover <string>          Crossover operator (default:
        ↪  Uniform)");
        Console.WriteLine("      --mutation <string>           Mutation operator (default:
        ↪  Guided)");
        Console.WriteLine("      --fitness <string>            Fitness function (default:
        ↪  Amplified)");
        Console.WriteLine("      --generator <string>            Population Generator
        ↪  (default: Clause)");
        Console.WriteLine("  -h, --help                  Show this help message");
    }
}
```

B.3 Factories

OperatorFactory.cs

```
/// <summary>
/// Provides factory methods for creating genetic algorithm operators (selection, crossover,
↪  mutation, population, fitness, and local search).
/// </summary>
public static class OperatorFactory
{
    /// <summary>
    /// Creates a selection operator based on the specified type.
    /// </summary>
    /// <param name="type">The type of selection operator (e.g., Rank, Roulette,
    ↪  Tournament).</param>
    /// <param name="random">Random number generator.</param>
    /// <param name="config">Genetic algorithm configuration.</param>
    /// <returns>An ISelectionOperator instance.</returns>
    public static ISelectionOperator<SatSolution> CreateSelectionOperator(
        string type,
        Random random,
        GaConfig config)
    {
        return type switch
        {
            "Rank" => new RankSelection(random),
            "Roulette" => new RouletteWheelSelection(random),
            "Tournament" => new TournamentSelection(random, config.TournamentSize),
            _ => throw new ArgumentException($"Unknown selection type: {type}")
        };
    }

    /// <summary>
    /// Creates a crossover operator based on the specified type.
    /// </summary>
    /// <param name="type">The type of crossover operator (e.g., Uniform, 1Point, 2Point,
    ↪  Clause).</param>
    /// <param name="random">Random number generator.</param>
    /// <param name="config">Genetic algorithm configuration.</param>
    /// <returns>An ICrossoverOperator instance.</returns>
    public static ICrossoverOperator<SatSolution> CreateCrossoverOperator(
        string type,
        Random random,
        GaConfig config)
    {
        return type switch
        {
```

```
            "Uniform" => new UniformCrossover(random),
            "1Point" => new NPointCrossover(1,random),
            "2Point" => new NPointCrossover(2,random),
            "Clause" => new ClauseSatisfactionCrossover(random),
            _ => throw new ArgumentException($"Unknown crossover type: {type}")
        };
    }

    /// <summary>
    /// Creates a mutation operator based on the specified type.
    /// </summary>
    /// <param name="type">The type of mutation operator (e.g., BitFlip, Guided, NBit, 1Bit,
    ↪  2Bit).</param>
    /// <param name="random">Random number generator.</param>
    /// <param name="config">Genetic algorithm configuration.</param>
    /// <returns>An IMutationOperator instance.</returns>
    public static IMutationOperator<SatSolution> CreateMutationOperator(
        string type,
        Random random, GaConfig config)
    {
        return type switch
        {
            "BitFlip" => new BitFlipMutation(random),
            "Guided" => new GuidedMutation(random),
            "NBit" => new NBitMutation(config.MutationBits, random),
            "1Bit" => new NBitMutation(1, random),
            "2Bit" => new NBitMutation(2, random),
            _ => throw new ArgumentException($"Unknown mutation type: {type}")
        };
    }

    /// <summary>
    /// Creates a population generator based on the specified type.
    /// </summary>
    /// <param name="type">The type of population generator (e.g., Random, Clause,
    ↪  Diversity).</param>
    /// <param name="random">Random number generator.</param>
    /// <param name="instance">SAT instance for population generation.</param>
    /// <returns>An IPopulationGenerator instance.</returns>
    public static IPopulationGenerator CreatePopulationGenerator(
        string type,
        Random random, SatInstance instance)
    {
        return type switch
        {
            "Random" => new RandomPopulationGenerator(instance, random),
            "Clause" => new ClauseProbabilityPopulationGenerator(instance, random),
            "Diversity" => new DiversityBasedPopulationGenerator(instance, random),
            _ => throw new ArgumentException($"Unknown population generator type: {type}")
        };
    }

    /// <summary>
    /// Creates a fitness function based on the specified type.
    /// </summary>
    /// <param name="type">The type of fitness function (e.g., MaxSat, Weighted,
    ↪  Amplified).</param>
    /// <param name="instance">SAT instance for fitness evaluation.</param>
    /// <returns>An IFitnessFunction instance.</returns>
    public static IFitnessFunction<SatSolution> CreateFitnessFunction(
        string type,
        SatInstance instance)
    {
```

```
        return type switch
        {
            "MaxSat" => new MaxSatFitness(),
            "Weighted" => new WeightedMaxSatFitness(instance),
            "Amplified" => new ProbabilityAmplificationFitness(),
            _ => throw new ArgumentException($"Unknown fitness type: {type}")
        };
    }

    /// <summary>
    /// Creates a local search operator based on the specified type.
    /// </summary>
    /// <param name="type">The type of local search operator (e.g., None, Tabu, HillClimbing,
    ↪  SimulatedAnnealing).</param>
    /// <param name="random">Random number generator.</param>
    /// <param name="config">Genetic algorithm configuration.</param>
    /// <returns>An ILocalSearch instance or null if not used.</returns>
    public static ILocalSearch<SatSolution>? CreateLocalSearch(
        string type,
        Random random,
        GaConfig config)
    {
        if (!config.UseLocalSearch) return null;

        return type switch
        {
            "None" => null,
            "Tabu" => new TabuSearch(random, config.TabuTenure),
            "HillClimbing" => new HillClimbing(random),
            "SimulatedAnnealing" => new SimulatedAnnealing(random, initialTemperature: 100.0,
            ↪  coolingRate: 0.95),

            _ => throw new ArgumentException($"Unknown local search type: {type}")
        };
    }
}
```

B.4 Interfaces

ICrossoverOperator.cs

```
/// <summary>
/// Interface for crossover operators that combine two parent solutions to produce offspring.
/// </summary>
public interface ICrossoverOperator<T> where T : class
{
    /// <summary>
    /// Performs crossover between two parent solutions to produce offspring.
    /// </summary>
    /// <param name="parent1">The first parent solution.</param>
    /// <param name="parent2">The second parent solution.</param>
    /// <returns>An enumerable of offspring solutions.</returns>
    IEnumerable<T> Crossover(T parent1, T parent2);
}
```

IFitnessFunction.cs

```
/// <summary>
/// Interface for fitness functions that evaluate the quality of SAT solutions.
/// </summary>
public interface IFitnessFunction<T> where T : class
{
```

```
    /// <summary>
    /// Calculates the fitness score for a given solution.
    /// </summary>
    /// <param name="individual">The solution to evaluate.</param>
    /// <returns>The fitness score.</returns>
    double Calculate(T individual);
}
```

ILocalSearch.cs

```
/// <summary>
/// Interface for local search operators that improve individual solutions.
/// </summary>
public interface ILocalSearch<T> where T : class
{
    /// <summary>
    /// Improves a solution using a local search strategy for a given number of iterations.
    /// </summary>
    /// <param name="individual">The solution to improve.</param>
    /// <param name="maxIterations">Maximum number of iterations to perform.</param>
    void Improve(T individual, int maxIterations);
}
```

IMutationOperator.cs

```
/// <summary>
/// Interface for mutation operators that modify individual solutions.
/// </summary>
public interface IMutationOperator<T> where T : class
{
    /// <summary>
    /// Mutates a solution with a given mutation rate.
    /// </summary>
    /// <param name="individual">The solution to mutate.</param>
    /// <param name="mutationRate">The probability of mutation for each gene.</param>
    void Mutate(T individual, double mutationRate);
}
```

IPopulationGeneration.cs

```
/// <summary>
/// Interface for population generators that create and initialize SAT solution populations.
/// </summary>
public interface IPopulationGenerator
{
    /// <summary>
    /// Creates a new individual SAT solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    SatSolution CreateNewIndividual();
    /// <summary>
    /// Initializes a population of the specified size.
    /// </summary>
    /// <param name="populationSize">The number of individuals to generate.</param>
    /// <returns>A list of SatSolution instances representing the population.</returns>
    List<SatSolution> InitializePopulation(int populationSize);
    /// <summary>
    /// Optionally overrides the solution with fixed values or constraints.
    /// </summary>
    /// <param name="solution">The solution to override.</param>
```

```
    void OverrideSolution(SatSolution solution);
}
```

ISelectionOperator.cs

```
/// <summary>
/// Interface for selection operators that select individuals from a population for
↪  reproduction.
/// </summary>
public interface ISelectionOperator<T> where T : class
{
    /// <summary>
    /// Selects a subset of individuals from the population for the next generation.
    /// </summary>
    /// <param name="population">The current population.</param>
    /// <param name="selectionSize">The number of individuals to select.</param>
    /// <returns>A list of selected individuals.</returns>
    List<T> Select(List<T> population, int selectionSize);
}
```

B.5 Fitness

MaxSatFitness.cs

```
/// <summary>
/// Fitness function that scores SAT solutions by the number of satisfied clauses.
/// </summary>
public class MaxSatFitness : IFitnessFunction<SatSolution>
{
    /// <summary>
    /// Calculates the fitness score as the number of satisfied clauses in the solution.
    /// </summary>
    /// <param name="individual">The SAT solution to evaluate.</param>
    /// <returns>The number of satisfied clauses.</returns>
    public double Calculate(SatSolution individual)
    {
        return individual.SatisfiedClausesCount();
    }
}
```

ProbabilityAmplificationFitness.cs

```
/// <summary>
/// Fitness function that amplifies differences in SAT solution quality using a power
↪  function.
/// </summary>
public class ProbabilityAmplificationFitness : IFitnessFunction<SatSolution>
{
    private readonly double _amplificationFactor;

    /// <summary>
    /// Initializes a new instance of the ProbabilityAmplificationFitness class.
    /// </summary>
    /// <param name="amplificationFactor">The exponent used to amplify fitness
    ↪  differences.</param>
    public ProbabilityAmplificationFitness(double amplificationFactor = 2.0)
    {
        _amplificationFactor = amplificationFactor;
    }

    /// <summary>
```

```
    /// Calculates the amplified fitness score for a SAT solution.
    /// </summary>
    /// <param name="individual">The SAT solution to evaluate.</param>
    /// <returns>The amplified fitness score (between 0 and 1).</returns>
    public double Calculate(SatSolution individual)
    {
        var satisfied = individual.SatisfiedClausesCount();
        var total = individual.Instance.Clauses.Count;

        // Amplify differences in fitness
        return Math.Pow((double)satisfied / total, _amplificationFactor);
    }
}
```

WeightedMaxSatFitness.cs

```
/// <summary>
/// Fitness function that weights clauses based on difficulty and variable frequency for SAT
↪  solutions.
/// </summary>
public class WeightedMaxSatFitness : IFitnessFunction<SatSolution>
{
    /// <summary>
    /// Initializes a new instance of the WeightedMaxSatFitness class for a given SAT
    ↪  instance.
    /// </summary>
    /// <param name="instance">The SAT instance to use for clause weighting.</param>
    public WeightedMaxSatFitness(SatInstance instance)
    {
        _clauseWeights = CalculateClauseWeights(instance);
    }

    private readonly double[] _clauseWeights;

    /// <summary>
    /// Calculates normalized weights for each clause based on length, variable frequency, and
    ↪  literal distribution.
    /// </summary>
    /// <param name="instance">The SAT instance to analyze.</param>
    /// <returns>Array of normalized clause weights.</returns>
    private double[] CalculateClauseWeights(SatInstance instance)
    {
        var weights = new double[instance.Clauses.Count];
        var variableFrequency = CalculateVariableFrequency(instance);

        for (int i = 0; i < instance.Clauses.Count; i++)
        {
            var clause = instance.Clauses[i];

            // Base weight on clause length (more literals = harder to satisfy)
            double weight = 1.0 + (1.0 / clause.Literals.Count);

            // Adjust weight based on variable frequency
            foreach (var literal in clause.Literals)
            {
                var variable = Math.Abs(literal);
                // Variables that appear less frequently are harder to satisfy
                weight += 1.0 / (variableFrequency[variable] + 1);
            }

            // Adjust weight based on literal distribution
            int positiveCount = clause.Literals.Count(l => l > 0);
            int negativeCount = clause.Literals.Count - positiveCount;
```

```
            // Clauses with imbalanced positive/negative literals are harder
            weight += Math.Abs(positiveCount - negativeCount) * 0.5;

            weights[i] = weight;
        }

        // Normalize weights to sum to 1
        double sum = weights.Sum();
        for (int i = 0; i < weights.Length; i++)
        {
            weights[i] /= sum;
        }

        return weights;
    }

    /// <summary>
    /// Calculates the frequency of each variable in the SAT instance.
    /// </summary>
    /// <param name="instance">The SAT instance to analyze.</param>
    /// <returns>Dictionary mapping variable index to frequency.</returns>
    private Dictionary<int, int> CalculateVariableFrequency(SatInstance instance)
    {
        var frequency = new Dictionary<int, int>();

        foreach (var clause in instance.Clauses)
        {
            foreach (var literal in clause.Literals)
            {
                var variable = Math.Abs(literal);
                frequency.TryAdd(variable, 0);
                frequency[variable]++;
            }
        }

        return frequency;
    }

    /// <summary>
    /// Calculates the weighted fitness score for a SAT solution.
    /// </summary>
    /// <param name="individual">The SAT solution to evaluate.</param>
    /// <returns>The weighted fitness score.</returns>
    public double Calculate(SatSolution individual)
    {
        double score = 0;
        var clauses = individual.Instance.Clauses;

        for (int i = 0; i < clauses.Count; i++)
        {
            if (clauses[i].IsSatisfied(individual.Assignment))
            {
                score += _clauseWeights[i];
            }
        }

        return score;
    }
}
```

B.6 Local Search

HillClimbing.cs

```
/// <summary>
/// Implements the Hill Climbing local search algorithm for improving SAT solutions.
/// </summary>
public class HillClimbing : ILocalSearch<SatSolution>
{
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the HillClimbing class.
    /// </summary>
    /// <param name="random">Random number generator.</param>
    public HillClimbing(Random random)
    {
        _random = random;
    }

    /// <summary>
    /// Improves a SAT solution using hill climbing for a given number of iterations.
    /// </summary>
    /// <param name="individual">The SAT solution to improve.</param>
    /// <param name="maxIterations">Maximum number of iterations to perform.</param>
    public void Improve(SatSolution individual, int maxIterations)
    {
        var currentFitness = individual.SatisfiedClausesCount();

        for (int iter = 0; iter < maxIterations; iter++)
        {
            // Randomly select a variable to flip
            var flipIndex = _random.Next(individual.Assignment.Length);
            individual.Assignment[flipIndex] = !individual.Assignment[flipIndex];

            if (individual.IsSolution) return;

            var newFitness = individual.SatisfiedClausesCount();

            // Revert if not improving
            if (newFitness <= currentFitness)
            {
                individual.Assignment[flipIndex] = !individual.Assignment[flipIndex];
            }
            else
            {
                currentFitness = newFitness;
            }
        }
    }
}
```

SimulatedAnnealing.cs

```
/// <summary>
/// Implements the Simulated Annealing local search algorithm for improving SAT solutions.
/// </summary>
public class SimulatedAnnealing : ILocalSearch<SatSolution>
{
    private readonly Random _random;
    private readonly double _initialTemperature;
    private readonly double _coolingRate;
```

```
    /// <summary>
    /// Initializes a new instance of the SimulatedAnnealing class.
    /// </summary>
    /// <param name="random">Random number generator.</param>
    /// <param name="initialTemperature">Initial temperature for the annealing
    ↪  process.</param>
    /// <param name="coolingRate">Rate at which temperature decreases.</param>
    public SimulatedAnnealing(Random random, double initialTemperature = 1000.0, double
    ↪  coolingRate = 0.99)
    {
        _random = random;
        _initialTemperature = initialTemperature;
        _coolingRate = coolingRate;
    }

    /// <summary>
    /// Improves a SAT solution using simulated annealing for a given number of iterations.
    /// </summary>
    /// <param name="individual">The SAT solution to improve.</param>
    /// <param name="maxIterations">Maximum number of iterations to perform.</param>
    public void Improve(SatSolution individual, int maxIterations)
    {
        double currentTemperature = _initialTemperature;
        var currentFitness = individual.SatisfiedClausesCount();

        for (int iter = 0; iter < maxIterations; iter++)
        {
            if (currentTemperature < 1e-6) break; // Stop if temperature is too low

            // Generate a neighbor by flipping a random variable
            var flipIndex = _random.Next(individual.Assignment.Length);
            var newAssignment = (bool[])individual.Assignment.Clone();
            newAssignment[flipIndex] = !newAssignment[flipIndex];

            var newFitness = individual.Instance.EvaluateFitness(newAssignment);

            if (newFitness > currentFitness)
            {
                individual.Assignment = newAssignment;
                currentFitness = newFitness;
            }
            else
            {
                // Accept worse solution with probability based on temperature
                var acceptanceProbability = Math.Exp((newFitness - currentFitness) /
                ↪  currentTemperature);
                if (_random.NextDouble() < acceptanceProbability)
                {
                    individual.Assignment = newAssignment;
                    currentFitness = newFitness;
                }
            }

            currentTemperature *= _coolingRate;
        }
    }
}
```

TabuSearch.cs

```
/// <summary>
/// Implements the Tabu Search local search algorithm for improving SAT solutions.
/// </summary>
```

```
public class TabuSearch : ILocalSearch<SatSolution>
{
    private readonly Random _random;
    private readonly int _tabuTenure;
    private readonly List<Tuple<int, bool>> _tabuList;

    /// <summary>
    /// Initializes a new instance of the TabuSearch class.
    /// </summary>
    /// <param name="random">Random number generator.</param>
    /// <param name="tabuTenure">Number of generations to keep a move in the tabu
    ↪  list.</param>
    public TabuSearch(Random random, int tabuTenure = 5)
    {
        _random = random;
        _tabuTenure = tabuTenure;
        _tabuList = new List<Tuple<int, bool>>();
    }

    /// <summary>
    /// Improves a SAT solution using tabu search for a given number of iterations.
    /// </summary>
    /// <param name="individual">The SAT solution to improve.</param>
    /// <param name="maxIterations">Maximum number of iterations to perform.</param>
    public void Improve(SatSolution individual, int maxIterations)
    {
        var currentFitness = individual.SatisfiedClausesCount();
        var bestSolution = (SatSolution)individual.Clone();
        var bestFitness = currentFitness;

        for (int iter = 0; iter < maxIterations; iter++)
        {
            var neighbors = GenerateNeighbors(individual);
            var bestNeighbor = neighbors.OrderByDescending(n =>
            ↪  n.SatisfiedClausesCount()).First();

            if (bestNeighbor.SatisfiedClausesCount() > currentFitness)
            {
                individual.Assignment = bestNeighbor.Assignment;
                currentFitness = bestNeighbor.SatisfiedClausesCount();
                if (currentFitness > bestFitness)
                {
                    bestSolution = (SatSolution)individual.Clone();
                    bestFitness = currentFitness;
                }
                _tabuList.Add(new Tuple<int, bool>(iter, individual.Assignment[0])); // Add to
                ↪  tabu list
                if (_tabuList.Count > _tabuTenure)
                {
                    _tabuList.RemoveAt(0);
                }
            }
            else
            {
                // Accept worse solution if it's not tabu or has a better fitness
                if (!_tabuList.Any(t => t.Item1 == iter - _tabuTenure && t.Item2 ==
                ↪  individual.Assignment[0]))
                {
                    individual.Assignment = bestNeighbor.Assignment;
                    currentFitness = bestNeighbor.SatisfiedClausesCount();
                    if (currentFitness > bestFitness)
                    {
                        bestSolution = (SatSolution)individual.Clone();
```

```
                    bestFitness = currentFitness;
                }
                _tabuList.Add(new Tuple<int, bool>(iter, individual.Assignment[0])); //
                ↪  Add to tabu list
                if (_tabuList.Count > _tabuTenure)
                {
                    _tabuList.RemoveAt(0);
                }
            }
        }
    }
}

/// <summary>
/// Generates a list of neighboring solutions by flipping a single variable.
/// </summary>
/// <param name="individual">The current SAT solution.</param>
/// <returns>A list of neighboring solutions.</returns>
private List<SatSolution> GenerateNeighbors(SatSolution individual)
{
    var neighbors = new List<SatSolution>();
    for (int i = 0; i < individual.Assignment.Length; i++)
    {
        var neighbor = (SatSolution)individual.Clone();
        neighbor.Assignment[i] = !neighbor.Assignment[i];
        neighbors.Add(neighbor);
    }
    return neighbors;
}
}
```

B.7 Models

SatClause.cs

```
/// <summary>
/// Represents a single clause in a SAT problem.
/// </summary>
public class SatClause
{
    /// <summary>
    /// Initializes a new instance of the SatClause class.
    /// </summary>
    /// <param name="literals">The literals in the clause.</param>
    public SatClause(IEnumerable<int> literals)
    {
        Literals = new List<int>(literals);
    }

    /// <summary>
    /// The literals in the clause.
    /// </summary>
    public List<int> Literals { get; }

    /// <summary>
    /// Checks if the given assignment satisfies the clause.
    /// </summary>
    /// <param name="assignment">The assignment to check.</param>
    /// <returns>True if the clause is satisfied, false otherwise.</returns>
    public bool IsSatisfied(bool[] assignment)
    {
        return Literals.Any(literal =>
            (literal > 0 && assignment[Math.Abs(literal) - 1]) ||
```

```
            (literal < 0 && !assignment[Math.Abs(literal) - 1]));
    }
}
```

SatInstance.cs

```
/// <summary>
/// Represents the entire SAT problem instance.
/// </summary>
public class SatInstance
{
    /// <summary>
    /// Initializes a new instance of the SatInstance class.
    /// </summary>
    /// <param name="variableCount">The number of variables in the instance.</param>
    /// <param name="clauses">The clauses of the SAT instance.</param>
    public SatInstance(int variableCount, IEnumerable<SatClause> clauses)
    {
        VariableCount = variableCount;
        Clauses = new List<SatClause>(clauses);
    }

    /// <summary>
    /// The number of variables in the SAT instance.
    /// </summary>
    public int VariableCount { get; }

    /// <summary>
    /// The clauses of the SAT instance.
    /// </summary>
    public List<SatClause> Clauses { get; }

    /// <summary>
    /// Evaluates the fitness of a given assignment.
    /// </summary>
    /// <param name="assignment">The assignment to evaluate.</param>
    /// <returns>The fitness score.</returns>
    public double EvaluateFitness(bool[] assignment)
    {
        return Clauses.Sum(clause => clause.IsSatisfied(assignment) ? 1.0 : 0.0);
    }
}
```

SatSolution.cs

```
/// <summary>
/// Represents a potential solution to a SAT problem.
/// </summary>
public class SatSolution : ICloneable
{
    /// <summary>
    /// Initializes a new instance of the SatSolution class.
    /// </summary>
    /// <param name="instance">The SAT instance this solution belongs to.</param>
    /// <param name="assignment">The assignment of variables (true for positive, false for
    ↪  negative).</param>
    /// <param name="generations">Number of generations this solution has been part
    ↪  of.</param>
    /// <param name="restarts">Number of times this solution has been restarted.</param>
    public SatSolution(SatInstance instance, bool[] assignment, int generations = 0, int
    ↪  restarts = 0)
    {
```

```
        Instance = instance;
        Assignment = assignment;
        Generations = generations;
        Restarts = restarts;
    }

    /// <summary>
    /// The SAT instance this solution belongs to.
    /// </summary>
    public SatInstance Instance { get; }

    /// <summary>
    /// The assignment of variables (true for positive, false for negative).
    /// </summary>
    public bool[] Assignment { get; set; }

    /// <summary>
    /// Number of generations this solution has been part of.
    /// </summary>
    public int Generations { get; set; }

    /// <summary>
    /// Number of times this solution has been restarted.
    /// </summary>
    public int Restarts { get; set; }

    /// <summary>
    /// Checks if the current assignment satisfies all clauses.
    /// </summary>
    /// <returns>True if the solution is a valid SAT assignment, false otherwise.</returns>
    public bool IsSolution => Instance.EvaluateFitness(Assignment) == Instance.Clauses.Count;

    /// <summary>
    /// Counts the number of satisfied clauses in the current assignment.
    /// </summary>
    /// <returns>The number of satisfied clauses.</returns>
    public int SatisfiedClausesCount()
    {
        return Instance.EvaluateFitness(Assignment);
    }

    /// <summary>
    /// Creates a deep copy of the current solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    public object Clone()
    {
        return new SatSolution(Instance, (bool[])Assignment.Clone(), Generations, Restarts);
    }

    /// <summary>
    /// Converts the solution to a printable string format.
    /// </summary>
    /// <returns>A string representation of the assignment.</returns>
    public string ToPrintable()
    {
        return string.Join(" ", Assignment.Select(b => b ? "1" : "0"));
    }
}
```

B.8 Operators

B.8.1 CrossOver

ClauseSatisfactionCrossover.cs

```
/// <summary>
/// Crossover operator that combines two parent solutions by selecting literals from one
↪  parent
/// based on the satisfaction of clauses in the other parent.
/// </summary>
public class ClauseSatisfactionCrossover : ICrossoverOperator<SatSolution>
{
    /// <summary>
    /// Performs crossover between two parent solutions to produce offspring.
    /// </summary>
    /// <param name="parent1">The first parent solution.</param>
    /// <param name="parent2">The second parent solution.</param>
    /// <returns>An enumerable of offspring solutions.</returns>
    public IEnumerable<SatSolution> Crossover(SatSolution parent1, SatSolution parent2)
    {
        var offspring1 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());
        var offspring2 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());

        var random = new Random();
        var crossoverPoint = random.Next(parent1.Assignment.Length);

        for (int i = 0; i < crossoverPoint; i++)
        {
            offspring1.Assignment[i] = parent1.Assignment[i];
            offspring2.Assignment[i] = parent2.Assignment[i];
        }

        for (int i = crossoverPoint; i < parent1.Assignment.Length; i++)
        {
            offspring1.Assignment[i] = parent2.Assignment[i];
            offspring2.Assignment[i] = parent1.Assignment[i];
        }

        return new[] { offspring1, offspring2 };
    }
}
```

NPointCrossover.cs

```
/// <summary>
/// Crossover operator that combines two parent solutions using N-point crossover.
/// </summary>
public class NPointCrossover : ICrossoverOperator<SatSolution>
{
    /// <summary>
    /// Performs crossover between two parent solutions to produce offspring.
    /// </summary>
    /// <param name="parent1">The first parent solution.</param>
    /// <param name="parent2">The second parent solution.</param>
    /// <returns>An enumerable of offspring solutions.</returns>
    public IEnumerable<SatSolution> Crossover(SatSolution parent1, SatSolution parent2)
    {
        var offspring1 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());
        var offspring2 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());
```

```
        var random = new Random();
        var n = random.Next(1, parent1.Assignment.Length); // Ensure at least one crossover
        ↪  point
        var crossoverPoints = new List<int>();

        for (int i = 0; i < n; i++)
        {
            int point;
            do
            {
                point = random.Next(parent1.Assignment.Length);
            } while (crossoverPoints.Contains(point));
            crossoverPoints.Add(point);
        }

        crossoverPoints.Sort();

        int currentStart = 0;
        foreach (var point in crossoverPoints)
        {
            for (int i = currentStart; i < point; i++)
            {
                offspring1.Assignment[i] = parent2.Assignment[i];
                offspring2.Assignment[i] = parent1.Assignment[i];
            }
            currentStart = point;
        }

        // Handle remaining part of the parents
        for (int i = currentStart; i < parent1.Assignment.Length; i++)
        {
            offspring1.Assignment[i] = parent2.Assignment[i];
            offspring2.Assignment[i] = parent1.Assignment[i];
        }

        return new[] { offspring1, offspring2 };
    }
}
```

UniformCrossOver.cs

```
/// <summary>
/// Crossover operator that combines two parent solutions using uniform crossover.
/// </summary>
public class UniformCrossOver : ICrossoverOperator<SatSolution>
{
    /// <summary>
    /// Performs crossover between two parent solutions to produce offspring.
    /// </summary>
    /// <param name="parent1">The first parent solution.</param>
    /// <param name="parent2">The second parent solution.</param>
    /// <returns>An enumerable of offspring solutions.</returns>
    public IEnumerable<SatSolution> Crossover(SatSolution parent1, SatSolution parent2)
    {
        var offspring1 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());
        var offspring2 = new SatSolution(parent1.Instance,
        ↪  (bool[])parent1.Assignment.Clone());

        var random = new Random();
        for (int i = 0; i < parent1.Assignment.Length; i++)
        {
```

```
            if (random.NextDouble() < 0.5)
            {
                offspring1.Assignment[i] = parent1.Assignment[i];
                offspring2.Assignment[i] = parent2.Assignment[i];
            }
            else
            {
                offspring1.Assignment[i] = parent2.Assignment[i];
                offspring2.Assignment[i] = parent1.Assignment[i];
            }
        }

        return new[] { offspring1, offspring2 };
    }
}
```

B.8.2 Mutation

BitFlipMutation.cs

```
/// <summary>
/// Mutation operator that flips a single bit in the solution.
/// </summary>
public class BitFlipMutation : IMutationOperator<SatSolution>
{
    /// <summary>
    /// Mutates a solution with a given mutation rate.
    /// </summary>
    /// <param name="individual">The solution to mutate.</param>
    /// <param name="mutationRate">The probability of mutation for each gene.</param>
    public void Mutate(SatSolution individual, double mutationRate)
    {
        var random = new Random();
        for (int i = 0; i < individual.Assignment.Length; i++)
        {
            if (random.NextDouble() < mutationRate)
            {
                individual.Assignment[i] = !individual.Assignment[i];
            }
        }
    }
}
```

GuidedMutation.cs

```
/// <summary>
/// Mutation operator that flips variables based on their frequency in the instance.
/// </summary>
public class GuidedMutation : IMutationOperator<SatSolution>
{
    private readonly SatInstance _instance;
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the GuidedMutation class.
    /// </summary>
    /// <param name="instance">The SAT instance to guide mutation.</param>
    /// <param name="random">Random number generator.</param>
    public GuidedMutation(SatInstance instance, Random random)
    {
        _instance = instance;
        _random = random;
```

```
    }

    /// <summary>
    /// Mutates a solution with a given mutation rate.
    /// </summary>
    /// <param name="individual">The solution to mutate.</param>
    /// <param name="mutationRate">The probability of mutation for each gene.</param>
    public void Mutate(SatSolution individual, double mutationRate)
    {
        var variableFrequency = CalculateVariableFrequency(_instance);
        var totalVariables = _instance.VariableCount;

        for (int i = 0; i < individual.Assignment.Length; i++)
        {
            if (random.NextDouble() < mutationRate)
            {
                var variable = i + 1; // Assuming variable index 1-based
                var flipProbability = (double)variableFrequency[variable] / totalVariables;
                if (_random.NextDouble() < flipProbability)
                {
                    individual.Assignment[i] = !individual.Assignment[i];
                }
            }
        }
    }

    /// <summary>
    /// Calculates the frequency of each variable in the SAT instance.
    /// </summary>
    /// <param name="instance">The SAT instance to analyze.</param>
    /// <returns>Dictionary mapping variable index to frequency.</returns>
    private Dictionary<int, int> CalculateVariableFrequency(SatInstance instance)
    {
        var frequency = new Dictionary<int, int>();

        foreach (var clause in instance.Clauses)
        {
            foreach (var literal in clause.Literals)
            {
                var variable = Math.Abs(literal);
                frequency.TryAdd(variable, 0);
                frequency[variable]++;
            }
        }

        return frequency;
    }
}
```

NBitMutation.cs

```
/// <summary>
/// Mutation operator that flips N bits in the solution.
/// </summary>
public class NBitMutation : IMutationOperator<SatSolution>
{
    private readonly int _mutationBits;
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the NBitMutation class.
    /// </summary>
    /// <param name="mutationBits">Number of bits to mutate.</param>
```

```
    /// <param name="random">Random number generator.</param>
    public NBitMutation(int mutationBits, Random random)
    {
        _mutationBits = mutationBits;
        _random = random;
    }

    /// <summary>
    /// Mutates a solution with a given mutation rate.
    /// </summary>
    /// <param name="individual">The solution to mutate.</param>
    /// <param name="mutationRate">The probability of mutation for each gene.</param>
    public void Mutate(SatSolution individual, double mutationRate)
    {
        var bitsToFlip = new List<int>();
        for (int i = 0; i < individual.Assignment.Length; i++)
        {
            if (_random.NextDouble() < mutationRate)
            {
                bitsToFlip.Add(i);
            }
        }

        var indicesToFlip = bitsToFlip.Take(_mutationBits).ToList();
        foreach (var index in indicesToFlip)
        {
            individual.Assignment[index] = !individual.Assignment[index];
        }
    }
}
```

B.8.3 Selection

RankSelection.cs

```
/// <summary>
/// Selection operator that selects individuals based on their rank in the population.
/// </summary>
public class RankSelection : ISelectionOperator<SatSolution>
{
    /// <summary>
    /// Selects a subset of individuals from the population for the next generation.
    /// </summary>
    /// <param name="population">The current population.</param>
    /// <param name="selectionSize">The number of individuals to select.</param>
    /// <returns>A list of selected individuals.</returns>
    public List<SatSolution> Select(List<SatSolution> population, int selectionSize)
    {
        var fitnessScores = population.Select(ind => ind.SatisfiedClausesCount()).ToList();
        var minFitness = fitnessScores.Min();
        var maxFitness = fitnessScores.Max();

        // Normalize fitness scores to a range [0, 1]
        var normalizedScores = fitnessScores.Select(f => (f - minFitness) / (maxFitness -
        ↪  minFitness)).ToList();

        // Calculate cumulative probabilities
        var cumulativeProbabilities = new List<double>();
        double sum = 0;
        foreach (var prob in normalizedScores)
        {
            sum += prob;
            cumulativeProbabilities.Add(sum);
```

```
        }

        var selected = new List<SatSolution>();
        for (int i = 0; i < selectionSize; i++)
        {
            var random = new Random();
            var pick = random.NextDouble();

            int index = 0;
            while (index < cumulativeProbabilities.Count && pick >
            ↪  cumulativeProbabilities[index])
            {
                index++;
            }
            selected.Add(population[index]);
        }

        return selected;
    }
}
```

RouletteWheelSelection.cs

```
/// <summary>
/// Selection operator that selects individuals based on their fitness using a roulette wheel.
/// </summary>
public class RouletteWheelSelection : ISelectionOperator<SatSolution>
{
    /// <summary>
    /// Selects a subset of individuals from the population for the next generation.
    /// </summary>
    /// <param name="population">The current population.</param>
    /// <param name="selectionSize">The number of individuals to select.</param>
    /// <returns>A list of selected individuals.</returns>
    public List<SatSolution> Select(List<SatSolution> population, int selectionSize)
    {
        var fitnessScores = population.Select(ind => ind.SatisfiedClausesCount()).ToList();
        var minFitness = fitnessScores.Min();
        var maxFitness = fitnessScores.Max();

        // Normalize fitness scores to a range [0, 1]
        var normalizedScores = fitnessScores.Select(f => (f - minFitness) / (maxFitness -
        ↪  minFitness)).ToList();

        // Calculate cumulative probabilities
        var cumulativeProbabilities = new List<double>();
        double sum = 0;
        foreach (var prob in normalizedScores)
        {
            sum += prob;
            cumulativeProbabilities.Add(sum);
        }

        var selected = new List<SatSolution>();
        for (int i = 0; i < selectionSize; i++)
        {
            var random = new Random();
            var pick = random.NextDouble();

            int index = 0;
            while (index < cumulativeProbabilities.Count && pick >
            ↪  cumulativeProbabilities[index])
            {
```

```
                index++;
            }
            selected.Add(population[index]);
        }

        return selected;
    }
}
```

TournamentSelection.cs

```
/// <summary>
/// Selection operator that selects individuals based on a tournament.
/// </summary>
public class TournamentSelection : ISelectionOperator<SatSolution>
{
    private readonly int _tournamentSize;
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the TournamentSelection class.
    /// </summary>
    /// <param name="tournamentSize">Size of the tournament.</param>
    /// <param name="random">Random number generator.</param>
    public TournamentSelection(int tournamentSize, Random random)
    {
        _tournamentSize = tournamentSize;
        _random = random;
    }

    /// <summary>
    /// Selects a subset of individuals from the population for the next generation.
    /// </summary>
    /// <param name="population">The current population.</param>
    /// <param name="selectionSize">The number of individuals to select.</param>
    /// <returns>A list of selected individuals.</returns>
    public List<SatSolution> Select(List<SatSolution> population, int selectionSize)
    {
        var selected = new List<SatSolution>();
        for (int i = 0; i < selectionSize; i++)
        {
            var tournamentParticipants = new List<SatSolution>();
            for (int j = 0; j < _tournamentSize; j++)
            {
                tournamentParticipants.Add(population[_random.Next(population.Count)]);
            }
            selected.Add(tournamentParticipants.OrderByDescending(ind =>
            ↪  ind.SatisfiedClausesCount()).First());
        }
        return selected;
    }
}
```

B.9 Population

ClauseProbabilityPopulationGenerator.cs

```
/// <summary>
/// Population generator that creates individuals based on clause satisfaction probabilities.
/// </summary>
public class ClauseProbabilityPopulationGenerator : IPopulationGenerator
{
```

```
    private readonly SatInstance _instance;
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the ClauseProbabilityPopulationGenerator class.
    /// </summary>
    /// <param name="instance">The SAT instance to generate individuals for.</param>
    /// <param name="random">Random number generator.</param>
    public ClauseProbabilityPopulationGenerator(SatInstance instance, Random random)
    {
        _instance = instance;
        _random = random;
    }

    /// <summary>
    /// Creates a new individual SAT solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    public SatSolution CreateNewIndividual()
    {
        var assignment = new bool[_instance.VariableCount];
        for (int i = 0; i < _instance.VariableCount; i++)
        {
            assignment[i] = _random.NextDouble() < 0.5; // Assign true or false randomly
        }
        return new SatSolution(_instance, assignment);
    }

    /// <summary>
    /// Initializes a population of the specified size.
    /// </summary>
    /// <param name="populationSize">The number of individuals to generate.</param>
    /// <returns>A list of SatSolution instances representing the population.</returns>
    public List<SatSolution> InitializePopulation(int populationSize)
    {
        var population = new List<SatSolution>();
        for (int i = 0; i < populationSize; i++)
        {
            population.Add(CreateNewIndividual());
        }
        return population;
    }

    /// <summary>
    /// Optionally overrides the solution with fixed values or constraints.
    /// </summary>
    /// <param name="solution">The solution to override.</param>
    public void OverrideSolution(SatSolution solution)
    {
        // This method is not directly used in the genetic algorithm flow,
        // but can be implemented if specific constraints need to be applied
        // before the GA starts.
    }
}
```

DiversityBasedPopulationGenerator.cs

```
/// <summary>
/// Population generator that creates individuals based on diversity and fitness.
/// </summary>
public class DiversityBasedPopulationGenerator : IPopulationGenerator
{
    private readonly SatInstance _instance;
```

```
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the DiversityBasedPopulationGenerator class.
    /// </summary>
    /// <param name="instance">The SAT instance to generate individuals for.</param>
    /// <param name="random">Random number generator.</param>
    public DiversityBasedPopulationGenerator(SatInstance instance, Random random)
    {
        _instance = instance;
        _random = random;
    }

    /// <summary>
    /// Creates a new individual SAT solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    public SatSolution CreateNewIndividual()
    {
        var assignment = new bool[_instance.VariableCount];
        for (int i = 0; i < _instance.VariableCount; i++)
        {
            assignment[i] = _random.NextDouble() < 0.5; // Assign true or false randomly
        }
        return new SatSolution(_instance, assignment);
    }

    /// <summary>
    /// Initializes a population of the specified size.
    /// </summary>
    /// <param name="populationSize">The number of individuals to generate.</param>
    /// <returns>A list of SatSolution instances representing the population.</returns>
    public List<SatSolution> InitializePopulation(int populationSize)
    {
        var population = new List<SatSolution>();
        for (int i = 0; i < populationSize; i++)
        {
            population.Add(CreateNewIndividual());
        }
        return population;
    }

    /// <summary>
    /// Optionally overrides the solution with fixed values or constraints.
    /// </summary>
    /// <param name="solution">The solution to override.</param>
    public void OverrideSolution(SatSolution solution)
    {
        // This method is not directly used in the genetic algorithm flow,
        // but can be implemented if specific constraints need to be applied
        // before the GA starts.
    }
}
```

PopulationGenerator.cs

```
/// <summary>
/// Population generator that creates individuals based on a specified type.
/// </summary>
public class PopulationGenerator : IPopulationGenerator
{
    private readonly Random _random;
    private readonly SatInstance _instance;
```

```
    private readonly string _generatorType;

    /// <summary>
    /// Initializes a new instance of the PopulationGenerator class.
    /// </summary>
    /// <param name="random">Random number generator.</param>
    /// <param name="instance">The SAT instance to generate individuals for.</param>
    /// <param name="generatorType">Type of population generator to use (e.g., "Clause",
    ↪  "Random", "Diversity").</param>
    public PopulationGenerator(Random random, SatInstance instance, string generatorType)
    {
        _random = random;
        _instance = instance;
        _generatorType = generatorType;
    }

    /// <summary>
    /// Creates a new individual SAT solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    public SatSolution CreateNewIndividual()
    {
        switch (_generatorType)
        {
            case "Clause":
                return new ClauseProbabilityPopulationGenerator(_instance,
                ↪  _random).CreateNewIndividual();
            case "Random":
                return new RandomPopulationGenerator(_instance,
                ↪  _random).CreateNewIndividual();
            case "Diversity":
                return new DiversityBasedPopulationGenerator(_instance,
                ↪  _random).CreateNewIndividual();
            default:
                throw new ArgumentException($"Unknown generator type: {_generatorType}");
        }
    }

    /// <summary>
    /// Initializes a population of the specified size.
    /// </summary>
    /// <param name="populationSize">The number of individuals to generate.</param>
    /// <returns>A list of SatSolution instances representing the population.</returns>
    public List<SatSolution> InitializePopulation(int populationSize)
    {
        var population = new List<SatSolution>();
        for (int i = 0; i < populationSize; i++)
        {
            population.Add(CreateNewIndividual());
        }
        return population;
    }

    /// <summary>
    /// Optionally overrides the solution with fixed values or constraints.
    /// </summary>
    /// <param name="solution">The solution to override.</param>
    public void OverrideSolution(SatSolution solution)
    {
        // This method is not directly used in the genetic algorithm flow,
        // but can be implemented if specific constraints need to be applied
        // before the GA starts.
    }
```

```
}
```

RandomPopulationGenerator.cs

```
/// <summary>
/// Population generator that creates individuals by assigning random values to variables.
/// </summary>
public class RandomPopulationGenerator : IPopulationGenerator
{
    private readonly SatInstance _instance;
    private readonly Random _random;

    /// <summary>
    /// Initializes a new instance of the RandomPopulationGenerator class.
    /// </summary>
    /// <param name="instance">The SAT instance to generate individuals for.</param>
    /// <param name="random">Random number generator.</param>
    public RandomPopulationGenerator(SatInstance instance, Random random)
    {
        _instance = instance;
        _random = random;
    }

    /// <summary>
    /// Creates a new individual SAT solution.
    /// </summary>
    /// <returns>A new SatSolution instance.</returns>
    public SatSolution CreateNewIndividual()
    {
        var assignment = new bool[_instance.VariableCount];
        for (int i = 0; i < _instance.VariableCount; i++)
        {
            assignment[i] = _random.NextDouble() < 0.5; // Assign true or false randomly
        }
        return new SatSolution(_instance, assignment);
    }

    /// <summary>
    /// Initializes a population of the specified size.
    /// </summary>
    /// <param name="populationSize">The number of individuals to generate.</param>
    /// <returns>A list of SatSolution instances representing the population.</returns>
    public List<SatSolution> InitializePopulation(int populationSize)
    {
        var population = new List<SatSolution>();
        for (int i = 0; i < populationSize; i++)
        {
            population.Add(CreateNewIndividual());
        }
        return population;
    }

    /// <summary>
    /// Optionally overrides the solution with fixed values or constraints.
    /// </summary>
    /// <param name="solution">The solution to override.</param>
    public void OverrideSolution(SatSolution solution)
    {
        // This method is not directly used in the genetic algorithm flow,
        // but can be implemented if specific constraints need to be applied
        // before the GA starts.
    }
```

```
}
```

B.10 Utils

DimacsParser.cs

```
/// <summary>
/// Parses a DIMACS CNF file into a SatInstance object.
/// </summary>
public class DimacsParser
{
    /// <summary>
    /// Parses the content of a DIMACS CNF file.
    /// </summary>
    /// <param name="dimacsContent">The content of the DIMACS file.</param>
    /// <returns>A SatInstance object representing the parsed SAT instance.</returns>
    public SatInstance Parse(string dimacsContent)
    {
        var lines = dimacsContent.Split(new[] { '\r', '\n' },
        ↪  StringSplitOptions.RemoveEmptyEntries);
        var variableCount = 0;
        var clauseCount = 0;
        var clauses = new List<SatClause>();

        foreach (var line in lines)
        {
            if (line.StartsWith("c") || line.StartsWith("p")) continue; // Skip comments and
            ↪  problem line

            var literals = new List<int>();
            foreach (var literal in line.Split(new[] { ' ' },
            ↪  StringSplitOptions.RemoveEmptyEntries))
            {
                if (literal == "0") break; // End of clause
                literals.Add(int.Parse(literal));
            }
            if (literals.Count > 0)
            {
                clauses.Add(new SatClause(literals));
                clauseCount++;
            }
        }

        // The problem line should contain the number of variables and clauses
        var problemLine = lines.FirstOrDefault(l => l.StartsWith("p"));
        if (problemLine == null)
        {
            throw new InvalidDataException("Problem line not found in DIMACS file.");
        }

        var parts = problemLine.Split(new[] { ' ' }, StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length < 4)
        {
            throw new InvalidDataException("Problem line format is incorrect.");
        }

        variableCount = int.Parse(parts[2]);
        clauseCount = int.Parse(parts[3]);

        return new SatInstance(variableCount, clauses);
    }
}
```

OutputWriter.cs

```
/// <summary>
/// Writes the output of the genetic algorithm to the console or a file.
/// </summary>
public class OutputWriter
{
    /// <summary>
    /// Initializes a new instance of the OutputWriter class.
    /// </summary>
    /// <param name="hideOutput">If true, output is suppressed.</param>
    public OutputWriter(bool hideOutput = false)
    {
        HideOutput = hideOutput;
        _outputStream = Console.Out; // Default to console output
    }

    /// <summary>
    /// If true, output is suppressed.
    /// </summary>
    public bool HideOutput { get; }

    /// <summary>
    /// The path of the file being processed.
    /// </summary>
    public string FilePath { get; set; } = string.Empty;

    /// <summary>
    /// The number of variables in the current SAT instance.
    /// </summary>
    public int VariableCount { get; set; }

    /// <summary>
    /// The number of clauses in the current SAT instance.
    /// </summary>
    public int ClauseCount { get; set; }

    /// <summary>
    /// The current SAT solution being evaluated.
    /// </summary>
    public SatSolution Solution { get; set; } = null;

    /// <summary>
    /// Whether the current solution satisfies all clauses.
    /// </summary>
    public bool IsSatisfied { get; set; } = false;

    /// <summary>
    /// The stopwatch for timing the GA run.
    /// </summary>
    public Stopwatch StopWatch { get; set; } = new Stopwatch();

    /// <summary>
    /// Writes a line to the output stream.
    /// </summary>
    /// <param name="line">The line to write.</param>
    public void WriteLine(string line)
    {
        if (!HideOutput)
        {
            _outputStream.WriteLine(line);
        }
    }
```

```
    /// <summary>
    /// Writes a completion message.
    /// </summary>
    /// <param name="message">The message to write.</param>
    public void WriteCompletion(string message)
    {
        if (!HideOutput)
        {
            _outputStream.WriteLine(message);
        }
    }

    /// <summary>
    /// Resets the instance for the next file.
    /// </summary>
    public void ResetInstance()
    {
        Solution = null;
        IsSatisfied = false;
        StopWatch.Reset();
    }

    private readonly TextWriter _outputStream;
}
```

C Build and Run Instructions

Prerequisites

- .NET 8.0 SDK or later
- Any OS (Windows/Linux/macOS)

Building the Project

```
git clone https://github.com/craigiparker/SAT.GA.git
cd SAT.GA
dotnet build
```

Running the Application

```
SAT.GA <input.cnf>|<folder> [options]
```

Command Line Options

Option	Description	Default
-p, –population	Population size	30
-g, –generations	Maximum generations	20000
-m, –mutation-rate	Mutation probability	0.01
-c, –crossover-rate	Crossover probability	0.9
-e, –elitism-rate	Elitism rate	0.1
-s, –seed	Random seed	(none)
-t, –tournament	Tournament size	3
–tabu	Tabu tenure	5
-a, –amplification	Amplification factor	2.0
-l, –local-search	Enable local search	false
-fc, –file-count	Limit files processed (folder)	1000
-tc, –thread-count	Number of threads	1
–mutation-bits	Number of bits to mutate	1

–local-search-method	Local search method	Tabu
–selection	Selection operator	Tournament
–crossover	Crossover operator	1Point
–mutation	Mutation operator	Guided
–fitness	Fitness function	Weighted
–generator	Population generator	Clause
-h, –help	Show help message	

Example Usage

```
SAT.GA benchmarks_folder/uf50-218.cnf -p 50 -g 1000 -m 0.02 -tc 4
SAT.GA benchmarks_folder -p 30 -fc 100 -tc 8
```

Output

The solver outputs either:

```
SATISFIABLE
<solution as space-separated literals>
```

or

```
TIMEOUT
```

Runtime statistics are also displayed, including:

- Number of generations
- Restarts performed
- Elapsed time
- Best fitness achieved